Physlib Monthly Report

1–31 May 2026

Contributors: Alex Meiburg, Christian Krause, Daniel Morrison, Dennj, Florian Wiesner, Gregory Loges, Hannah, Joseph Tooby-Smith, juanjfndz, Matteo Cipollina, Nicola Bernini, Pietro Monticone, Pranav Magdum, wock, Zhi Kai Pong

Reviewers: Alex Meiburg, Daniel Morrison, Joseph Tooby-Smith, Rodolfo R. Soldati, wock, Zhi Kai Pong

Generated 14 August 2026 from commit `d44dd71`.

Abstract

Physlib is a community library of formalised physics for the Lean 4 proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month's additions to it.

It covers 1–31 May 2026 on branch `master` of `leanprover-community/physlib`, between commits `cd22b0c` and `d44dd71`. Over that period 15 contributors landed 60 commits, changing 26,360 lines across 186 files and adding 999 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at GitHub compare.

Contents

1 Introduction

During May 2026, 15 contributors submitted 60 commits that changed 26,360 lines (+21,442 / -4,918) across 186 files spanning 32 top-level areas of the repository. Of those, 50 files were newly added and 10 were removed outright. This report catalogues 999 newly added Lean declarations: 592 lemmas, 183 defs, 152 theorems, 43 instances, 22 abbrevs, and 7 structures. We're delighted to recognize the following first-time contributors this month: Christian Krause, Dennj, Florian Wiesner, Hannah, Pranav Magdum, and wock.

See the full compare diff and the list of commits on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Alex Meiburg	10,037	1
Christian Krause	75	1
Daniel Morrison	2	1
Dennj	2,933	9
Florian Wiesner	1,738	2
Gregory Loges	3,632	12
Hannah	54	1
Joseph Tooby-Smith	7,024	20
juanjfndz	81	1
Matteo Cipollina	782	2
Nicola Bernini	102	3
Pietro Monticone	13	1
Pranav Magdum	180	1
wock	267	4
Zhi Kai Pong	38	1

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Alex Meiburg	6
Daniel Morrison	7
Joseph Tooby-Smith	37
Rodolfo R. Soldati	2
wock	2
Zhi Kai Pong	11

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
`QuantumInfo/ForMathlib`	31	+15	-1	+9,892	-226

Physlib/Electromagnetism	24	+10	–	+2,631	-1,475
Physlib/QuantumMechanics	17	+7	–	+2,409	-718
Physlib/Relativity	16	+1	–	+904	-743
QuantumInfo/Entropy	6	+1	–	+1,592	-10
Physlib/ClassicalMechanics	6	+3	–	+1,168	-200
QuantumInfo/Channels	7	–	–	+408	-152
QuantumInfo/Finite	5	–	-5	+0	-557
QuantumInfo/States	9	+3	–	+499	-48
Physlib/FluidDynamics	4	+4	–	+534	-0
Physlib/SpaceAndTime	8	+2	–	+405	-27
Physlib/StatisticalMechanics	3	+2	–	+366	-41
Physlib/Particles	17	–	–	+174	-185
QuantumInfo/StatMech	2	–	-2	+0	-227
Physlib/Mathematics	2	+1	–	+141	-1
QuantumInfo/Capacity	2	–	–	+38	-39
QuantumInfo/InfiniteDim	1	–	-1	+0	-76
QuantumInfo	2	–	-1	+35	-39
QuantumInfo/ResourceTheory	4	–	–	+33	-41
QuantumInfo.lean	1	–	–	+36	-25
docs	3	–	–	+38	-12
Physlib/Meta	2	+1	–	+41	-2
QuantumInfo/Operators	1	–	–	+1	-38
Physlib.lean	1	–	–	+32	-0
QuantumInfo/Measurements	1	–	–	+23	-3
Physlib/QFT	2	–	–	+1	-17
Physlib/Units	1	–	–	+17	-1
QuantumInfo/ClassicalInfo	2	–	–	+9	-9
scripts/MetaPrograms	3	–	–	+11	-5
.codespellignore	1	–	–	+2	-0
.github/workflows	1	–	–	+1	-1
scripts	1	–	–	+1	-0

1.4 New declarations by kind

Kind	Count
lemma	592
def	183
theorem	152
instance	43
abbrev	22
structure	7

2 Details by file

2.1 Physlib.ClassicalMechanics.DampedHarmonicOscillator.Basic

Physlib/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean on GitHub.

def force [source]

The force of the damped harmonic oscillator at a given position and time.

```
noncomputable def force (S : DampedHarmonicOscillator)
    (xₜ : Time → EuclideanSpace ℝ (Fin 1)) (t : Time) :
    EuclideanSpace ℝ (Fin 1)
```

lemma equationOfMotion_iff_newtons_2nd_law [source]

```
lemma equationOfMotion_iff_newtons_2nd_law (xₜ : Time → EuclideanSpace ℝ (Fin 1)):
    S.EquationOfMotion xₜ ↔
    (∀ t : Time, S.m • ∂ₜ (∂ₜ xₜ) t = force S xₜ t)
```

def decayRate [source]

The exponential decay rate `γ / (2 * m)`.

```
noncomputable def decayRate : ℝ
```

def IsUndamped [source]

The system is undamped when $\gamma = 0$.

```
def IsUndamped : Prop
```

def angularFrequency [source]

The real frequency selected by the damping regime.

In the underdamped regime this is the oscillation frequency. In the critically damped regime it is `0`. In the overdamped regime this is the real split rate between the two roots.

```
noncomputable def angularFrequency : ℝ
```

lemma discriminant_eq_four_mul_m_sq_mul_decayRate_sq_sub_ω_sq [source]

The relationship between the discriminant, decay rate, and natural angular frequency.

```
lemma discriminant_eq_four_mul_m_sq_mul_decayRate_sq_sub_ω_sq :
    S.discriminant = 4 * S.m^2 * (S.decayRate^2 - S.ω^2)
```

lemma decayRate_nonneg [source]

The decay rate is nonnegative.

```
lemma decayRate_nonneg : 0 ≤ S.decayRate
```

lemma isUnderdamped_of_gamma_eq_zero [source]

An undamped oscillator lies in the underdamped regime.

```
lemma isUnderdamped_of_gamma_eq_zero (hγ : S.γ = 0) : S.IsUnderdamped
```

lemma isUnderdamped_decayRate [source]

An underdamped system has decay rate less than the natural frequency.

```
lemma isUnderdamped_decayRate (hS : S.IsUnderdamped) : S.decayRate < S.ω
```

lemma isCriticallyDamped_decayRate [source]

A critically damped system has decay rate equal to the natural frequency.

```
lemma isCriticallyDamped_decayRate (hS : S.IsCriticallyDamped) : S.ω = S.decayRate
```

lemma gamma_eq_two_mul_m_mul_decayRate [source]

The damping coefficient is twice mass times the decay rate.

```
lemma gamma_eq_two_mul_m_mul_decayRate : S.γ = 2 * S.m * S.decayRate
```

lemma k_eq_m_mul_ω_sq [source]

The spring constant is `m * ω^2`.

```
lemma k_eq_m_mul_ω_sq : S.k = S.m * S.ω^2
```

lemma k_eq_m_mul_decayRate_sq_of_criticallyDamped [source]

In the critically damped regime, `k = m * decayRate^2`.

```
lemma k_eq_m_mul_decayRate_sq_of_criticallyDamped (hS : S.IsCriticallyDamped) :
    S.k = S.m * S.decayRate^2
```

lemma isOverdamped_decayRate [source]

An overdamped system has decay rate greater than the natural frequency.

```
lemma isOverdamped_decayRate (hS : S.IsOverdamped) : S.ω < S.decayRate
```

lemma angularFrequency_eq_underdamped [source]

In the underdamped regime, the selected frequency uses the oscillation frequency.

```
lemma angularFrequency_eq_underdamped (hS : S.IsUnderdamped) :
    S.angularFrequency = sqrt (- S.discriminant) / (2 * S.m)
```

lemma angularFrequency_eq_criticallyDamped [source]

In the critically damped regime, the selected frequency is zero.

```
lemma angularFrequency_eq_criticallyDamped (hS : S.IsCriticallyDamped) :
    S.angularFrequency = 0
```

lemma angularFrequency_eq_overdamped [source]

In the overdamped regime, the selected frequency uses the real split rate.

```
lemma angularFrequency_eq_overdamped (hS : S.IsOverdamped) :
    S.angularFrequency = sqrt S.discriminant / (2 * S.m)
```

lemma angularFrequency_sq_of_underdamped [source]

In the underdamped regime, the selected angular frequency squares to `ω^2 - decayRate^2`.

```
lemma angularFrequency_sq_of_underdamped (hS : S.IsUnderdamped) :
    S.angularFrequency^2 = S.ω^2 - S.decayRate^2
```

lemma angularFrequency_pos_of_underdamped [source]

The selected angular frequency is positive in the underdamped regime.

```
lemma angularFrequency_pos_of_underdamped (hS : S.IsUnderdamped) :
    0 < S.angularFrequency
```

lemma angularFrequency_ne_zero_of_underdamped [source]

The selected angular frequency is nonzero in the underdamped regime.

```
lemma angularFrequency_ne_zero_of_underdamped (hS : S.IsUnderdamped) :
    S.angularFrequency ≠ 0
```

lemma angularFrequency_sq_of_overdamped [source]

In the overdamped regime, the selected angular frequency squares to `decayRate^2 - ω^2`.

```
lemma angularFrequency_sq_of_overdamped (hS : S.IsOverdamped) :
    S.angularFrequency^2 = S.decayRate^2 - S.ω^2
```

lemma angularFrequency_pos_of_overdamped [source]

The selected angular frequency is positive in the overdamped regime.

```
lemma angularFrequency_pos_of_overdamped (hS : S.IsOverdamped) :
    0 < S.angularFrequency
```

lemma angularFrequency_ne_zero_of_overdamped [source]

The selected angular frequency is nonzero in the overdamped regime.

```
lemma angularFrequency_ne_zero_of_overdamped (hS : S.IsOverdamped) :
    S.angularFrequency ≠ 0
```

def toUndamped [source]

Convert a damped oscillator to its underlying undamped oscillator when `γ = 0`.

```
@[nolint unusedArguments]
def toUndamped (S : DampedHarmonicOscillator) (_hS : S.IsUndamped) :
    HarmonicOscillator
```

lemma toUndamped_equationOfMotion [source]

When $\gamma = 0$, the damped equation of motion is equivalent to the equation of motion for the corresponding undamped harmonic oscillator.

```
lemma toUndamped_equationOfMotion (S : DampedHarmonicOscillator) (hS : S.IsUndamped)
    (xₜ : Time → EuclideanSpace ℝ (Fin 1)) (hx : ContDiff ℝ ∞ xₜ) :
    S.EquationOfMotion xₜ ↔ (S.toUndamped hS).EquationOfMotion xₜ
```

2.2 Physlib.ClassicalMechanics.DampedHarmonicOscillator.Solution

Physlib/ClassicalMechanics/DampedHarmonicOscillator/Solution.lean on GitHub.

structure InitialConditions [source]

The initial conditions for the damped harmonic oscillator, specified by an initial position and an initial velocity.

```
@[ext]
structure InitialConditions where
  x₀ : EuclideanSpace ℝ (Fin 1)
  v₀ : EuclideanSpace ℝ (Fin 1)
```

def underdampedBase [source]

The oscillatory part of the underdamped trajectory before exponential decay.

```
noncomputable def underdampedBase
    (IC : InitialConditions) : Time → EuclideanSpace ℝ (Fin 1)
```

def criticallyDampedBase [source]

The polynomial part of the critically damped trajectory before exponential decay.

```
noncomputable def criticallyDampedBase
    (IC : InitialConditions) : Time → EuclideanSpace ℝ (Fin 1)
```

def overdampedBase [source]

The hyperbolic part of the overdamped trajectory before exponential decay.

```
noncomputable def overdampedBase
    (IC : InitialConditions) : Time → EuclideanSpace ℝ (Fin 1)
```

def trajectory [source]

Given initial conditions, the solution selected from the damping regime of the oscillator.

```
noncomputable def trajectory
    (IC : InitialConditions) : Time → EuclideanSpace ℝ (Fin 1)
```

lemma trajectory_eq_of_underdamped [source]

In the underdamped regime, the selected trajectory uses the trigonometric base.

```
lemma trajectory_eq_of_underdamped (IC : InitialConditions) (hS : S.IsUnderdamped) :
    S.trajectory IC =
      fun t : Time => exp (-S.decayRate * t) • S.underdampedBase IC t
```

lemma trajectory_eq_of_criticallyDamped [source]

In the critically damped regime, the selected trajectory uses the polynomial base.

```
lemma trajectory_eq_of_criticallyDamped (IC : InitialConditions) (hS : S.IsCriticallyDamped) :
    S.trajectory IC =
      fun t : Time => exp (-S.decayRate * t) • S.criticallyDampedBase IC t
```

lemma trajectory_eq_of_overdamped [source]

In the overdamped regime, the selected trajectory uses the hyperbolic base.

```
lemma trajectory_eq_of_overdamped (IC : InitialConditions) (hS : S.IsOverdamped) :
    S.trajectory IC =
      fun t : Time => exp (-S.decayRate * t) • S.overdampedBase IC t
```

lemma exp_decay_smul_velocity [source]

```
private lemma exp_decay_smul_velocity
    (a : ℝ) (y : Time → EuclideanSpace ℝ (Fin 1)) (hy : Differentiable ℝ y) :
    ∂ₜ (fun t : Time => exp (-a * t.val) • y t) =
      fun t : Time => exp (-a * t.val) • (∂ₜ y t - a • y t)
```

lemma exp_decay_smul_acceleration [source]

```
private lemma exp_decay_smul_acceleration
    (a μ : ℝ) (y : Time → EuclideanSpace ℝ (Fin 1))
    (hy : Differentiable ℝ y) (hdy : Differentiable ℝ (∂ₜ y))
    (hy'' : ∂ₜ (∂ₜ y) = fun t => μ • y t) :
    ∂ₜ (∂ₜ (fun t : Time => exp (-a * t.val) • y t)) =
      fun t : Time => exp (-a * t.val) •
        (μ • y t - (2 * a) • ∂ₜ y t + a^2 • y t)
```

lemma exp_decay_smul_equationOfMotion [source]

```
private lemma exp_decay_smul_equationOfMotion
    (a μ : ℝ) (y : Time → EuclideanSpace ℝ (Fin 1))
    (hy : Differentiable ℝ y) (hdy : Differentiable ℝ (∂ₜ y))
    (hy'' : ∂ₜ (∂ₜ y) = fun t => μ • y t)
    (hγ : S.γ = 2 * S.m * a) (hk : S.k = S.m * (a^2 - μ)) :
    S.EquationOfMotion (fun t : Time => exp (-a * t.val) • y t)
```

lemma criticallyDampedBase_velocity [source]

```
private lemma criticallyDampedBase_velocity (IC : InitialConditions) :
    ∂ₜ (S.criticallyDampedBase IC) =
      fun _ : Time => IC.v₀ + S.decayRate • IC.x₀
```

lemma criticallyDampedBase_acceleration [source]

```
private lemma criticallyDampedBase_acceleration (IC : InitialConditions) :
    ∂ₜ (∂ₜ (S.criticallyDampedBase IC)) =
      fun _ => (0 : EuclideanSpace ℝ (Fin 1))
```

lemma underdampedBase_velocity [source]

```
private lemma underdampedBase_velocity (IC : InitialConditions) (hS : S.IsUnderdamped) :
    ∂ₜ (fun t : Time =>
      cos (S.angularFrequency * t.val) • IC.x₀ +
        (sin (S.angularFrequency * t.val) / S.angularFrequency) •
          (IC.v₀ + S.decayRate • IC.x₀)) =
    fun t : Time =>
      (-S.angularFrequency * sin (S.angularFrequency * t.val)) • IC.x₀ +
        cos (S.angularFrequency * t.val) •
          (IC.v₀ + S.decayRate • IC.x₀)
```

lemma underdampedBase_acceleration [source]

```
private lemma underdampedBase_acceleration (IC : InitialConditions) (hS : S.IsUnderdamped) :
    ∂ₜ (∂ₜ (fun t : Time =>
      cos (S.angularFrequency * t.val) • IC.x₀ +
        (sin (S.angularFrequency * t.val) / S.angularFrequency) •
          (IC.v₀ + S.decayRate • IC.x₀))) =
    fun t : Time => -S.angularFrequency^2 •
      (cos (S.angularFrequency * t.val) • IC.x₀ +
        (sin (S.angularFrequency * t.val) / S.angularFrequency) •
          (IC.v₀ + S.decayRate • IC.x₀))
```

lemma overdampedBase_velocity [source]

```
private lemma overdampedBase_velocity (IC : InitialConditions) (hS : S.IsOverdamped) :
    ∂ₜ (fun t : Time =>
      cosh (S.angularFrequency * t.val) • IC.x₀ +
        (sinh (S.angularFrequency * t.val) / S.angularFrequency) •
          (IC.v₀ + S.decayRate • IC.x₀)) =
    fun t : Time =>
      (S.angularFrequency * sinh (S.angularFrequency * t.val)) • IC.x₀ +
        cosh (S.angularFrequency * t.val) •
          (IC.v₀ + S.decayRate • IC.x₀)
```

lemma overdampedBase_acceleration [source]

```
private lemma overdampedBase_acceleration (IC : InitialConditions) (hS : S.IsOverdamped) :
    ∂ₜ (∂ₜ (fun t : Time =>
      cosh (S.angularFrequency * t.val) • IC.x₀ +
        (sinh (S.angularFrequency * t.val) / S.angularFrequency) •
          (IC.v₀ + S.decayRate • IC.x₀))) =
```

```
    fun t : Time => S.angularFrequency^2 •
      (cosh (S.angularFrequency * t.val) • IC.x₀ +
        (sinh (S.angularFrequency * t.val) / S.angularFrequency) •
          (IC.v₀ + S.decayRate • IC.x₀))
```

lemma trajectory_equationOfMotion_of_criticallyDamped [source]

In the critically damped regime, the selected trajectory satisfies the damped equation of motion.

```
lemma trajectory_equationOfMotion_of_criticallyDamped (IC : InitialConditions)
    (hS : S.IsCriticallyDamped) :
    S.EquationOfMotion (S.trajectory IC)
```

lemma trajectory_equationOfMotion_of_underdamped [source]

In the underdamped regime, the selected trajectory satisfies the damped equation of motion.

```
lemma trajectory_equationOfMotion_of_underdamped (IC : InitialConditions)
    (hS : S.IsUnderdamped) :
    S.EquationOfMotion (S.trajectory IC)
```

lemma trajectory_equationOfMotion_of_overdamped [source]

In the overdamped regime, the selected trajectory satisfies the damped equation of motion.

```
lemma trajectory_equationOfMotion_of_overdamped (IC : InitialConditions)
    (hS : S.IsOverdamped) :
    S.EquationOfMotion (S.trajectory IC)
```

lemma trajectory_equationOfMotion [source]

The selected trajectory satisfies the damped equation of motion.

```
lemma trajectory_equationOfMotion (IC : InitialConditions) :
    S.EquationOfMotion (S.trajectory IC)
```

2.3 Physlib.ClassicalMechanics.FreeParticle.Basic

Physlib/ClassicalMechanics/FreeParticle/Basic.lean on GitHub.

structure FreeParticle [source]

A classical free particle with positive mass.

A free particle is a mechanical system evolving in the absence of external forces. The dynamics are therefore entirely determined by Newton's second law with zero force.

The only parameter of the system is the particle mass. The assumption that the mass is strictly positive is physically natural and is used throughout the development when simplifying the equation of motion.

```
structure FreeParticle where
  mass : ℝ
  mass_pos : 0 < mass
```

abbrev Trajectory [source]

A trajectory is a time-dependent position function describing the motion of the particle in one spatial dimension. Defining the trajectory.

```
abbrev Trajectory
```

def velocity [source]

The velocity of a trajectory at a given time. This is defined as the time derivative of the position function.

```
@[nolint unusedArguments]
noncomputable
def velocity (s : FreeParticle) (q : Trajectory) (t : Time) : ℝ
```

def kineticEnergy [source]

The kinetic energy of the free particle along a trajectory.

This is given by the classical expression `E = (1 / 2) m v²`, where `m` is the particle mass and `v` is the velocity.

```
noncomputable
def kineticEnergy (s : FreeParticle) (q : Trajectory) (t : Time) : ℝ
```

def NewtonsSecondLaw [source]

Newton's second law for the free particle.

Since no external forces act on the particle, Newton's second law reduces to the equation `m q'' = 0`, expressing that the acceleration vanishes identically.

```
def NewtonsSecondLaw (s : FreeParticle) (q : Trajectory) (t : Time) : Prop
```

lemma accel_zero [source]

Newton's second law for a free particle implies that the acceleration vanishes identically.

Since the particle mass is strictly positive, the equation `m q'' = 0` can be simplified to `q'' = 0` by cancelling the mass factor.

```
lemma accel_zero (s : FreeParticle) (q : Trajectory) (h : ∀ t, s.NewtonsSecondLaw q t) :
    ∀ t, deriv (deriv q) t = 0
```

lemma velocity_const_of_zero_acc [source]

If the acceleration of a trajectory vanishes everywhere, then the velocity is constant.

More precisely, if the second derivative of the trajectory is zero for all times, then there exists a constant `v₀` such that the velocity is equal to `v₀` at every time.

The continuity assumption on `deriv q` is included to apply standard results from real analysis relating vanishing derivatives to constant functions.

```
@[sorryful]
lemma velocity_const_of_zero_acc (q : Time → ℝ) (h : ∀ t, deriv (deriv q) t = 0)
    (hcont : ContDiff ℝ 1 q) : ∃ v₀, ∀ t, deriv q t = v₀
```

theorem kineticEnergy_conserved [source]

A free particle satisfying the equation of motion conserves kinetic energy.

The proof follows the standard argument from classical mechanics: Newton's second law implies that the acceleration vanishes, which in turn implies that the velocity is constant. Since the kinetic energy depends only on the square of the velocity, it follows that the kinetic energy is constant in time.

```
@[sorryful]
theorem kineticEnergy_conserved (s : FreeParticle) (q : Trajectory)
    (h : ∀ t, s.NewtonsSecondLaw q t) (hcont : ContDiff ℝ 1 q) :
    ∃ E, ∀ t, s.kineticEnergy q t = E
```

2.4 Physlib.ClassicalMechanics.HarmonicOscillator.Solution

Physlib/ClassicalMechanics/HarmonicOscillator/Solution.lean on GitHub.

def period [source]

The period of a harmonic oscillator is `2 * π / ω`.

```
noncomputable def period (S : HarmonicOscillator) : ℝ
```

lemma period_eq [source]

```
lemma period_eq : T S = 2 * π / S.ω
```

lemma period_pos [source]

```
lemma period_pos : 0 < T S
```

lemma trajectory_periodic [source]

The trajectory of the harmonic oscillator is periodic with period of `2 * π / ω`.

```
lemma trajectory_periodic (IC : InitialConditions) :
    Function.Periodic (IC.trajectory S) (T S)
```

2.5 Physlib.ClassicalMechanics.OrbitalMechanics.VisViva

Physlib/ClassicalMechanics/OrbitalMechanics/VisViva.lean on GitHub.

structure VisViva [source]

System parameters for the vis-viva equation in circular orbital mechanics.

```
structure VisViva where
  G : ℝ
  M : ℝ
  m : ℝ
```

structure ConfigurationSpace [source]

Configuration space for orbital mechanics, defining the orbital radius.

```
structure ConfigurationSpace where
  r : ℝ
```

def speedCircular [source]

The orbital speed required for a circular orbit at radius `r`.

```
noncomputable def speedCircular (sys : VisViva) (cfg : ConfigurationSpace) : ℝ
```

lemma speedCircular_sq [source]

Lemma: the square of the circular orbit speed equals G M / r.

```
lemma speedCircular_sq (sys : VisViva) (cfg : ConfigurationSpace) (hr : 0 < cfg.r) (hG : 0 <
    sys.G)
    (hM : 0 < sys.M) :
    (speedCircular sys cfg)^2 = sys.G * sys.M / cfg.r
```

2.6 Physlib.Electromagnetism.Distributional.Basic

Physlib/Electromagnetism/Distributional/Basic.lean on GitHub.

def ofScalarPotential [source]

The creation of an electromagnetic potential from a scalar potential.

```
noncomputable def ofScalarPotential {d} (c : SpeedOfLight) :
    ((Time × Space d) →d[ℝ] ℝ) →ₗ[ℝ] DistElectromagneticPotential d where
```

def ofStaticScalarPotential [source]

The creation of an electromagnetic potential from a static scalar potential.

```
noncomputable def ofStaticScalarPotential {d} (c : SpeedOfLight) :
    ((Space d) →d[ℝ] ℝ) →ₗ[ℝ] DistElectromagneticPotential d
```

2.7 Physlib.Electromagnetism.Kinematics.ElectricField

Physlib/Electromagnetism/Kinematics/ElectricField.lean on GitHub.

lemma ofElectromagneticField_electricField [source]

The electric field of the electromagnetic potential created from the electric field `E` and the magnetic field `B` is `E`, as long as Gauss's law for magnetism and Faraday's law are satisfied.

```
lemma ofElectromagneticField_electricField {c : SpeedOfLight}
    (E : Time → Space 3 → EuclideanSpace ℝ (Fin 3)) (B : Time → Space 3 → EuclideanSpace ℝ (Fin
     3))
    (E_contDiff : ContDiff ℝ 1 ↿E) (B_contDiff : ContDiff ℝ 2 ↿B)
    (B_grad : ∀ t, ∇ · (B t) = 0) (faraday : ∀ t x, curl (E t) x = - ∂ₜ (B · x) t) :
    (ofElectromagneticField c E B).electricField c = E
```

2.8 Physlib.Electromagnetism.Kinematics.EMPotential

Physlib/Electromagnetism/Kinematics/EMPotential.lean on GitHub.

def ofScalarPotential [source]

The electromagnetic potential from a scalar potential, where the vector potential is set to zero.

```
noncomputable def ofScalarPotential {d} (c : SpeedOfLight)
    (ϕ : Time → Space d → ℝ) : ElectromagneticPotential d where
```

def ofStaticScalarPotential [source]

The creation of an electromagnetic potential from a static scalar potential.

```
noncomputable def ofStaticScalarPotential {d} (c : SpeedOfLight)
    (ϕ : Space d → ℝ) : ElectromagneticPotential d
```

def ofVectorPotential [source]

The electromagnetic potential from a vector potential, where the scalar potential is set equal to zero.

```
noncomputable def ofVectorPotential {d} (c : SpeedOfLight)
    (A : Time → Space d → EuclideanSpace ℝ (Fin d)) :
    ElectromagneticPotential d where
```

def ofStaticVectorPotential [source]

The creation of an electromagnetic potential from a static vector potential.

```
noncomputable def ofStaticVectorPotential {d} (c : SpeedOfLight)
    (A : Space d → EuclideanSpace ℝ (Fin d)) : ElectromagneticPotential d
```

def ofPotentials [source]

The creation of an electromagnetic potential from the non-relativistic potentials.

```
noncomputable def ofPotentials {d} (c : SpeedOfLight) (ϕ : Time → Space d → ℝ)
    (A : Time → Space d → EuclideanSpace ℝ (Fin d)) :
    ElectromagneticPotential d where
```

lemma ofPotentials_eq_add [source]

```
lemma ofPotentials_eq_add {d} (c : SpeedOfLight) (ϕ : Time → Space d → ℝ)
    (A : Time → Space d → EuclideanSpace ℝ (Fin d)) :
    ofPotentials c ϕ A = ofScalarPotential c ϕ + ofVectorPotential c A
```

def ofStaticPotentials [source]

The creation of of an electromagnetic potential from static potentials.

```
noncomputable def ofStaticPotentials {d} (c : SpeedOfLight) (ϕ : Space d → ℝ)
    (A : Space d → EuclideanSpace ℝ (Fin d)) : ElectromagneticPotential d
```

lemma ofStaticPotentials_eq_ofPotentials [source]

```
lemma ofStaticPotentials_eq_ofPotentials {d} (c : SpeedOfLight) (ϕ : Space d → ℝ)
    (A : Space d → EuclideanSpace ℝ (Fin d)) :
    ofStaticPotentials c ϕ A = ofPotentials c (fun _ => ϕ) (fun _ => A)
```

def ofElectromagneticField [source]

The electromagnetic potential from an electric and a magnetic field. This defines the electromagnetic potential in the Poincare gauge.

```
noncomputable def ofElectromagneticField (c : SpeedOfLight)
    (E : Time → Space 3 → EuclideanSpace ℝ (Fin 3))
    (B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)) :
    ElectromagneticPotential 3
```

lemma contDiff_action [source]

```
@[fun_prop]
lemma contDiff_action {d} (Λ : LorentzGroup d) (A : ElectromagneticPotential d)
    (hA : ContDiff ℝ n A) : ContDiff ℝ n (fun x => Λ • A (Λ⁻¹ • x))
```

lemma differentiable_deriv [source]

```
@[fun_prop]
lemma differentiable_deriv {d} {A : ElectromagneticPotential d}
    (hA : ContDiff ℝ 2 A) (μ ν : Fin 1 ⊕ Fin d) :
    Differentiable ℝ (fun x => ∂_ μ A x ν)
```

lemma differentiable_deriv_of_smooth [source]

```
@[fun_prop]
lemma differentiable_deriv_of_smooth {d} {A : ElectromagneticPotential d}
    (hA : ContDiff ℝ ∞ A) (μ ν : Fin 1 ⊕ Fin d) :
    Differentiable ℝ (fun x => ∂_ μ A x ν)
```

lemma contDiff_deriv [source]

```
@[fun_prop]
lemma contDiff_deriv {n} {d} {A : ElectromagneticPotential d}
    (hA : ContDiff ℝ (n + 1) A) (μ ν : Fin 1 ⊕ Fin d) :
```

```
    ContDiff ℝ n (fun x => ∂_ μ A x ν)
```

lemma differentiable_ofScalarPotential [source]

```
lemma differentiable_ofScalarPotential {d} (c : SpeedOfLight) (φ : Time → Space d → ℝ)
    (hϕ : Differentiable ℝ ↿φ) : Differentiable ℝ (ofScalarPotential c φ)
```

lemma contDiff_ofScalarPotential [source]

```
lemma contDiff_ofScalarPotential {n} {d} (c : SpeedOfLight) (φ : Time → Space d → ℝ)
    (hϕ : ContDiff ℝ n ↿φ) : ContDiff ℝ n (ofScalarPotential c φ)
```

lemma differentiable_ofVectorPotential [source]

```
lemma differentiable_ofVectorPotential {d} (c : SpeedOfLight)
    (A : Time → Space d → EuclideanSpace ℝ (Fin d))
    (hA : Differentiable ℝ ↿A) : Differentiable ℝ (ofVectorPotential c A)
```

lemma contDiff_ofVectorPotential [source]

```
lemma contDiff_ofVectorPotential {n} {d} (c : SpeedOfLight)
    (A : Time → Space d → EuclideanSpace ℝ (Fin d))
    (hA : ContDiff ℝ n ↿A) : ContDiff ℝ n (ofVectorPotential c A)
```

lemma differentiable_ofPotentials [source]

```
lemma differentiable_ofPotentials {d} (c : SpeedOfLight) (φ : Time → Space d → ℝ)
    (A : Time → Space d → EuclideanSpace ℝ (Fin d)) (hϕ : Differentiable ℝ ↿φ)
    (hA : Differentiable ℝ ↿A) : Differentiable ℝ (ofPotentials c φ A)
```

lemma contDiff_ofPotentials [source]

```
lemma contDiff_ofPotentials {n} {d} (c : SpeedOfLight) (φ : Time → Space d → ℝ)
    (A : Time → Space d → EuclideanSpace ℝ (Fin d)) (hϕ : ContDiff ℝ n ↿φ)
    (hA : ContDiff ℝ n ↿A) : ContDiff ℝ n (ofPotentials c φ A)
```

lemma contDiff_ofElectromagneticField [source]

```
lemma contDiff_ofElectromagneticField {n : ℕ} (c : SpeedOfLight)
    (E : Time → Space 3 → EuclideanSpace ℝ (Fin 3))
    (B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)) (hE : ContDiff ℝ n ↿E)
    (hB : ContDiff ℝ n ↿B) : ContDiff ℝ n (ofElectromagneticField c E B)
```

2.9 Physlib.Electromagnetism.Kinematics.FieldStrength

Physlib/Electromagnetism/Kinematics/FieldStrength.lean on GitHub.

lemma toFieldStrength_eq_sub_tensorDeriv [source]

```
lemma toFieldStrength_eq_sub_tensorDeriv {d} {A : ElectromagneticPotential d}
    (hA : Differentiable ℝ A) (x : SpaceTime d) :
    toFieldStrength A x =
```

```
    Tensorial.toTensor.symm (permT id PermCond.auto {η d | μ μ' ⊗ tensorDeriv A x | μ' ν}ᵀ)
    - Tensorial.toTensor.symm (permT ![1, 0] PermCond.auto
    {η d | μ μ' ⊗ tensorDeriv A x | μ' ν}ᵀ)
```

lemma toFieldStrength_eq_sum_basis_eval [source]

The statement that `F = F^{μν} eᵤ ⊗ eᵥ` written explicitly, with the components extracted via `toField`.

```
lemma toFieldStrength_eq_sum_basis_eval {d} {A : ElectromagneticPotential d} :
    A.toFieldStrength = fun x => ∑ μ, ∑ ν, toField {A.toFieldStrength x| [μ] [ν]}ᵀ •
      Vector.basis μ ⊗ₜ[ℝ] Vector.basis ν
```

lemma toFieldStrength_eq_sum_basis [source]

The statement that `F = F^{μν} eᵤ ⊗ eᵥ` written explicitly, with the components given by `∑ κ, (η μ κ * ∂_ κ A x ν - η ν κ * ∂_ κ A x μ)`.

```
lemma toFieldStrength_eq_sum_basis {d} {A : ElectromagneticPotential d}
    (hA : Differentiable ℝ A) (x : SpaceTime d) :
    A.toFieldStrength x = ∑ μ, ∑ ν, (∑ κ, (η μ κ *  ∂_ κ A x ν - η ν κ * ∂_ κ A x μ)) •
      Lorentz.Vector.basis μ ⊗ₜ Lorentz.Vector.basis ν
```

lemma toFieldStrength_eq_sum_basis_single [source]

The statement that `F = F^{μν} eᵤ ⊗ eᵥ` written explicitly, with with the components given by `(η μ μ * ∂_ μ A x ν - η ν ν * ∂_ ν A x μ)`.

```
lemma toFieldStrength_eq_sum_basis_single {d} {A : ElectromagneticPotential d}
    (hA : Differentiable ℝ A) (x : SpaceTime d) :
    A.toFieldStrength x = ∑ μ, ∑ ν, (η μ μ * ∂_ μ A x ν - η ν ν * ∂_ ν A x μ) •
      Lorentz.Vector.basis μ ⊗ₜ Lorentz.Vector.basis ν
```

lemma toFieldStrength_action_eq_sum [source]

This lemma expresses the component form of the transformed field strength tensor: when a Lorentz transformation Λ acts on the potential A, the resulting field strength tensor's components are given by the standard tensor transformation rule involving the Lorentz matrix elements $\Lambda\hat{}\mu\_\kappa$ and $\Lambda\hat{}\nu\_\rho$ applied to the original field components.

```
lemma toFieldStrength_action_eq_sum {d} (A : ElectromagneticPotential d) (Λ : LorentzGroup d)
    (hf : Differentiable ℝ A) (x : SpaceTime d) :
    (Λ • A).toFieldStrength x = ∑ μ, ∑ ν,
      (∑ κ, ∑ ρ, Λ.1 μ κ * Λ.1 ν ρ * toField {A.toFieldStrength (Λ⁻¹ • x) | [κ] [ρ]}ᵀ) •
      Vector.basis μ ⊗ₜ[ℝ] Vector.basis ν
```

lemma differentiable_toFieldStrength [source]

```
@[fun_prop]
lemma differentiable_toFieldStrength {d} {A : ElectromagneticPotential d} (hA : ContDiff ℝ 2 A)
    :
    Differentiable ℝ A.toFieldStrength
```

lemma differentiable_toFieldStrength_of_smooth [source]

```
@[fun_prop]
lemma differentiable_toFieldStrength_of_smooth {d}
    {A : ElectromagneticPotential d} (hA : ContDiff ℝ ∞ A) :
    Differentiable ℝ A.toFieldStrength
```

lemma contDiff_toFieldStrength [source]

```
@[fun_prop]
lemma contDiff_toFieldStrength {d} {n : WithTop ℕ∞} {A : ElectromagneticPotential d}
    (hA : ContDiff ℝ (n + 1) A) : ContDiff ℝ n A.toFieldStrength
```

2.10 Physlib.Electromagnetism.Kinematics.MagneticField

Physlib/Electromagnetism/Kinematics/MagneticField.lean on GitHub.

lemma ofElectromagneticField_magneticField [source]

The magnetic field of the electromagnetic potential created from the electric field `E` and the magnetic field `B` is `B`, as long as Gauss's law is satisfied.

```
lemma ofElectromagneticField_magneticField {c : SpeedOfLight}
    (E : ElectricField) (B : MagneticField) (B_contDiff : ∀ t, ContDiff ℝ 1 (B t))
    (B_grad : ∀ t, ∇ · (B t) = 0) :
    (ofElectromagneticField c E B).magneticField c = B
```

2.11 Physlib.Electromagnetism.Kinematics.ScalarPotential

Physlib/Electromagnetism/Kinematics/ScalarPotential.lean on GitHub.

lemma ofScalarPotential_scalarPotential [source]

```
@[simp]
lemma ofScalarPotential_scalarPotential {d} (c : SpeedOfLight)
    (φ : Time → Space d → ℝ) : (ofScalarPotential c φ).scalarPotential c = φ
```

lemma ofStaticScalarPotential_scalarPotential [source]

```
@[simp]
lemma ofStaticScalarPotential_scalarPotential {d} (c : SpeedOfLight)
    (φ : Space d → ℝ) : (ofStaticScalarPotential c φ).scalarPotential c = fun _ => φ
```

lemma ofVectorPotential_scalarPotential [source]

```
@[simp]
lemma ofVectorPotential_scalarPotential {d} (c : SpeedOfLight)
    (A : Time → Space d → EuclideanSpace ℝ (Fin d)) :
    (ofVectorPotential c A).scalarPotential = 0
```

lemma ofStaticVectorPotential_scalarPotential [source]

```
@[simp]
lemma ofStaticVectorPotential_scalarPotential {d} (c : SpeedOfLight)
    (A : Space d → EuclideanSpace ℝ (Fin d)) :
    (ofStaticVectorPotential c A).scalarPotential = 0
```

lemma ofPotentials_scalarPotential [source]

```
@[simp]
lemma ofPotentials_scalarPotential {d} (c : SpeedOfLight) (φ : Time → Space d → ℝ)
    (A : Time → Space d → EuclideanSpace ℝ (Fin d)) :
    (ofPotentials c φ A).scalarPotential c = φ
```

lemma ofStaticPotentials_scalarPotential [source]

```
@[simp]
lemma ofStaticPotentials_scalarPotential {d} (c : SpeedOfLight) (φ : Space d → ℝ)
    (A : Space d → EuclideanSpace ℝ (Fin d)) :
    (ofStaticPotentials c φ A).scalarPotential c = fun _ => φ
```

lemma ofElectromagneticField_scalarPotential [source]

```
lemma ofElectromagneticField_scalarPotential (c : SpeedOfLight)
    (E : Time → Space → EuclideanSpace ℝ (Fin 3))
    (B : Time → Space → EuclideanSpace ℝ (Fin 3)) :
    (ofElectromagneticField c E B).scalarPotential c = fun t x =>
    - ∫ u in (0 : ℝ)..1, ⟪E t (u • x), basis.repr x⟫_ℝ ∂(volume)
```

lemma ofElectromagneticField_scalarPotential_eq_add_vectorPotential [source]

```
lemma ofElectromagneticField_scalarPotential_eq_add_vectorPotential (c : SpeedOfLight)
    (E : Time → Space → EuclideanSpace ℝ (Fin 3))
    (B : Time → Space → EuclideanSpace ℝ (Fin 3)) (hb : ContDiff ℝ 1 ↿B) :
    (ofElectromagneticField c E B).scalarPotential c = fun t x =>
    - ∫ u in (0 : ℝ)..1, ⟪E t (u • x) +
    ∂ₜ ((ofElectromagneticField c E B).vectorPotential c ·
      (u • x)) t, basis.repr x⟫_ℝ ∂(volume)
```

2.12 Physlib.Electromagnetism.Kinematics.VectorPotential

Physlib/Electromagnetism/Kinematics/VectorPotential.lean on GitHub.

lemma ofScalarPotential_vectorPotential [source]

```
@[simp]
lemma ofScalarPotential_vectorPotential {d} (c : SpeedOfLight)
    (φ : Time → Space d → ℝ) : (ofScalarPotential c φ).vectorPotential c = 0
```

lemma ofStaticScalarPotential_vectorPotential [source]

```
@[simp]
lemma ofStaticScalarPotential_vectorPotential {d} (c : SpeedOfLight)
    (φ : Space d → ℝ) : (ofStaticScalarPotential c φ).vectorPotential c = 0
```

lemma ofVectorPotential_vectorPotential [source]

```
@[simp]
lemma ofVectorPotential_vectorPotential {d} (c : SpeedOfLight)
    (A : Time → Space d → EuclideanSpace ℝ (Fin d)) :
    (ofVectorPotential c A).vectorPotential c = A
```

lemma ofStaticVectorPotential_vectorPotential [source]

```
@[simp]
lemma ofStaticVectorPotential_vectorPotential {d} (c : SpeedOfLight)
    (A : Space d → EuclideanSpace ℝ (Fin d)) :
    (ofStaticVectorPotential c A).vectorPotential c = fun _ => A
```

lemma ofPotentials_vectorPotential [source]

```
@[simp]
lemma ofPotentials_vectorPotential {d} (c : SpeedOfLight) (φ : Time → Space d → ℝ)
    (A : Time → Space d → EuclideanSpace ℝ (Fin d)) :
    (ofPotentials c φ A).vectorPotential c = A
```

lemma ofStaticPotentials_vectorPotential [source]

```
@[simp]
lemma ofStaticPotentials_vectorPotential {d} (c : SpeedOfLight) (φ : Space d → ℝ)
    (A : Space d → EuclideanSpace ℝ (Fin d)) :
    (ofStaticPotentials c φ A).vectorPotential c = fun _ => A
```

lemma ofElectromagneticField_vectorPotential [source]

```
lemma ofElectromagneticField_vectorPotential (c : SpeedOfLight)
    (E : Time → Space 3 → EuclideanSpace ℝ (Fin 3))
    (B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)) :
    (ofElectromagneticField c E B).vectorPotential c =
    fun t x => - ∫ u in 0..(1 : ℝ), (u • Space.basis.repr x) ×ₑ₃ B t (u • x) ∂volume
```

lemma ofElectromagneticField_vectorPotential_apply [source]

```
lemma ofElectromagneticField_vectorPotential_apply {t x} (c : SpeedOfLight)
    (E : Time → Space 3 → EuclideanSpace ℝ (Fin 3))
    (B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)) (i : Fin 3) (hB : Continuous ↿B)  :
    (ofElectromagneticField c E B).vectorPotential c t x i =
    - ∫ u in 0..(1 : ℝ), ((u • Space.basis.repr x) ×ₑ₃ B t (u • x)) i ∂volume
```

lemma ofElectromagneticField_vectorPotential_apply_eq_expand [source]

```
lemma ofElectromagneticField_vectorPotential_apply_eq_expand {t x} {c : SpeedOfLight}
    {E : Time → Space 3 → EuclideanSpace ℝ (Fin 3)}
    {B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)} (hB : Continuous ↿B) (i : Fin 3) :
    (ofElectromagneticField c E B).vectorPotential c t x i =
    x (i + 2) * ∫ u in 0..(1 : ℝ), u * B t (u • x) (i + 1) ∂volume -
    x (i + 1) * ∫ u in 0..(1 : ℝ), u * B t (u • x) (i + 2) ∂volume
```

lemma contDiff_vectorPotential_ofElectromagneticField [source]

```
@[fun_prop]
lemma contDiff_vectorPotential_ofElectromagneticField {n : ℕ} (c : SpeedOfLight)
    (E : Time → Space 3 → EuclideanSpace ℝ (Fin 3))
    (B : Time → Space 3 → EuclideanSpace ℝ (Fin 3))
    (hB : ContDiff ℝ n ↿B) : ContDiff ℝ n ↿((ofElectromagneticField c E B).vectorPotential c)
```

lemma vectorPotential_inner_radial_eq_zero_ofElectromagneticField [source]

```
lemma vectorPotential_inner_radial_eq_zero_ofElectromagneticField
    {c : SpeedOfLight} {E B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)}
    {t : Time} {x : Space 3} {a : ℝ} (hB : Continuous ↿B) :
    ⟪(ofElectromagneticField c E B).vectorPotential c t (a • x), Space.basis.repr x⟫_ℝ = 0
```

lemma time_deriv_vectorPotential_inner_radial_eq_zero_ofElectromagneticField [source]

```
@[simp]
lemma time_deriv_vectorPotential_inner_radial_eq_zero_ofElectromagneticField
    {c : SpeedOfLight} {E B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)}
    {t : Time} {x : Space 3} {a : ℝ} (hB : ContDiff ℝ 1 ↿B) :
    ⟪∂ₜ ((ofElectromagneticField c E B).vectorPotential c · (a • x)) t,
      Space.basis.repr x⟫_ℝ = 0
```

2.13 Physlib.FluidDynamics.FluidState

Physlib/FluidDynamics/FluidState.lean on GitHub.

abbrev ScalarField [source]

A scalar field on `d`-dimensional space, depending on time.

```
abbrev ScalarField (d : ℕ)
```

abbrev VectorField [source]

A vector field on `d`-dimensional space, depending on time.

```
abbrev VectorField (d : ℕ)
```

abbrev MassDensity [source]

A mass density field on `d`-dimensional space.

```
abbrev MassDensity (d : ℕ)
```

abbrev VelocityField [source]

A velocity field on `d`-dimensional space.

```
abbrev VelocityField (d : ℕ)
```

abbrev MomentumDensityField [source]

A momentum density field on `d`-dimensional space.

```
abbrev MomentumDensityField (d : ℕ)
```

abbrev StressTensor [source]

A matrix-valued stress tensor field on `d`-dimensional space.

```
abbrev StressTensor (d : ℕ)
```

abbrev BodyForce [source]

A body-force field per unit mass on `d`-dimensional space.

```
abbrev BodyForce (d : ℕ)
```

structure FluidState [source]

The density and velocity fields of a fluid on `d`-dimensional space.

```
structure FluidState (d : ℕ) where
  rho : MassDensity d
  velocity : VelocityField d
```

structure FluidInMomentumBalance [source]

The fields needed for a momentum balance: fluid state, stress, and body force.

```
structure FluidInMomentumBalance (d : ℕ) extends FluidState d where
  stress : StressTensor d
  bodyForce : BodyForce d
```

2.14 Physlib.FluidDynamics.NavierStokes.Basic

Physlib/FluidDynamics/NavierStokes/Basic.lean on GitHub.

def NavierStokes [source]

The conservative Navier-Stokes balance-law form with an externally supplied stress tensor.

```
def NavierStokes (d : ℕ) (data : FluidInMomentumBalance d) : Prop
```

def ConvectiveNavierStokes [source]

The convective Navier-Stokes form with an externally supplied stress tensor.

```
def ConvectiveNavierStokes (d : ℕ) (data : FluidInMomentumBalance d) : Prop
```

theorem navierStokes_iff_convectiveNavierStokes [source]

The conservative and convective Navier-Stokes forms are equivalent when the fields are differentiable enough for the product rules.

```
theorem navierStokes_iff_convectiveNavierStokes
    (d : ℕ) (data : FluidInMomentumBalance d)
    (hRhoTime : ∀ t x, DifferentiableAt ℝ (data.rho · x) t)
    (hVelocityTime : ∀ t x, DifferentiableAt ℝ (data.velocity · x) t)
    (hMomentumDensity : ∀ t,
      Differentiable ℝ (FluidDynamics.NavierStokes.momentumDensity d data.toFluidState t))
    (hVelocitySpace : ∀ t, Differentiable ℝ (data.velocity t)) :
    NavierStokes d data ↔ ConvectiveNavierStokes d data
```

2.15 Physlib.FluidDynamics.NavierStokes.Continuity

Physlib/FluidDynamics/NavierStokes/Continuity.lean on GitHub.

def ClassicalContinuityEquation [source]

Classical conservation of mass in conservative form, `partial_t rho + div (rho u) = 0`.

The equation is asserted at points where the time derivative of `rho` and the spatial divergence of `rho u` are classical derivatives.

```
def ClassicalContinuityEquation (d : ℕ) (fluid : FluidState d) : Prop
```

def continuityResidual [source]

The scalar continuity-equation residual `partial_t rho + div (rho u)`.

```
noncomputable def continuityResidual (d : ℕ) (fluid : FluidState d) : Time → Space d → ℝ
```

def SmoothContinuityEquation [source]

A stronger continuity equation for globally differentiable fields.

This version records the first-order regularity needed by the classical continuity equation: the density is differentiable in time, the mass flux `rho u` is differentiable in space, and the continuity residual vanishes everywhere.

```
def SmoothContinuityEquation (d : ℕ) (fluid : FluidState d) : Prop
```

lemma toClassical [source]

A smooth continuity equation satisfies the guarded classical continuity equation.

```
lemma SmoothContinuityEquation.toClassical (d : ℕ) (fluid : FluidState d) :
    SmoothContinuityEquation d fluid → ClassicalContinuityEquation d fluid
```

2.16 Physlib.FluidDynamics.NavierStokes.Momentum

Physlib/FluidDynamics/NavierStokes/Momentum.lean on GitHub.

def momentumDensity [source]

The momentum density `rho u`.

```
def momentumDensity (d : ℕ) (fluid : FluidState d) : MomentumDensityField d
```

def momentumFlux [source]

The convective momentum flux `rho u ⊗ u`.

```
def momentumFlux (d : ℕ) (fluid : FluidState d) : Time → Space d → Matrix (Fin d) (Fin d) ℝ
```

def MomentumEquation [source]

Conservation of momentum in conservative matrix-divergence form.

The equation is

`partial_t (rho u) + matrixDiv (rho u ⊗ u) = matrixDiv sigma + rho f`.

Here `stress` is intentionally not yet specialized to a Newtonian stress law.

```
def MomentumEquation (d : ℕ) (data : FluidInMomentumBalance d) : Prop
```

def convectiveTerm [source]

The nonlinear transport term `(u · ∇)u`.

```
noncomputable def convectiveTerm (d : ℕ) (fluid : FluidState d) : VectorField d
```

def materialAcceleration [source]

The material acceleration `∂ₜ u + (u · ∇)u`.

```
noncomputable def materialAcceleration (d : ℕ) (fluid : FluidState d) : VectorField d
```

def ConvectiveMomentumEquation [source]

Conservation of momentum in convective form.

The equation is

`rho (partial_t u + (u · ∇)u) = matrixDiv sigma + rho f`.

Here `stress` is intentionally not yet specialized to a Newtonian stress law.

```
def ConvectiveMomentumEquation (d : ℕ) (data : FluidInMomentumBalance d) : Prop
```

def conservativeMomentumLHS [source]

The left-hand side of the conservative momentum equation.

```
noncomputable def conservativeMomentumLHS (d : ℕ) (fluid : FluidState d) : VectorField d
```

def convectiveMomentumLHS [source]

The left-hand side of the convective momentum equation.

```
noncomputable def convectiveMomentumLHS (d : ℕ) (fluid : FluidState d) : VectorField d
```

lemma timeDeriv_smul_velocity [source]

Product rule for the time derivative of a scalar field times a velocity field.

```
lemma timeDeriv_smul_velocity (d : ℕ) (rhoAtPosition : Time → ℝ)
    (velocityAtPosition : Time → EuclideanSpace ℝ (Fin d)) (t : Time)
    (hRho : DifferentiableAt ℝ rhoAtPosition t)
    (hVelocity : DifferentiableAt ℝ velocityAtPosition t) :
    ∂ₜ (fun t' => rhoAtPosition t' • velocityAtPosition t') t =
      rhoAtPosition t • ∂ₜ velocityAtPosition t + ∂ₜ rhoAtPosition t • velocityAtPosition t
```

lemma timeDeriv_momentumDensity [source]

Product rule for the time derivative of the momentum density `rho u`.

```
lemma timeDeriv_momentumDensity (d : ℕ) (fluid : FluidState d)
    (t : Time) (x : Space d)
    (hRho : DifferentiableAt ℝ (fluid.rho · x) t)
    (hVelocity : DifferentiableAt ℝ (fluid.velocity · x) t) :
    ∂ₜ (momentumDensity d fluid · x) t =
      fluid.rho t x • ∂ₜ (fluid.velocity · x) t + ∂ₜ (fluid.rho · x) t • fluid.velocity t x
```

lemma spaceDeriv_momentumFlux_component [source]

Product rule for one spatial derivative of one component of `rho u ⊗ u`.

```
lemma spaceDeriv_momentumFlux_component (d : ℕ) (fluid : FluidState d)
    (t : Time) (x : Space d) (i j : Fin d)
    (hMomentumDensity : Differentiable ℝ (momentumDensity d fluid t))
    (hVelocity : Differentiable ℝ (fluid.velocity t)) :
    ∂[j] (fun x' => momentumFlux d fluid t x' i j) x =
      fluid.velocity t x i • ∂[j] (fun x' => momentumDensity d fluid t x' j) x +
      ∂[j] (fun x' => fluid.velocity t x' i) x • momentumDensity d fluid t x j
```

lemma matrixDiv_momentumFlux [source]

The matrix divergence of `rho u ⊗ u` split into continuity and convective parts.

```
lemma matrixDiv_momentumFlux (d : ℕ) (fluid : FluidState d)
    (t : Time) (x : Space d)
    (hMomentumDensity : Differentiable ℝ (momentumDensity d fluid t))
    (hVelocity : Differentiable ℝ (fluid.velocity t)) :
    matrixDiv d (momentumFlux d fluid t) x =
      (∇ · momentumDensity d fluid t) x • fluid.velocity t x +
        fluid.rho t x • convectiveTerm d fluid t x
```

lemma conservativeMomentumLHS_eq_convectiveMomentumLHS_add_continuityResidual_smul [source]

The algebraic bridge between conservative and convective momentum.

The conservative momentum left-hand side equals the convective momentum left-hand side plus the continuity residual times the velocity field.

```
lemma conservativeMomentumLHS_eq_convectiveMomentumLHS_add_continuityResidual_smul
    (d : ℕ) (fluid : FluidState d)
    (t : Time) (x : Space d)
    (hRhoTime : DifferentiableAt ℝ (fluid.rho · x) t)
    (hVelocityTime : DifferentiableAt ℝ (fluid.velocity · x) t)
    (hMomentumDensity : Differentiable ℝ (momentumDensity d fluid t))
    (hVelocitySpace : Differentiable ℝ (fluid.velocity t)) :
    conservativeMomentumLHS d fluid t x =
      convectiveMomentumLHS d fluid t x + continuityResidual d fluid t x • fluid.velocity t x
```

theorem momentumEquation_iff_convectiveMomentumEquation [source]

The conservative and convective momentum equations are equivalent when the classical continuity equation holds.

The differentiability assumptions are exactly the product-rule assumptions used to rewrite `partial_t (rho u)` and `matrixDiv (rho u ⊗ u)`.

```
theorem momentumEquation_iff_convectiveMomentumEquation
    (d : ℕ) (data : FluidInMomentumBalance d)
    (hContinuity : ClassicalContinuityEquation d data.toFluidState)
    (hRhoTime : ∀ t x, DifferentiableAt ℝ (data.rho · x) t)
    (hVelocityTime : ∀ t x, DifferentiableAt ℝ (data.velocity · x) t)
    (hMomentumDensity : ∀ t,
      Differentiable ℝ (momentumDensity d data.toFluidState t))
    (hVelocitySpace : ∀ t, Differentiable ℝ (data.velocity t)) :
    MomentumEquation d data ↔ ConvectiveMomentumEquation d data
```

2.17 Physlib.Mathematics.Calculus.ParametricIntegration

Physlib/Mathematics/Calculus/ParametricIntegration.lean on GitHub.

lemma hasFDerivAt_parametric_intervalIntegral_of_contDiff [source]

```
lemma hasFDerivAt_parametric_intervalIntegral_of_contDiff
    {F : M → ℝ → N} (hf : ContDiff ℝ 1 ↿F) (x₀ : M) :
    HasFDerivAt (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume))
      (∫ (t : ℝ) in 0..1, fderiv ℝ (F · t) x₀ ∂(volume)) x₀
```

lemma fderiv_apply_parameteric_intervalIntegral [source]

```
lemma fderiv_apply_parameteric_intervalIntegral
    {F : M → ℝ → N} (hf : ContDiff ℝ 1 ↿F) (x₀ : M) (v : M) :
    fderiv ℝ (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume)) x₀ v =
      ∫ (t : ℝ) in 0..1, fderiv ℝ (F · t) x₀ v ∂(volume)
```

lemma fderiv_parameteric_intervalIntegral [source]

```
lemma fderiv_parameteric_intervalIntegral
    {F : M → ℝ → N} (hf : ContDiff ℝ 1 ↿F) (x₀ : M) :
    fderiv ℝ (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume))  =
      fun x => ∫ (t : ℝ) in 0..1, fderiv ℝ (F · t) x ∂(volume)
```

lemma contDiff_one_parametric_intervalIntegral_of_contDiff [source]

```
lemma contDiff_one_parametric_intervalIntegral_of_contDiff
    {F : M → ℝ → N} (hf : ContDiff ℝ 1 ↿F) :
    ContDiff ℝ 1 (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume))
```

lemma contDiff_succ_parametric_intervalIntegral_of_contDiff [source]

```
lemma contDiff_succ_parametric_intervalIntegral_of_contDiff {n : ℕ} [FiniteDimensional ℝ M]
    {F : M → ℝ → N} (hf : ContDiff ℝ (n + 1) ↿F) :
    ContDiff ℝ (n + 1) (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume))
```

lemma contDiff_parametric_intervalIntegral_of_contDiff [source]

```
lemma contDiff_parametric_intervalIntegral_of_contDiff {n : ℕ} {M : Type}
    [NormedAddCommGroup M] [NormedSpace ℝ M] [ProperSpace M] [FiniteDimensional ℝ M]
    {F : M → ℝ → N} (hf : ContDiff ℝ n ↿F) :
    ContDiff ℝ n (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume))
```

2.18 Physlib.Particles.StandardModel.AnomalyCancellation.NoGrav.One.LinearParameterization

Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/One/LinearParameterization.lean on GitHub.

lemma bijection_coe_val [source]

```
lemma bijection_coe_val (S : linearParametersQENeqZero) :
    (bijection S).1.val = (bijectionLinearParameters S : linearParameters).asCharges
```

2.19 Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Basic

Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Basic.lean on GitHub.

lemma cubicACC_apply [source]

```
lemma cubicACC_apply (S : MSSMACC.Charges) : MSSMACC.cubicACC S = cubeTriLin.toCubic S
```

2.20 Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.LineY3B3

Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/LineY3B3.lean on GitHub.

lemma lineY₃B₃Charges_val [source]

```
lemma lineY₃B₃Charges_val (a b : ℚ) :
    (lineY₃B₃Charges a b).val = a • Y₃.1.1.val + b • B₃.1.1.val
```

lemma lineY₃B₃_val [source]

```
lemma lineY₃B₃_val (a b : ℚ) : (lineY₃B₃ a b).val = a • Y₃.val + b • B₃.val
```

2.21 Physlib.QuantumMechanics.DDimensions.Basic

Physlib/QuantumMechanics/DDimensions/Basic.lean on GitHub.

structure SpaceDQuantumSystem [source]

A single-particle quantum system with Hilbert space `SpaceDHilbertSpace d`, characterized by the number of spatial dimensions `d : ℕ`, particle's mass `m > 0` and potential function `potential : Space d → ℝ`.

```
@[ext]
structure SpaceDQuantumSystem where
  d : ℕ
  m : ℝ
  hm : 0 < m
  potential : Space d → ℝ
```

abbrev HS [source]

The Hilbert space, `SpaceDHilbertSpace Q.d`.

```
abbrev HS
```

lemma m_pos [source]

```
@[simp]
lemma m_pos : 0 < Q.m
```

lemma m_nonneg [source]

```
@[simp]
lemma m_nonneg : 0 ≤ Q.m
```

lemma m_ne_zero [source]

```
@[simp]
lemma m_ne_zero : Q.m ≠ 0
```

def kineticCLM [source]

The kinetic operator $(2m)^{-1}\mathbf{p}^2$ as a continuous linear map on Schwartz space.

```
def kineticCLM : 𝓢(Space Q.d, ℂ) →L[ℂ] 𝓢(Space Q.d, ℂ)
```

lemma kineticCLM_eq [source]

```
lemma kineticCLM_eq : Q.kineticCLM = (2 * Q.m)⁻¹ • (𝐩 ⬝ᵥ 𝐩)
```

def kineticOperator [source]

The kinetic operator as an unbounded operator with domain `schwartzSubmodule Q.d`.

```
def kineticOperator : Q.HS →ₗ.[ℂ] Q.HS
```

lemma kineticOperator_eq [source]

```
lemma kineticOperator_eq : Q.kineticOperator = ofReal (2 * Q.m)⁻¹ • momentumSqOperator
```

def potentialCLM [source]

The potential operator as a continuous linear map on Schwartz space, where `Q.potential` is a function of temperate growth.

```
def potentialCLM : 𝓢(Space Q.d, ℂ) →L[ℂ] 𝓢(Space Q.d, ℂ)
```

lemma potentialCLM_eq [source]

```
lemma potentialCLM_eq : Q.potentialCLM = smulLeftCLM ℂ (ofReal ∘ Q.potential)
```

lemma potentialCLM_apply [source]

```
lemma potentialCLM_apply (h_HTG : Q.potential.HasTemperateGrowth) (ψ : 𝓢(Space Q.d, ℂ)) :
    Q.potentialCLM ψ = fun x ↦ Q.potential x • ψ x
```

lemma potentialCLM_apply_apply [source]

```
@[simp]
lemma potentialCLM_apply_apply
    (h_HTG : Q.potential.HasTemperateGrowth) (ψ : 𝓢(Space Q.d, ℂ)) (x : Space Q.d) :
    Q.potentialCLM ψ x = Q.potential x • ψ x
```

def potentialOperator [source]

The potential operator as a self-adjoint, unbounded multiplication operator with domain `{ψ ∈ Q.HS | Q.potential • ψ ∈ Q.HS}`.

```
def potentialOperator : Q.HS →ₗ.[ℂ] Q.HS
```

lemma potentialOperator_eq [source]

```
lemma potentialOperator_eq : Q.potentialOperator = 𝓜 (ofReal ∘ Q.potential)
```

lemma potentialOperator_isSelfAdjoint [source]

```
lemma potentialOperator_isSelfAdjoint (h_AESM : AEStronglyMeasurable Q.potential) :
    IsSelfAdjoint Q.potentialOperator
```

lemma potentialOperator_domain_ge [source]

```
lemma potentialOperator_domain_ge (h_HTG : Q.potential.HasTemperateGrowth) :
    schwartzSubmodule Q.d ≤ Q.potentialOperator.domain
```

def hamiltonianCLM [source]

The Hamiltonian operator as a continuous linear map on Schwartz space, where `Q.potential` is a function of temperate growth.

```
def hamiltonianCLM : 𝓢(Space Q.d, ℂ) →L[ℂ] 𝓢(Space Q.d, ℂ)
```

lemma hamiltonianCLM_eq [source]

```
lemma hamiltonianCLM_eq : Q.hamiltonianCLM = Q.kineticCLM + Q.potentialCLM
```

def hamiltonianOperator [source]

The Hamilontian operator as a symmetric unbounded operator.

```
def hamiltonianOperator : Q.HS →ₗ.[ℂ] Q.HS
```

lemma hamiltonianOperator_eq [source]

```
lemma hamiltonianOperator_eq : Q.hamiltonianOperator = Q.kineticOperator + Q.potentialOperator
```

2.22 Physlib.QuantumMechanics.DDimensions.Hydrogen.Basic

Physlib/QuantumMechanics/DDimensions/Hydrogen/Basic.lean on GitHub.

lemma potential_eq [source]

```
@[simp]
lemma potential_eq : H.potential = fun x ↦ -H.k * ‖x‖⁻¹
```

lemma potential_AESM [source]

```
@[fun_prop]
lemma potential_AESM : AEStronglyMeasurable H.potential
```

lemma potential_AEM [source]

```
@[fun_prop]
lemma potential_AEM : AEMeasurable H.potential
```

def hamiltonianRegCLM [source]

The hydrogen atom Hamiltonian regularized by $\varepsilon \neq 0$ *is defined to be* $\mathbf{H}(\varepsilon) = (2m)^{-1}\mathbf{p}^2 - k\cdot\mathbf{r}(\varepsilon)^{-1}$.

```
def hamiltonianRegCLM (ε : ℝˣ) : 𝓢(Space H.d, ℂ) →L[ℂ] 𝓢(Space H.d, ℂ)
```

lemma hamiltonianRegCLM_eq [source]

```
lemma hamiltonianRegCLM_eq (ε : ℝˣ) :
    H.hamiltonianRegCLM ε = (2 * H.m)⁻¹ • (𝐩 ⬝ᵥ 𝐩) - H.k • 𝐫₀ ε (-1)
```

2.23 Physlib.QuantumMechanics.DDimensions.Operators.Covariance

Physlib/QuantumMechanics/DDimensions/Operators/Covariance.lean on GitHub.

def covariance [source]

Covariance, defined as the real part of the centered inner product.

```
def covariance : ℝ
```

lemma covariance_eq_re_inner_centered [source]

Covariance, unfolded to the real part of the centered inner product.

```
lemma covariance_eq_re_inner_centered :
    covariance A B ψ hψB =
      (⟪centered A ψ, centered B ⟨ψ, hψB⟩⟫_ℂ).re
```

lemma covariance_comm [source]

Swapping the two observables does not change the covariance.

```
lemma covariance_comm :
    covariance A B ψ hψB = covariance B A ⟨ψ, hψB⟩ ψ.2
```

lemma covariance_eq_re_symm_centered [source]

Covariance as the real part of the symmetrized centered inner product.

```
lemma covariance_eq_re_symm_centered :
    covariance A B ψ hψB =
      ((⟪centered A ψ, centered B ⟨ψ, hψB⟩⟫_ℂ +
        ⟪centered B ⟨ψ, hψB⟩, centered A ψ⟫_ℂ).re) / 2
```

lemma covariance_self_eq_variance [source]

```
@[simp]
lemma covariance_self_eq_variance (A : H →ₗ.[ℂ] H) (ψ : A.domain) :
    covariance A A ψ (by exact ψ.2) = variance A ψ
```

2.24 Physlib.QuantumMechanics.DDimensions.Operators.Momentum

Physlib/QuantumMechanics/DDimensions/Operators/Momentum.lean on GitHub.

def momentumCLM [source]

Component `i` of the momentum operator is the continuous linear map from `𝓢(Space d, ℂ)` to itself which maps `ψ` to `-iħ ∂ᵢψ`.

```
def momentumCLM : 𝓢(Space d, ℂ) →L[ℂ] 𝓢(Space d, ℂ)
```

lemma momentumCLM_apply_fun [source]

```
lemma momentumCLM_apply_fun (ψ : 𝓢(Space d, ℂ)) : 𝐩 i ψ = (-I * ℏ) • ∂[i] ψ
```

lemma momentumCLM_apply [source]

```
@[simp]
lemma momentumCLM_apply (ψ : 𝓢(Space d, ℂ)) (x : Space d) : 𝐩 i ψ x = -I * ℏ * ∂[i] ψ x
```

lemma momentumOperator_apply_ae [source]

```
lemma momentumOperator_apply_ae (ψ : schwartzSubmodule d) :
    𝓟 i ψ =ᵐ[volume] 𝐩 i (schwartzEquiv.symm ψ)
```

lemma momentumOperator_range [source]

```
lemma momentumOperator_range (ψ : schwartzSubmodule d) : 𝓟 i ψ ∈ schwartzSubmodule d
```

lemma momentumOperator_hasDenseDomain [source]

```
lemma momentumOperator_hasDenseDomain : (𝓟 i).HasDenseDomain
```

lemma momentumOperator_isSymmetric [source]

```
lemma momentumOperator_isSymmetric : (𝓟 i).IsSymmetric
```

lemma momentumOperator_isUnbounded [source]

```
lemma momentumOperator_isUnbounded : (𝓟 i).IsUnbounded
```

def momentumSqOperator [source]

The square of the momentum operator.

```
def momentumSqOperator : SpaceDHilbertSpace d →ₗ.[ℂ] SpaceDHilbertSpace d
```

lemma momentumSqOperator_eq [source]

```
lemma momentumSqOperator_eq :
    momentumSqOperator (d := d) = sum fun i ↦ (𝓟 i).comp (𝓟 i) (momentumOperator_range i)
```

lemma momentumSqOperator_domain_eq [source]

```
lemma momentumSqOperator_domain_eq : momentumSqOperator.domain = schwartzSubmodule d
```

2.25 Physlib.QuantumMechanics.DDimensions.Operators.Multiplication

Physlib/QuantumMechanics/DDimensions/Operators/Multiplication.lean on GitHub.

def mulOperator [source]

The LinearPMap which maps `ψ` to `f • ψ` with domain `{ψ | f • ψ ∈ SpaceDHilbertSpace d}`.

```
def mulOperator (f : Space d → ℂ) : SpaceDHilbertSpace d →ₗ.[ℂ] SpaceDHilbertSpace d where
```

lemma mem_mulOperator_domain_iff [source]

```
lemma mem_mulOperator_domain_iff
    {f : Space d → ℂ} {ψ : SpaceDHilbertSpace d} : ψ ∈ (𝓜 f).domain ↔ MemHS (f • ψ.val.cast)
```

lemma mulOperator_apply_ae [source]

```
lemma mulOperator_apply_ae {f : Space d → ℂ} (ψ : (𝓜 f).domain) : (𝓜 f) ψ =ᵐ[volume] f • ψ
```

lemma mulOperator_hasDenseDomain [source]

```
lemma mulOperator_hasDenseDomain {f : Space d → ℂ} (hf : AEStronglyMeasurable f) :
    (𝓜 f).HasDenseDomain
```

lemma mulOperator_domain_ge_of_hasTemperateGrowth [source]

```
lemma mulOperator_domain_ge_of_hasTemperateGrowth
    {f : Space d → ℂ} (hf : f.HasTemperateGrowth) : schwartzSubmodule d ≤ (𝓜 f).domain
```

lemma mulOperator_conj_domain [source]

```
lemma mulOperator_conj_domain {f : Space d → ℂ} (hf : AEStronglyMeasurable f) :
    (𝓜 (conj ∘ f)).domain = (𝓜 f).domain
```

lemma exists_monotone_sets_hasFiniteIntegral [source]

```
private lemma exists_monotone_sets_hasFiniteIntegral
    (f g : Space d → ℂ) (hf : AEStronglyMeasurable f) (hg : AEStronglyMeasurable g) :
    ∃ s : ℕ → Set (Space d), Monotone s ∧ ⋃ n, s n = Set.univ ∧ (∀ n, MeasurableSet (s n))
      ∧ ∀ k, k = 1 ∨ k = 2 →
        ∀ n, HasFiniteIntegral (fun x ↦ ‖f x ^ k * g x‖ ^ 2) (volume.restrict (s n))
```

lemma mulOperator_adjoint_domain_le [source]

```
lemma mulOperator_adjoint_domain_le {f : Space d → ℂ} (hf : AEStronglyMeasurable f) :
    (𝓜 f)†.domain ≤ (𝓜 (conj ∘ f)).domain
```

lemma mulOperator_adjoint_eq_conj [source]

```
lemma mulOperator_adjoint_eq_conj {f : Space d → ℂ} (hf : AEStronglyMeasurable f) :
    (𝓜 f)† = 𝓜 (conj ∘ f)
```

lemma mulOperator_isSelfAdjoint_ofReal [source]

```
lemma mulOperator_isSelfAdjoint_ofReal
    {f : Space d → ℂ} (hf : AEStronglyMeasurable f) (hf' : conj ∘ f = f) :
    IsSelfAdjoint (𝓜 f)
```

lemma mulOperator_isClosable [source]

```
lemma mulOperator_isClosable {f : Space d → ℂ} (hf : AEStronglyMeasurable f) :
    (𝓜 f).IsClosable
```

lemma mulOperator_isUnbounded [source]

```
lemma mulOperator_isUnbounded {f : Space d → ℂ} (hf : AEStronglyMeasurable f) :
    (𝓜 f).IsUnbounded
```

lemma mulOperator_compRestricted_le [source]

```
lemma mulOperator_compRestricted_le (f g : Space d → ℂ) : 𝓜 f ∘ᵣ 𝓜 g ≤ 𝓜 (f • g)
```

lemma mulOperator_compRestricted_eq [source]

```
lemma mulOperator_compRestricted_eq (f : Space d → ℂ) {g : Space d → ℂ} (h : (𝓜 g).domain = ⊤)
    :
    𝓜 f ∘ᵣ 𝓜 g = 𝓜 (f • g)
```

2.26 Physlib.QuantumMechanics.DDimensions.Operators.Position

Physlib/QuantumMechanics/DDimensions/Operators/Position.lean on GitHub.

def positionCLM [source]

Component `i` of the position operator is the continuous linear map from `𝓢(Space d, ℂ)` to itself which maps `ψ` to `xᵢψ`.

```
def positionCLM : 𝓢(Space d, ℂ) →L[ℂ] 𝓢(Space d, ℂ)
```

lemma positionCLM_apply_fun [source]

```
lemma positionCLM_apply_fun (ψ : 𝓢(Space d, ℂ)) : 𝐱 i ψ = (fun x : Space d ↦ x i) • ⇑ψ
```

lemma positionCLM_apply [source]

```
@[simp]
lemma positionCLM_apply (ψ : 𝓢(Space d, ℂ)) (x : Space d) : 𝐱 i ψ x = x i * ψ x
```

lemma normRegularizedPow_eq [source]

```
lemma normRegularizedPow_eq (d : ℕ) (ε s : ℝ) :
    normRegularizedPow d ε s = fun x ↦ (‖x‖ ^ 2 + ε ^ 2) ^ (s / 2)
```

lemma normRegularizedPow_measurable [source]

```
@[fun_prop]
lemma normRegularizedPow_measurable (d : ℕ) (ε s : ℝ) :
    Measurable (normRegularizedPow d ε s)
```

def radiusRegPowCLM [source]

The radius operator to power `s`, regularized by `ε ≠ 0`, is the continuous linear map from `𝓢(Space d, ℂ)` to itself which maps `ψ` to `(‖x‖² + ε²)^(s/2) • ψ`.

```
def radiusRegPowCLM {d : ℕ} (ε : ℝˣ) (s : ℝ) : 𝓢(Space d, ℂ) →L[ℂ] 𝓢(Space d, ℂ)
```

lemma radiusRegPowCLM_apply_fun [source]

```
lemma radiusRegPowCLM_apply_fun {d : ℕ} (ε : ℝˣ) (s : ℝ) (ψ : 𝓢(Space d, ℂ)) :
    𝐫₀ ε s ψ = fun x ↦ (‖x‖ ^ 2 + ε ^ 2) ^ (s / 2) • ψ x
```

lemma radiusRegPowCLM_apply [source]

```
@[simp]
lemma radiusRegPowCLM_apply {d : ℕ} (ε : ℝˣ) (s : ℝ) (ψ : 𝓢(Space d, ℂ)) (x : Space d) :
    𝐫₀ ε s ψ x = (‖x‖ ^ 2 + ε ^ 2) ^ (s / 2) • ψ x
```

lemma radiusRegPowCLM_comp_eq [source]

```
@[simp]
lemma radiusRegPowCLM_comp_eq {d : ℕ} (ε : ℝˣ) (s t : ℝ) :
    𝐫₀[d] ε s ∘L 𝐫₀ ε t = 𝐫₀ ε (s+t)
```

lemma radiusRegPowCLM_zero [source]

```
@[simp]
lemma radiusRegPowCLM_zero {d : ℕ} (ε : ℝˣ) :
    𝐫₀ ε 0 = ContinuousLinearMap.id ℂ 𝓢(Space d, ℂ)
```

lemma positionSqCLM_eq [source]

```
lemma positionSqCLM_eq {d : ℕ} (ε : ℝˣ) :
    ∑ i, 𝐱 i ∘L 𝐱 i = 𝐫₀ ε 2 - ε.1 ^ 2 • ContinuousLinearMap.id ℂ 𝓢(Space d, ℂ)
```

def radiusPowLM [source]

The radius operator to power `s` is the linear map from `𝓢(Space d, ℂ)` to `Space d → ℂ` that maps `ψ` to `x ↦ ‖x‖ˢψ(x)` (which is 'nearly' Schwartz for general `s`).

```
def radiusPowLM {d : ℕ} (s : ℝ) : 𝓢(Space d, ℂ) →ₗ[ℂ] Space d → ℂ where
```

lemma radiusPowLM_apply_fun [source]

```
lemma radiusPowLM_apply_fun {d : ℕ} (s : ℝ) (ψ : 𝓢(Space d, ℂ)) :
    𝐫 s ψ = fun x ↦ ‖x‖ ^ s • ψ x
```

lemma radiusPowLM_apply [source]

```
@[simp]
lemma radiusPowLM_apply {d : ℕ} (s : ℝ) (ψ : 𝓢(Space d, ℂ)) (x : Space d) :
    𝐫 s ψ x = ‖x‖ ^ s • ψ x
```

lemma radiusPowLM_apply_contDiffAt [source]

`x ↦ ‖x‖ˢψ(x)` is smooth away from `x = 0`.

```
@[fun_prop]
lemma radiusPowLM_apply_contDiffAt {d : ℕ} (s : ℝ) (n : ℕ∞) (ψ : 𝓢(Space d, ℂ)) {x : Space d}
    (hx : x ≠ 0) : ContDiffAt ℝ n (𝐫 s ψ) x
```

lemma radiusPowLM_apply_stronglyMeasurable [source]

$x \mapsto \|x\|^s \psi(x)$ is strongly measurable.

```
@[fun_prop]
lemma radiusPowLM_apply_stronglyMeasurable {d : ℕ} (s : ℝ) (ψ : 𝓢(Space d, ℂ)) :
    StronglyMeasurable (𝐫 s ψ)
```

lemma radiusPowLM_apply_memHS [source]

$x \mapsto \|x\|^s \psi(x)$ is square-integrable provided `s` is not too negative.

```
lemma radiusPowLM_apply_memHS {d : ℕ} (s : ℝ) (ψ : 𝓢(Space d, ℂ)) (a : ℕ)
    (hψ : ψ ∈ polyBddSchwartzMap d a) (h : 0 < d + 2 * (a + s)) :
    MemHS (𝐫 s ψ)
```

def positionOperator [source]

The operator on `SpaceDHilbertSpace d` acting by multiplication by `fun x ↦ xᵢ`.

```
def positionOperator : SpaceDHilbertSpace d →ₗ[ℂ] SpaceDHilbertSpace d
```

lemma positionOperator_hasDenseDomain [source]

```
lemma positionOperator_hasDenseDomain : (𝓧 i).HasDenseDomain
```

lemma positionOperator_isSelfAdjoint [source]

```
lemma positionOperator_isSelfAdjoint : IsSelfAdjoint (𝓧 i)
```

lemma positionOperator_isUnbounded [source]

```
lemma positionOperator_isUnbounded : (𝓧 i).IsUnbounded
```

def radiusRegPowOperator [source]

The operator on `SpaceDHilbertSpace d` acting by multiplication by `fun x ↦ (‖x‖² + ε²)^(s/2)`.

```
def radiusRegPowOperator (ε : ℝˣ) (s : ℝ) : SpaceDHilbertSpace d →ₗ[ℂ] SpaceDHilbertSpace d
```

lemma radiusRegPowOperator_hasDenseDomain [source]

```
lemma radiusRegPowOperator_hasDenseDomain (ε : ℝˣ) (s : ℝ) : (𝓡₀[d] ε s).HasDenseDomain
```

lemma radiusRegPowOperator_isSelfAdjoint [source]

```
lemma radiusRegPowOperator_isSelfAdjoint (ε : ℝˣ) (s : ℝ) : IsSelfAdjoint (𝓡₀[d] ε s)
```

lemma radiusRegPowOperator_isUnbounded [source]

```
lemma radiusRegPowOperator_isUnbounded (ε : ℝˣ) (s : ℝ) : (𝓡₀[d] ε s).IsUnbounded
```

def radiusPowOperator [source]

The operator on `SpaceDHilbertSpace d` acting by multiplication by `fun x ↦ ‖x‖ ^ s`.

```
def radiusPowOperator (s : ℝ) : SpaceDHilbertSpace d →ₗ.[ℂ] SpaceDHilbertSpace d
```

lemma radiusPowOperator_hasDenseDomain [source]

```
lemma radiusPowOperator_hasDenseDomain (s : ℝ) : (𝓡[d] s).HasDenseDomain
```

lemma radiusPowOperator_isSelfAdjoint [source]

```
lemma radiusPowOperator_isSelfAdjoint (s : ℝ) : IsSelfAdjoint (𝓡[d] s)
```

lemma radiusPowOperator_isUnbounded [source]

```
lemma radiusPowOperator_isUnbounded (s : ℝ) : (𝓡[d] s).IsUnbounded
```

lemma add_floor_toNat_pos_aux [source]

```
private lemma add_floor_toNat_pos_aux (d : ℕ) (s : ℝ) :
    0 < d + 2 * (⌊1 - d / 2 - s⌋.toNat + s)
```

lemma radiusPowLM_apply_polyBddSchwartz_memHS [source]

```
lemma radiusPowLM_apply_polyBddSchwartz_memHS {d : ℕ} {s : ℝ}
    (ψ : polyBddSchwartzSubmodule d ⌊1 - d / 2 - s⌋.toNat) :
    MemHS (𝐫[d] s (polyBddSchwartzEquiv.symm ψ))
```

lemma radiusPowOperator_domain_ge [source]

```
lemma radiusPowOperator_domain_ge {d : ℕ} (s : ℝ) :
    polyBddSchwartzSubmodule d ⌊1 - d / 2 - s⌋.toNat ≤ (radiusPowOperator s).domain
```

2.27 Physlib.QuantumMechanics.DDimensions.Operators.StateObservables.ExpectedValue

Physlib/QuantumMechanics/DDimensions/Operators/StateObservables/ExpectedValue.lean on GitHub.

lemma conj_inner_apply_self_eq_of_isSymmetric [source]

```
private lemma conj_inner_apply_self_eq_of_isSymmetric (T : H →ₗ.[ℂ] H) (hT : T.IsSymmetric)
    (ψ : T.domain) :
    (starRingEnd ℂ) ⟪(ψ : H), T ψ⟫_ℂ = ⟪(ψ : H), T ψ⟫_ℂ
```

def expectedValue [source]

Expectation value `re ⟪ψ, Tψ⟫_ℂ` for `ψ ∈ T.domain`.

For symmetric `T`, this agrees with `⟪ψ, Tψ⟫_ℂ` after coercion from ℝ; see `expectedValue_eq_inner`.

```
def expectedValue (T : H →ₗ.[ℂ] H) (ψ : T.domain) : ℝ
```

lemma expectedValue_eq_re_inner [source]

The expectation value, unfolded as a real part.

```
lemma expectedValue_eq_re_inner (T : H →ₗ.[ℂ] H) (ψ : T.domain) :
    expectedValue T ψ = (⟪(ψ : H), T ψ⟫_ℂ).re
```

lemma expectedValue_eq_inner [source]

If `T` is symmetric, `⟪ψ, Tψ⟫_ℂ` is the expectation value, coerced to `ℂ`.

```
lemma expectedValue_eq_inner (T : H →ₗ.[ℂ] H) (hT : T.IsSymmetric) (ψ : T.domain) :
    ⟪(ψ : H), T ψ⟫_ℂ = (expectedValue T ψ : ℂ)
```

lemma inner_eq_expectedValue [source]

Reverse orientation of `LinearPMap.expectedValue_eq_inner`.

```
lemma inner_eq_expectedValue (T : H →ₗ.[ℂ] H) (hT : T.IsSymmetric) (ψ : T.domain) :
    (expectedValue T ψ : ℂ) = ⟪(ψ : H), T ψ⟫_ℂ
```

def centered [source]

The centered vector `Tψ - ⟨T⟩_ψ ψ`.

```
def centered (T : H →ₗ.[ℂ] H) (ψ : T.domain) : H
```

lemma centered_eq [source]

The centered vector, unfolded to its raw expression.

```
lemma centered_eq (T : H →ₗ.[ℂ] H) (ψ : T.domain) :
    centered T ψ = T ψ - (expectedValue T ψ : ℂ) • (ψ : H)
```

lemma centered_eq_zero_iff [source]

A centered vector vanishes exactly when `Tψ = ⟨T⟩_ψ ψ`.

```
lemma centered_eq_zero_iff (T : H →ₗ.[ℂ] H) (ψ : T.domain) :
    centered T ψ = 0 ↔ T ψ = (expectedValue T ψ : ℂ) • (ψ : H)
```

lemma inner_state_centered_eq_zero [source]

For a unit vector and symmetric `T`, the centered vector is orthogonal to the state.

```
lemma inner_state_centered_eq_zero (T : H →ₗ.[ℂ] H) (hT : T.IsSymmetric)
    (ψ : T.domain) (hψ_norm : ‖(ψ : H)‖ = 1) :
    ⟪(ψ : H), centered T ψ⟫_ℂ = 0
```

lemma inner_centered_state_eq_zero [source]

The conjugate orientation of `LinearPMap.inner_state_centered_eq_zero`.

```
lemma inner_centered_state_eq_zero (T : H →ₗ.[ℂ] H) (hT : T.IsSymmetric)
    (ψ : T.domain) (hψ_norm : ‖(ψ : H)‖ = 1) :
    ⟪centered T ψ, (ψ : H)⟫_ℂ = 0
```

2.28 Physlib.QuantumMechanics.DDimensions.Operators.StateObservables.IsEigenvector

Physlib/QuantumMechanics/DDimensions/Operators/StateObservables/IsEigenvector.lean on GitHub.

def IsEigenvector [source]

A nonzero vector in the domain of `T` satisfying `T ψ = μ • ψ`.

```
def IsEigenvector (T : H →ₗ.[ℂ] H) (ψ : T.domain) (μ : ℂ) : Prop
```

lemma apply_eq [source]

The eigenvalue equation for a partial-map eigenvector.

```
lemma IsEigenvector.apply_eq {T : H →ₗ.[ℂ] H} {ψ : T.domain} {μ : ℂ}
    (hψ : T.IsEigenvector ψ μ) :
    T ψ = μ • (ψ : H)
```

lemma ne_zero [source]

A partial-map eigenvector is nonzero.

```
lemma IsEigenvector.ne_zero {T : H →ₗ.[ℂ] H} {ψ : T.domain} {μ : ℂ}
    (hψ : T.IsEigenvector ψ μ) :
    (ψ : H) ≠ 0
```

2.29 Physlib.QuantumMechanics.DDimensions.Operators.StateObservables.Variance

Physlib/QuantumMechanics/DDimensions/Operators/StateObservables/Variance.lean on GitHub.

def variance [source]

Variance `‖Tψ - ⟨T⟩_ψ ψ‖ ^ 2`; only `ψ ∈ T.domain` is required.

```
def variance (T : H →ₗ.[ℂ] H) (ψ : T.domain) : ℝ
```

lemma variance_eq_centered_norm_sq [source]

The variance is the squared norm of the centered vector.

```
lemma variance_eq_centered_norm_sq (T : H →ₗ.[ℂ] H) (ψ : T.domain) :
    variance T ψ = ‖centered T ψ‖ ^ 2
```

lemma variance_eq_norm_sub_sq [source]

`variance` with `centered` unfolded to `Tψ - ⟨T⟩_ψ • ψ`.

```
lemma variance_eq_norm_sub_sq (T : H →ₗ.[ℂ] H) (ψ : T.domain) :
    variance T ψ =
      ‖T ψ - (expectedValue T ψ : ℂ) • (ψ : H)‖ ^ 2
```

lemma variance_eq_norm_sq_sub_expectedValue_sq [source]

For symmetric `T` and `‖ψ‖ = 1`, variance equals `‖Tψ‖ ^ 2 - ⟨T⟩_ψ ^ 2`.

```
lemma variance_eq_norm_sq_sub_expectedValue_sq (T : H →ₗ.[ℂ] H)
    (hT : T.IsSymmetric) (ψ : T.domain) (hψ_norm : ‖(ψ : H)‖ = 1) :
    variance T ψ = ‖T ψ‖ ^ 2 - expectedValue T ψ ^ 2
```

lemma variance_nonneg [source]

Variance is nonnegative.

```
lemma variance_nonneg (T : H →ₗ.[ℂ] H) (ψ : T.domain) :
    0 ≤ variance T ψ
```

lemma variance_eq_zero_iff_centered_eq_zero [source]

Zero variance is the same as a zero centered vector.

```
lemma variance_eq_zero_iff_centered_eq_zero (T : H →ₗ.[ℂ] H) (ψ : T.domain) :
    variance T ψ = 0 ↔ centered T ψ = 0
```

lemma variance_eq_zero_iff [source]

Zero variance is the same as `Tψ = ⟨T⟩_ψ ψ`.

```
lemma variance_eq_zero_iff (T : H →ₗ.[ℂ] H) (ψ : T.domain) :
    variance T ψ = 0 ↔ T ψ = (expectedValue T ψ : ℂ) • (ψ : H)
```

lemma variance_eq_zero_iff_isEigenvector [source]

For `‖ψ‖ = 1`, zero variance iff `ψ` is an eigenvector with eigenvalue `⟨T⟩_ψ`.

```
lemma variance_eq_zero_iff_isEigenvector (T : H →ₗ.[ℂ] H)
    (ψ : T.domain) (hψ_norm : ‖(ψ : H)‖ = 1) :
    variance T ψ = 0 ↔
      T.IsEigenvector ψ (expectedValue T ψ : ℂ)
```

def standardDeviation [source]

Standard deviation `√(variance)` for `ψ ∈ T.domain`.

```
def standardDeviation (T : H →ₗ.[ℂ] H) (ψ : T.domain) : ℝ
```

lemma standardDeviation_eq_sqrt_variance [source]

The standard deviation, unfolded to the square root of the variance.

```
lemma standardDeviation_eq_sqrt_variance (T : H →ₗ.[ℂ] H) (ψ : T.domain) :
    standardDeviation T ψ = Real.sqrt (variance T ψ)
```

lemma standardDeviation_nonneg [source]

Standard deviation is nonnegative.

```
lemma standardDeviation_nonneg (T : H →ₗ.[ℂ] H) (ψ : T.domain) :
    0 ≤ standardDeviation T ψ
```

lemma standardDeviation_sq [source]

```
@[simp]
lemma standardDeviation_sq (T : H →ₗ.[ℂ] H) (ψ : T.domain) :
    standardDeviation T ψ ^ 2 = variance T ψ
```

lemma standardDeviation_eq_zero_iff_centered_eq_zero [source]

Zero standard deviation is the same as a zero centered vector.

```
lemma standardDeviation_eq_zero_iff_centered_eq_zero (T : H →ₗ.[ℂ] H)
    (ψ : T.domain) :
    standardDeviation T ψ = 0 ↔ centered T ψ = 0
```

lemma standardDeviation_eq_zero_iff [source]

Zero standard deviation is the same as `Tψ = ⟨T⟩_ψ ψ`.

```
lemma standardDeviation_eq_zero_iff (T : H →ₗ.[ℂ] H) (ψ : T.domain) :
    standardDeviation T ψ = 0 ↔ T ψ = (expectedValue T ψ : ℂ) • (ψ : H)
```

lemma standardDeviation_eq_zero_iff_isEigenvector [source]

For `‖ψ‖ = 1`, zero standard deviation iff the eigenvector condition holds.

```
lemma standardDeviation_eq_zero_iff_isEigenvector (T : H →ₗ.[ℂ] H)
    (ψ : T.domain) (hψ_norm : ‖(ψ : H)‖ = 1) :
    standardDeviation T ψ = 0 ↔
      T.IsEigenvector ψ (expectedValue T ψ : ℂ)
```

lemma re_inner_apply_sq_eq_norm_sq [source]

For symmetric `T`, `re ⟪ψ, T(Tψ)⟫` is `‖Tψ‖ ^ 2`.

```
lemma re_inner_apply_sq_eq_norm_sq :
    (⟪(ψ : H), T ⟨T ψ, hTψ⟩⟫_ℂ).re = ‖T ψ‖ ^ 2
```

lemma variance_eq_re_inner_sub_expectedValue_sq [source]

When `Tψ ∈ T.domain`, variance equals `⟨T^2⟩_ψ - ⟨T⟩_ψ ^ 2`.

```
lemma variance_eq_re_inner_sub_expectedValue_sq :
    variance T ψ =
      (⟪(ψ : H), T ⟨T ψ, hTψ⟩⟫_ℂ).re - expectedValue T ψ ^ 2
```

2.30 Physlib.QuantumMechanics.DDimensions.Operators.Unbounded

Physlib/QuantumMechanics/DDimensions/Operators/Unbounded.lean on GitHub.

instance instDistribMulAction [source]

```
instance instDistribMulAction : DistribMulAction M (E →ₗ.[R] F) where
```

def LinearPMap.sum [source]

A finite sum of partial linear maps.

`sum f` and `∑ a, f a` are equal, but not by definition. With `sum f` both `domain` and `toFun` are made explicit.

```
def sum : E →ₗ.[R] F where
```

lemma sum_domain [source]

```
lemma sum_domain : (sum f).domain = ⨆ a, (f a).domain
```

lemma sum_domain_le [source]

```
lemma sum_domain_le (a : α) : (sum f).domain ≤ (f a).domain
```

lemma sum_apply [source]

```
@[simp]
lemma sum_apply (ψ : (sum f).domain) : sum f ψ = ∑ a, f a ⟨ψ, sum_domain_le f a ψ.2⟩
```

def compRestricted [source]

`g ∘ᵣ f` is the composition of `g` with `f` restricted to a domain consisting of exactly those `x : f.domain` for which `f x ∈ g.domain`.

```
def compRestricted (g : F →ₗ.[R] G) (f : E →ₗ.[R] F) : E →ₗ.[R] G
```

lemma compRestricted_domain_le [source]

```
lemma compRestricted_domain_le (g : F →ₗ.[R] G) (f : E →ₗ.[R] F) : (g ∘ᵣ f).domain ≤ f.domain
```

lemma mem_compRestricted_domain_iff [source]

```
lemma mem_compRestricted_domain_iff {g : F →ₗ.[R] G} {f : E →ₗ.[R] F} {x : E} :
    x ∈ (g ∘ᵣ f).domain ↔ ∃ h : x ∈ f.domain, f ⟨x, h⟩ ∈ g.domain
```

lemma mem_domain_of_mem_compRestricted_domain [source]

```
lemma mem_domain_of_mem_compRestricted_domain
    {g : F →ₗ.[R] G} {f : E →ₗ.[R] F} (x : (g ∘ᵣ f).domain) : f ⟨x, x.2.2⟩ ∈ g.domain
```

lemma compRestricted_apply [source]

```
@[simp]
lemma compRestricted_apply {g : F →ₗ.[R] G} {f : E →ₗ.[R] F} (x : (g ∘ᵣ f).domain) :
    (g ∘ᵣ f) x = g ⟨f ⟨x, x.2.2⟩, mem_domain_of_mem_compRestricted_domain x⟩
```

lemma compRestricted_zero [source]

The zero map is right-absorbing.

```
@[simp]
lemma compRestricted_zero (g : F →ₗ.[R] G) : g ∘ᵣ (0 : E →ₗ.[R] F) = 0
```

lemma compRestricted_assoc [source]

```
lemma compRestricted_assoc {H : Type*} [AddCommGroup H] [Module R H]
    (f₁ : G →ₗ.[R] H) (f₂ : F →ₗ.[R] G) (f₃ : E →ₗ.[R] F) :
    (f₁ ∘ᵣ f₂) ∘ᵣ f₃ = f₁ ∘ᵣ f₂ ∘ᵣ f₃
```

lemma compRestricted_eq_comp [source]

`compRestricted` is the same as `comp` when the range of `f` is contained in `g.domain`.

```
lemma compRestricted_eq_comp
    {g : F →ₗ.[R] G} {f : E →ₗ.[R] F} (h : ∀ x : f.domain, f x ∈ g.domain) :
    g ∘ᵣ f = g.comp f h
```

lemma comp_le_compRestricted [source]

`compRestricted` is maximal amongst compositions of `g` with domain restrictions of `f`.

```
lemma comp_le_compRestricted {g : F →ₗ.[R] G} {f : E →ₗ.[R] F} {S : Submodule R E}
    (h : ∀ x : (f.domRestrict S).domain, f ⟨x, x.2.2⟩ ∈ g.domain) :
    g.comp (f.domRestrict S) h ≤ g ∘ᵣ f
```

lemma compRestricted_mono_left [source]

```
lemma compRestricted_mono_left {g g' : F →ₗ.[R] G} (h : g ≤ g') (f : E →ₗ.[R] F) :
    g ∘ᵣ f ≤ g' ∘ᵣ f
```

lemma compRestricted_mono_right [source]

```
lemma compRestricted_mono_right (g : F →ₗ.[R] G) {f f' : E →ₗ.[R] F} (h : f ≤ f') :
    g ∘ᵣ f ≤ g ∘ᵣ f'
```

instance instMonoid [source]

```
instance instMonoid : Monoid (E →ₗ.[R] E) where
```

lemma mul_def [source]

```
lemma mul_def (f₁ f₂ : E →ₗ.[R] E) : f₁ * f₂ = f₁ ∘ᵣ f₂
```

lemma one_domain [source]

```
@[simp]
lemma one_domain : (1 : E →ₗ.[R] E).domain = ⊤
```

instance IsScalarTower ℝ ℂ H [source]

```
instance : IsScalarTower ℝ ℂ H
```

def HasDenseDomain [source]

A LinearPMap `U` has dense domain iff `U.domain` is dense in `H`.

```
def HasDenseDomain (U : H →ₗ.[ℂ] H') : Prop
```

lemma hasDenseDomain_def [source]

```
lemma hasDenseDomain_def : U.HasDenseDomain ↔ Dense (U.domain : Set H)
```

def IsUnbounded [source]

A LinearPMap is an unbounded operator iff it has dense domain and is closable.

```
def IsUnbounded (U : H →ₗ.[ℂ] H') : Prop
```

lemma isUnbounded_def [source]

```
lemma isUnbounded_def : U.IsUnbounded ↔ U.HasDenseDomain ∧ U.IsClosable
```

def IsSymmetric [source]

A LinearPMap `U` is symmetric iff `⟪U x, y⟫_ℂ = ⟪x, U y⟫_ℂ` for all `x y : U.domain`.

```
def IsSymmetric (T : H →ₗ.[ℂ] H) : Prop
```

lemma isSymmetric_def [source]

```
lemma isSymmetric_def : T.IsSymmetric ↔ T.IsFormalAdjoint T
```

def IsEssentiallySelfAdjoint [source]

A LinearPMap is essentially self-adjoint iff its closure is self-adjoint.

```
def IsEssentiallySelfAdjoint [CompleteSpace H] (T : H →ₗ.[ℂ] H) : Prop
```

lemma isEssentiallySelfAdjoint_def [source]

```
lemma isEssentiallySelfAdjoint_def [CompleteSpace H] :
    T.IsEssentiallySelfAdjoint ↔ IsSelfAdjoint T.closure
```

lemma isUnbounded_iff_isClosable [source]

```
lemma HasDenseDomain.isUnbounded_iff_isClosable (h : U.HasDenseDomain) :
    U.IsUnbounded ↔ U.IsClosable
```

lemma HasDenseDomain.closure [source]

```
lemma HasDenseDomain.closure (h : U.HasDenseDomain) : U.closure.HasDenseDomain
```

lemma closure_domain_le_domain_closure [source]

```
lemma closure_domain_le_domain_closure (U : H →ₗ.[ℂ] H') : U.closure.domain ≤ U.domain.closure
```

lemma hasDenseDomain_iff_closure_hasDenseDomain [source]

```
lemma hasDenseDomain_iff_closure_hasDenseDomain : U.HasDenseDomain ↔ U.closure.HasDenseDomain
```

lemma HasDenseDomain.neg [source]

```
lemma HasDenseDomain.neg (h : U.HasDenseDomain) : (-U).HasDenseDomain
```

lemma HasDenseDomain.smul [source]

```
lemma HasDenseDomain.smul (h : U.HasDenseDomain) (c : ℂ) : (c • U).HasDenseDomain
```

lemma add_of_le [source]

```
lemma HasDenseDomain.add_of_le (h₁ : U₁.HasDenseDomain) (h_le : U₁.domain ≤ U₂.domain) :
    (U₁ + U₂).HasDenseDomain
```

lemma sub_of_le [source]

```
lemma HasDenseDomain.sub_of_le (h₁ : U₁.HasDenseDomain) (h_le : U₁.domain ≤ U₂.domain) :
    (U₁ - U₂).HasDenseDomain
```

lemma sum_of_le [source]

```
lemma HasDenseDomain.sum_of_le
    {E : Submodule ℂ H} (hE : Dense (E : Set H)) (h : ∀ a, E ≤ (W a).domain) :
    (sum W).HasDenseDomain
```

lemma HasDenseDomain.pow [source]

```
lemma HasDenseDomain.pow
    (h : T.HasDenseDomain) (h_range : ∀ x : T.domain, T x ∈ T.domain) (n : ℕ) :
    (T ^ n).HasDenseDomain
```

lemma pow_hasDenseDomain_of_le [source]

```
lemma pow_hasDenseDomain_of_le
    {n : ℕ} (h : (T ^ n).HasDenseDomain) {k : ℕ} (hle : k ≤ n) : (T ^ k).HasDenseDomain
```

lemma IsClosable.isClosed_iff [source]

```
lemma IsClosable.isClosed_iff (h : U.IsClosable) : U.IsClosed ↔ U.closure = U
```

lemma isClosable_of_exists_dense_formalAdjoint [source]

A LinearPMap with densely-defined formal adjoint is closable.

```
lemma isClosable_of_exists_dense_formalAdjoint [CompleteSpace H] [CompleteSpace H']
    (h : U.HasDenseDomain) (h_fadj : ∃ U' : H' →ₗ.[ℂ] H, U'.HasDenseDomain ∧ U'.IsFormalAdjoint
     U) :
    U.IsClosable
```

lemma isClosable_of_zero [source]

A zero LinearPMap (any domain) is closable.

```
lemma isClosable_of_zero (h_zero : ↥U = 0) : U.IsClosable
```

lemma IsClosable.smul [source]

```
@[aesop safe apply]
lemma IsClosable.smul (h : U.IsClosable) (c : ℂ) : (c • U).IsClosable
```

lemma smul_iff [source]

```
lemma IsClosable.smul_iff {c : ℂ} (hc : c ≠ 0) : (c • U).IsClosable ↔ U.IsClosable
```

lemma neg_eq_neg_one_smul [source]

```
lemma neg_eq_neg_one_smul (U : H →ₗ.[ℂ] H') : -U = (-1 : ℂ) • U
```

lemma IsClosable.neg [source]

```
@[aesop safe apply]
lemma IsClosable.neg (h : U.IsClosable) : (-U).IsClosable
```

lemma closure_smul [source]

```
lemma closure_smul (U : H →ₗ.[ℂ] H') {c : ℂ} (hc : c ≠ 0) : (c • U).closure = c • U.closure
```

lemma adjoint_one [source]

```
@[simp]
lemma adjoint_one [CompleteSpace H] : (1 : H →ₗ.[ℂ] H)† = 1
```

lemma adjoint_of_zero [source]

The adjoint of a zero LinearPMap (any domain) is zero (domain ⊤).

```
lemma adjoint_of_zero [CompleteSpace H] (h_zero : ↥U = 0) : U† = 0
```

lemma adjoint_smul [source]

```
@[simp]
lemma adjoint_smul [CompleteSpace H] (U : H →ₗ.[ℂ] H') {c : ℂ} (hc : c ≠ 0) :
    (c • U)† = conj c • U†
```

lemma adjoint_neg [source]

```
@[simp]
lemma adjoint_neg [CompleteSpace H] (U : H →ₗ.[ℂ] H') : (-U)† = -U†
```

lemma adjoint_antitone [source]

```
lemma adjoint_antitone [CompleteSpace H]
    (h₁₂ : U₁.HasDenseDomain ∨ ¬U₂.HasDenseDomain) (h_le : U₁ ≤ U₂) : U₂† ≤ U₁†
```

lemma adjoint_add_le_add_adjoint [source]

```
lemma adjoint_add_le_add_adjoint [CompleteSpace H]
    (U₁ U₂ : H →ₗ.[ℂ] H') (h₁₂ : (U₁ + U₂).HasDenseDomain) : U₁† + U₂† ≤ (U₁ + U₂)†
```

lemma adjoint_compRestricted_le_compRestricted_adjoint [source]

```
lemma adjoint_compRestricted_le_compRestricted_adjoint [CompleteSpace H] [CompleteSpace H']
    (hV : V.HasDenseDomain) (hVU : (V ∘ᵣ U).HasDenseDomain) : U† ∘ᵣ V† ≤ (V ∘ᵣ U)†
```

lemma adjoint_pow_le_pow_adjoint [source]

```
lemma adjoint_pow_le_pow_adjoint [CompleteSpace H] {n : ℕ} (h : (T ^ n).HasDenseDomain) :
    T† ^ n ≤ (T ^ n)†
```

lemma inner_map_polarization [source]

The analogue of `inner_map_polarization` for LinearPMap.

```
lemma inner_map_polarization (x y : T.domain) :
    ⟪T y, x⟫_ℂ = (⟪T (x + y), ↑(x + y)⟫_ℂ - ⟪T (x - y), ↑(x - y)⟫_ℂ
      + I * ⟪T (x + I • y), ↑(x + I • y)⟫_ℂ - I * ⟪T (x - I • y), ↑(x - I • y)⟫_ℂ) / 4
```

theorem inner_map_polarization' [source]

The analogue of `inner_map_polarization'` for LinearPMap.

```
theorem inner_map_polarization' (x y : T.domain) :
    ⟪T x, y⟫_ℂ = (⟪T (x + y), ↑(x + y)⟫_ℂ - ⟪T (x - y), ↑(x - y)⟫_ℂ
      - I * ⟪T (x + I • y), ↑(x + I • y)⟫_ℂ + I * ⟪T (x - I • y), ↑(x - I • y)⟫_ℂ) / 4
```

lemma isSymmetric_iff_inner_map_self_real [source]

```
lemma isSymmetric_iff_inner_map_self_real :
    T.IsSymmetric ↔ ∀ x : T.domain, conj ⟪T x, x⟫_ℂ = ⟪T x, x⟫_ℂ
```

lemma IsSymmetric.isClosable [source]

```
lemma IsSymmetric.isClosable [CompleteSpace H] (h : T.IsSymmetric) (h' : T.HasDenseDomain) :
    T.IsClosable
```

lemma isUnbounded_iff_hasDenseDomain [source]

```
lemma IsSymmetric.isUnbounded_iff_hasDenseDomain [CompleteSpace H] (h : T.IsSymmetric) :
    T.IsUnbounded ↔ T.HasDenseDomain
```

lemma isSymmetric_iff_le_adjoint [source]

```
lemma isSymmetric_iff_le_adjoint [CompleteSpace H] (h : T.HasDenseDomain) :
    T.IsSymmetric ↔ T ≤ T†
```

lemma isSelfAdjoint_iff [source]

```
lemma IsSymmetric.isSelfAdjoint_iff [CompleteSpace H] (h : T.IsSymmetric) (h' : T.
    HasDenseDomain) :
    IsSelfAdjoint T ↔ T†.domain = T.domain
```

lemma add_adjoint_isSymmetric [source]

```
lemma add_adjoint_isSymmetric [CompleteSpace H] (h : T.HasDenseDomain) :
    (T + T.adjoint).IsSymmetric
```

lemma IsSymmetric.pow [source]

```
@[aesop safe apply]
lemma IsSymmetric.pow (h : T.IsSymmetric) (n : ℕ) : (T ^ n).IsSymmetric
```

lemma IsSymmetric.neg [source]

```
@[aesop safe apply]
lemma IsSymmetric.neg (h : T.IsSymmetric) : (-T).IsSymmetric
```

lemma add [source]

```
@[aesop safe apply]
lemma IsSymmetric.add (h₁ : T₁.IsSymmetric) (h₂ : T₂.IsSymmetric) : (T₁ + T₂).IsSymmetric
```

lemma sub [source]

```
@[aesop safe apply]
lemma IsSymmetric.sub (h₁ : T₁.IsSymmetric) (h₂ : T₂.IsSymmetric) : (T₁ - T₂).IsSymmetric
```

lemma IsSymmetric.smul [source]

```
@[aesop safe apply]
lemma IsSymmetric.smul (h : T.IsSymmetric) {c : ℂ} (hc : conj c = c) : (c • T).IsSymmetric
```

lemma IsSymmetric.real_smul [source]

```
@[aesop safe apply]
lemma IsSymmetric.real_smul (h : T.IsSymmetric) (r : ℝ) : (r • T).IsSymmetric
```

lemma IsSymmetric.sum [source]

```
@[aesop safe apply]
lemma IsSymmetric.sum (h : ∀ a, (S a).IsSymmetric) : (sum S).IsSymmetric
```

lemma of_le [source]

```
lemma IsSymmetric.of_le (h₁ : T₁.IsSymmetric) (h_le : T₂ ≤ T₁) : T₂.IsSymmetric
```

lemma IsSelfAdjoint.isSymmetric [source]

```
lemma IsSelfAdjoint.isSymmetric [CompleteSpace H] (h : IsSelfAdjoint T) : T.IsSymmetric
```

lemma isClosed [source]

```
lemma IsSelfAdjoint.isClosed [CompleteSpace H] (h : IsSelfAdjoint T) : T.IsClosed
```

lemma IsSelfAdjoint.isClosable [source]

```
lemma IsSelfAdjoint.isClosable [CompleteSpace H] (h : IsSelfAdjoint T) : T.IsClosable
```

lemma IsSelfAdjoint.isUnbounded [source]

```
lemma IsSelfAdjoint.isUnbounded [CompleteSpace H] (h : IsSelfAdjoint T) : T.IsUnbounded
```

lemma isEssentiallySelfAdjoint [source]

```
lemma IsSelfAdjoint.isEssentiallySelfAdjoint [CompleteSpace H] (h : IsSelfAdjoint T) :
    T.IsEssentiallySelfAdjoint
```

lemma IsSelfAdjoint.adjoint [source]

```
@[aesop safe apply]
lemma IsSelfAdjoint.adjoint [CompleteSpace H] (h : IsSelfAdjoint T) : IsSelfAdjoint T†
```

lemma IsSelfAdjoint.smul [source]

```
@[aesop safe apply]
lemma IsSelfAdjoint.smul [CompleteSpace H]
    (h : IsSelfAdjoint T) {c : ℂ} (hc : c ≠ 0) (hc' : conj c = c) :
    IsSelfAdjoint (c • T)
```

lemma IsSelfAdjoint.real_smul [source]

```
@[aesop safe apply]
lemma IsSelfAdjoint.real_smul [CompleteSpace H] (h : IsSelfAdjoint T) {r : ℝ} (hr : r ≠ 0) :
    IsSelfAdjoint (r • T)
```

lemma IsSelfAdjoint.neg [source]

```
@[aesop safe apply]
lemma IsSelfAdjoint.neg [CompleteSpace H] (h : IsSelfAdjoint T) : IsSelfAdjoint (-T)
```

lemma IsEssentiallySelfAdjoint.hasDenseDomain [source]

```
lemma IsEssentiallySelfAdjoint.hasDenseDomain [CompleteSpace H] (h : T.IsEssentiallySelfAdjoint
    ) :
    T.HasDenseDomain
```

lemma IsEssentiallySelfAdjoint.isSymmetric [source]

```
lemma IsEssentiallySelfAdjoint.isSymmetric [CompleteSpace H] (h : T.IsEssentiallySelfAdjoint) :
    T.IsSymmetric
```

lemma IsEssentiallySelfAdjoint.isClosable [source]

```
lemma IsEssentiallySelfAdjoint.isClosable [CompleteSpace H] (h : T.IsEssentiallySelfAdjoint) :
    T.IsClosable
```

lemma IsEssentiallySelfAdjoint.isUnbounded [source]

```
lemma IsEssentiallySelfAdjoint.isUnbounded [CompleteSpace H] (h : T.IsEssentiallySelfAdjoint) :
    T.IsUnbounded
```

lemma unique_self_adjoint_extension [source]

The closure is the unique self-adjoint extension of an essentially self-adjoint operator.

```
lemma IsEssentiallySelfAdjoint.unique_self_adjoint_extension [CompleteSpace H]
    (h : T.IsEssentiallySelfAdjoint) {T₂ : H →ₗ.[ℂ] H} (h_le : T ≤ T₂) (h₂ : IsSelfAdjoint T₂) :

    T₂ = T.closure
```

lemma IsEssentiallySelfAdjoint.smul [source]

```
@[aesop safe apply]
lemma IsEssentiallySelfAdjoint.smul [CompleteSpace H]
    (h : T.IsEssentiallySelfAdjoint) {c : ℂ} (hc : c ≠ 0) (hc' : conj c = c) :
    (c • T).IsEssentiallySelfAdjoint
```

lemma IsEssentiallySelfAdjoint.real_smul [source]

```
@[aesop safe apply]
lemma IsEssentiallySelfAdjoint.real_smul [CompleteSpace H]
    (h : T.IsEssentiallySelfAdjoint) {r : ℝ} (hr : r ≠ 0) :
    (r • T).IsEssentiallySelfAdjoint
```

lemma IsEssentiallySelfAdjoint.neg [source]

```
@[aesop safe apply]
lemma IsEssentiallySelfAdjoint.neg [CompleteSpace H] (h : T.IsEssentiallySelfAdjoint) :
    (-T).IsEssentiallySelfAdjoint
```

lemma IsUnbounded.hasDenseDomain [source]

```
lemma IsUnbounded.hasDenseDomain (h : U.IsUnbounded) : U.HasDenseDomain
```

lemma IsUnbounded.isClosable [source]

```
lemma IsUnbounded.isClosable (h : U.IsUnbounded) : U.IsClosable
```

lemma IsUnbounded.adjoint [source]

```
lemma IsUnbounded.adjoint [CompleteSpace H] [CompleteSpace H'] (h : U.IsUnbounded) :
    U†.IsUnbounded
```

lemma IsUnbounded.closure [source]

```
lemma IsUnbounded.closure (h : U.IsUnbounded) : U.closure.IsUnbounded
```

lemma adjoint_closure_eq_adjoint [source]

```
@[simp]
lemma IsUnbounded.adjoint_closure_eq_adjoint [CompleteSpace H] (h : U.IsUnbounded) :
    U.closure† = U†
```

lemma adjoint_adjoint_eq_closure [source]

```
@[simp]
lemma IsUnbounded.adjoint_adjoint_eq_closure [CompleteSpace H] [CompleteSpace H']
    (h : U.IsUnbounded) :
    U†† = U.closure
```

lemma le_adjoint_adjoint [source]

```
lemma IsUnbounded.le_adjoint_adjoint [CompleteSpace H] [CompleteSpace H'] (h : U.IsUnbounded) :
    U ≤ U††
```

lemma IsUnbounded.isClosed_iff [source]

```
lemma IsUnbounded.isClosed_iff [CompleteSpace H] [CompleteSpace H'] (h : U.IsUnbounded) :
    U.IsClosed ↔ U†† = U
```

lemma isUnbounded_of_dense_of_isSymmetric [source]

A LinearPMap constructed from a symmetric LinearMap with dense domain is an unbounded operator.

```
lemma isUnbounded_of_dense_of_isSymmetric [CompleteSpace H] {E : Submodule ℂ H}
    (hE : Dense (E : Set H)) {f : E →ₗ[ℂ] H} (h : ∀ x y : E, ⟪f x, ↑y⟫_ℂ = ⟪↑x, f y⟫_ℂ) :
    (mk E f).IsUnbounded
```

lemma isUnbounded_of_dense_of_isSymmetric' [source]

Variant of `of_dense_of_isSymmetric` for an endomorphism satisfying `LinearMap.IsSymmetric`.

```
lemma isUnbounded_of_dense_of_isSymmetric' [CompleteSpace H]
    {E : Submodule ℂ H} (hE : Dense (E : Set H)) {f : E →ₗ[ℂ] E} (h : f.IsSymmetric) :
    (mk E (E.subtype ∘ₗ f)).IsUnbounded
```

2.31 Physlib.QuantumMechanics.DDimensions.Operators.Uncertainty

Physlib/QuantumMechanics/DDimensions/Operators/Uncertainty.lean on GitHub.

lemma inner_im_of_commutator_eq [source]

```
lemma inner_im_of_commutator_eq {u v : H} {c : ℝ}
    (h_comm : ⟪u, v⟫_ℂ - ⟪v, u⟫_ℂ = Complex.I * c) :
    (⟪u, v⟫_ℂ).im = c / 2
```

lemma inner_norm_sq_eq_re_sq_add_commutator_half_sq [source]

```
lemma inner_norm_sq_eq_re_sq_add_commutator_half_sq {u v : H} {c : ℝ}
    (h_comm : ⟪u, v⟫_ℂ - ⟪v, u⟫_ℂ = Complex.I * c) :
    ‖⟪u, v⟫_ℂ‖ ^ 2 = (⟪u, v⟫_ℂ).re ^ 2 + (c / 2) ^ 2
```

lemma sub_expectation_commutator_eq_raw [source]

```
lemma sub_expectation_commutator_eq_raw
    (ψ a b : H) (μa μb : ℝ)
    (hμa_right : ⟪ψ, a⟫_ℂ = (μa : ℂ)) (hμa_left : ⟪a, ψ⟫_ℂ = (μa : ℂ))
    (hμb_right : ⟪ψ, b⟫_ℂ = (μb : ℂ)) (hμb_left : ⟪b, ψ⟫_ℂ = (μb : ℂ)) (hψ_norm : ‖ψ‖ = 1) :
    ⟪a - (μa : ℂ) • ψ, b - (μb : ℂ) • ψ⟫_ℂ -
        ⟪b - (μb : ℂ) • ψ, a - (μa : ℂ) • ψ⟫_ℂ =
      ⟪a, b⟫_ℂ - ⟪b, a⟫_ℂ
```

lemma raw_commutator_eq_of_symmetric [source]

```
lemma raw_commutator_eq_of_symmetric
    (A B : H →ₗ.[ℂ] H) (hA : A.IsSymmetric) (hB : B.IsSymmetric)
    (ψ : A.domain) (hψB : (ψ : H) ∈ B.domain)
    (hBA : A ψ ∈ B.domain) (hAB : B ⟨ψ, hψB⟩ ∈ A.domain)
    {c : ℝ}
    (h_raw : ⟪(ψ : H), A ⟨B ⟨ψ, hψB⟩, hAB⟩ - B ⟨A ψ, hBA⟩⟫_ℂ = Complex.I * c) :
    ⟪A ψ, B ⟨ψ, hψB⟩⟫_ℂ - ⟪B ⟨ψ, hψB⟩, A ψ⟫_ℂ = Complex.I * c
```

def centeredCommutatorExpectation [source]

The scalar commutator of the centered vectors of `A` and `B` in the state `ψ`.

```
def centeredCommutatorExpectation (A B : H →ₗ.[ℂ] H)
    (ψ : A.domain) (hψB : (ψ : H) ∈ B.domain) : ℂ
```

def rawCommutatorExpectation [source]

The expectation of the raw commutator `[A, B]` in the state `ψ`, with explicit second-order domain witnesses.

```
def rawCommutatorExpectation (A B : H →ₗ.[ℂ] H)
    (ψ : A.domain) (hψB : (ψ : H) ∈ B.domain)
    (hBA : A ψ ∈ B.domain) (hAB : B ⟨ψ, hψB⟩ ∈ A.domain) : ℂ
```

lemma commutator_half_sq_le_mul_norm_sq [source]

```
lemma commutator_half_sq_le_mul_norm_sq {u v : H} {c : ℝ}
    (h_comm : ⟪u, v⟫_ℂ - ⟪v, u⟫_ℂ = Complex.I * c) :
    (|c| / 2) ^ 2 ≤ (‖u‖ * ‖v‖) ^ 2
```

lemma sqrt_mul_le_of_sq_le [source]

```
private lemma sqrt_mul_le_of_sq_le {x y z : ℝ}
    (hx : 0 ≤ x) (hz : 0 ≤ z) (hxy : z ^ 2 ≤ x * y) :
    z ≤ Real.sqrt x * Real.sqrt y
```

lemma state_uncertainty_squared_of_centered_commutator [source]

A centered commutator identity implies the squared Robertson uncertainty bound.

```
lemma state_uncertainty_squared_of_centered_commutator :
    (|c| / 2) ^ 2 ≤ variance A ψ * variance B ⟨ψ, hψB⟩
```

lemma state_uncertainty_squared_with_covariance_of_centered_commutator [source]

A centered commutator identity implies the Robertson–Schrödinger uncertainty bound.

```
lemma state_uncertainty_squared_with_covariance_of_centered_commutator :
    (covariance A B ψ hψB) ^ 2 + (c / 2) ^ 2 ≤
      variance A ψ * variance B ⟨ψ, hψB⟩
```

lemma state_uncertainty_of_centered_commutator [source]

A centered commutator identity implies the standard uncertainty bound.

```
lemma state_uncertainty_of_centered_commutator :
    |c| / 2 ≤ standardDeviation A ψ * standardDeviation B ⟨ψ, hψB⟩
```

lemma inner_centered_commutator_of_raw_commutator [source]

A raw commutator expectation determines the centered commutator expectation.

```
lemma inner_centered_commutator_of_raw_commutator :
    centeredCommutatorExpectation A B ψ hψB = Complex.I * c
```

lemma state_uncertainty_squared_of_raw_commutator [source]

A raw commutator expectation implies the squared Robertson uncertainty bound.

```
lemma state_uncertainty_squared_of_raw_commutator :
    (|c| / 2) ^ 2 ≤ variance A ψ * variance B ⟨ψ, hψB⟩
```

lemma state_uncertainty_squared_with_covariance_of_raw_commutator [source]

A raw commutator expectation implies the squared uncertainty bound with covariance term.

```
lemma state_uncertainty_squared_with_covariance_of_raw_commutator :
    (covariance A B ψ hψB) ^ 2 + (c / 2) ^ 2 ≤
      variance A ψ * variance B ⟨ψ, hψB⟩
```

lemma state_uncertainty_of_raw_commutator [source]

A raw commutator expectation implies the standard uncertainty bound.

```
lemma state_uncertainty_of_raw_commutator :
    |c| / 2 ≤ standardDeviation A ψ * standardDeviation B ⟨ψ, hψB⟩
```

2.32 Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.PolyBddSchwartzSubmodule

Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/PolyBddSchwartzSubmodule.lean on GitHub.

instance CoeOut (polyBddSchwartzMap d a) 𝓢(Space d, ℂ) [source]

```
instance {d : ℕ} {a : ℕ∞} : CoeOut (polyBddSchwartzMap d a) 𝓢(Space d, ℂ)
```

instance CoeFun (polyBddSchwartzMap d a) (fun _ ↦ Space d → ℂ) [source]

```
instance {d : ℕ} {a : ℕ∞} : CoeFun (polyBddSchwartzMap d a) (fun _ ↦ Space d → ℂ)
```

lemma toFun_eq_coe [source]

```
@[simp]
lemma toFun_eq_coe {d : ℕ} {a : ℕ∞} (f : polyBddSchwartzMap d a) (x : Space d) :
    f.val.toFun x = f.val x
```

lemma polyBddSchwartzEquiv_symm_apply_coe [source]

```
lemma polyBddSchwartzEquiv_symm_apply_coe {d : ℕ} {a : ℕ∞}
    {ψ : schwartzSubmodule d} (hψ : ↑ψ ∈ polyBddSchwartzSubmodule d a) :
    (polyBddSchwartzEquiv.symm ⟨ψ, hψ⟩).val = schwartzEquiv.symm ψ
```

lemma polyBddSchwartzEquiv_coe_ae [source]

```
lemma polyBddSchwartzEquiv_coe_ae {d : ℕ} {a : ℕ∞} (f : polyBddSchwartzMap d a) :
    polyBddSchwartzEquiv f =ᵐ[volume] f.val
```

lemma polyBddSchwartzMap_zero_eq_top [source]

```
lemma polyBddSchwartzMap_zero_eq_top (d : ℕ) : polyBddSchwartzMap d 0 = ⊤
```

lemma polyBddSchwartzMap_antitone [source]

```
lemma polyBddSchwartzMap_antitone (d : ℕ) {a b : ℕ∞} (h : a ≤ b) :
    polyBddSchwartzMap d b ≤ polyBddSchwartzMap d a
```

lemma of_zero_eq [source]

```
lemma of_zero_eq (d : ℕ) : polyBddSchwartzSubmodule d 0 = schwartzSubmodule d
```

lemma le_schwartzSubmodule [source]

```
lemma le_schwartzSubmodule (d : ℕ) (a : ℕ∞) : polyBddSchwartzSubmodule d a ≤ schwartzSubmodule
    d
```

lemma antitone [source]

```
lemma antitone (d : ℕ) {a b : ℕ∞} (h : a ≤ b) :
    polyBddSchwartzSubmodule d b ≤ polyBddSchwartzSubmodule d a
```

lemma enorm_bump_mul_le_enorm [source]

```
private lemma enorm_bump_mul_le_enorm {𝕜 E : Type*} [RCLike 𝕜] [NormedAddCommGroup E]
    [NormedSpace ℝ E] [HasContDiffBump E] {c : E} (f : ContDiffBump c) (g : E → 𝕜) (x : E) :
    ‖f x * g x‖ₑ ≤ ‖g x‖ₑ
```

lemma dense_zero_top [source]

```
private lemma dense_zero_top :
    Dense (polyBddSchwartzSubmodule 0 ⊤ : Set (SpaceDHilbertSpace 0))
```

lemma dense_top [source]

```
lemma dense_top (d : ℕ) : Dense (polyBddSchwartzSubmodule d ⊤ : Set (SpaceDHilbertSpace d))
```

lemma dense [source]

```
lemma dense (d : ℕ) (a : ℕ∞) : Dense (polyBddSchwartzSubmodule d a : Set (SpaceDHilbertSpace d)
    )
```

2.33 Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.SchwartzSubmodule

Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/SchwartzSubmodule.lean on GitHub.

instance CoeFun (schwartzSubmodule d) fun _ ↦ Space d → ℂ [source]

```
instance : CoeFun (schwartzSubmodule d) fun _ ↦ Space d → ℂ
```

lemma schwartzEquiv_apply_coe [source]

```
lemma schwartzEquiv_apply_coe : ↑(schwartzEquiv f) = schwartzIncl f
```

lemma schwartzEquiv_coe_ae [source]

```
lemma schwartzEquiv_coe_ae : schwartzEquiv f =ᵐ[volume] f
```

lemma schwartzEquiv_symm_coe_ae [source]

```
lemma schwartzEquiv_symm_coe_ae : schwartzEquiv.symm ψ =ᵐ[volume] ψ
```

lemma schwartzEquiv_ae_eq [source]

```
lemma schwartzEquiv_ae_eq (h : schwartzEquiv f =ᵐ[volume] schwartzEquiv g) : f = g
```

lemma zero_eq_top [source]

```
@[simp]
lemma zero_eq_top : schwartzSubmodule 0 = ⊤
```

lemma dense [source]

```
lemma dense (d : ℕ) : Dense (schwartzSubmodule d : Set (SpaceDHilbertSpace d))
```

lemma schwartzEquiv_inner [source]

```
lemma schwartzEquiv_inner :
    ⟪schwartzEquiv f, schwartzEquiv g⟫_ℂ = ∫ x : Space d, starRingEnd ℂ (f x) * g x
```

lemma schwartzIncl_ker [source]

```
lemma schwartzIncl_ker : schwartzIncl.ker = (⊥ : Submodule ℂ 𝓢(Space d, ℂ))
```

2.34 Physlib.Relativity.LorentzAlgebra.Basis

Physlib/Relativity/LorentzAlgebra/Basis.lean on GitHub.

lemma boostGenerator_transpose [source]

The boost generators are symmetric.

```
@[simp]
lemma boostGenerator_transpose (i : Fin 3) :
    (boostGenerator i)ᵀ = boostGenerator i
```

lemma boostGenerator_trace [source]

The boost generators are traceless.

```
@[simp]
lemma boostGenerator_trace (i : Fin 3) :
    Matrix.trace (boostGenerator i) = 0
```

lemma rotationGenerator_transpose [source]

The rotation generators are antisymmetric.

```
@[simp]
lemma rotationGenerator_transpose (i : Fin 3) :
    (rotationGenerator i)ᵀ = -rotationGenerator i
```

lemma rotationGenerator_trace [source]

The rotation generators are traceless.

```
@[simp]
lemma rotationGenerator_trace (i : Fin 3) :
    Matrix.trace (rotationGenerator i) = 0
```

2.35 Physlib.Relativity.Tensors.Basic

Physlib/Relativity/Tensors/Basic.lean on GitHub.

lemma succSuccAbove [source]

```
lemma PermCond.succSuccAbove {n n1 : ℕ} {c : Fin (n + 1 + 1) → C}
    {c1 : Fin (n1 + 1 + 1) → C}
    (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j)
    {σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)} (hσ : PermCond c c1 σ) :
    PermCond (c ∘ succSuccAbove (σ i) (σ j))
      (c1 ∘ succSuccAbove i j) (funPredPredAbove i j hij σ hσ.1)
```

lemma succSuccAbove_comm [source]

```
lemma PermCond.succSuccAbove_comm {n : ℕ} {c : Fin (n + 1 + 1 + 1 + 1) → C}
    (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))
    (hij1 : i1 ≠ j1) (hij2 : i2 ≠ j2) :
    let i2'
```

lemma append_right_succSuccAbove [source]

```
lemma PermCond.append_right_succSuccAbove {n n1 : ℕ} {c : Fin (n + 1 + 1) → C}
    {c1 : Fin n1 → C} (i j : Fin (n + 1 + 1)) :
    PermCond (Fin.append c1 c ∘ (Fin.succSuccAbove (finSumFinEquiv (m := n1) (Sum.inr i))
        (finSumFinEquiv (m := n1) (Sum.inr j))))
      (Fin.append c1 (c ∘ (i.succSuccAbove j))) id
```

lemma permT_basis [source]

```
lemma permT_basis {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    {σ : Fin m → Fin n} (h : PermCond c c1 σ)
    (b : ComponentIdx c) :
```

```
    (permT σ h) (basis (S := S) c b) = basis c1 (fun i =>
      basisIdxCongr (by simp [h.2]) (b (σ i)))
```

lemma toField_permT [source]

```
lemma toField_permT {c c1 : Fin 0 → C} (σ : Fin 0 → Fin 0) (h : PermCond c c1 σ) (t : S.Tensor
    c) :
    toField (permT σ h t) = toField t
```

2.36 Physlib.Relativity.Tensors.Contraction.Basis

Physlib/Relativity/Tensors/Contraction/Basis.lean on GitHub.

lemma mem_iff_apply_succSuccAbove_eq [source]

```
lemma mem_iff_apply_succSuccAbove_eq {n : ℕ} {c : Fin (n + 1 + 1) → C}
    {i j : Fin (n + 1 + 1)}
    (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
    (b' : ComponentIdx c) :
    b' ∈ DropPairSection (S := S) b ↔
      ∀ m, b' (Fin.succSuccAbove i j m) = b m
```

lemma ofFin_mem_succSuccAboveSection [source]

```
lemma ofFin_mem_succSuccAboveSection {n : ℕ} {c : Fin (n + 1 + 1) → C}
    {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
    (x : basisIdx (c i) × basisIdx (c j)) :
    ofFin (S := S) hij b x ∈ DropPairSection b
```

lemma contrT_basis [source]

```
lemma contrT_basis {n : ℕ} {c : Fin (n + 1 + 1) → C} {i j : Fin (n + 1 + 1)}
    (h : i ≠ j ∧ S.τ (c i) = c j) (b : ComponentIdx (S := S) c):
    contrT n i j h (basis c b) =
    Pure.contrPCoeff i j h (Pure.basisVector c b) •
      basis (c ∘ Fin.succSuccAbove i j) (b.dropPair i j)
```

2.37 Physlib.Relativity.Tensors.Contraction.Pure

Physlib/Relativity/Tensors/Contraction/Pure.lean on GitHub.

lemma dropPair_update_succSuccAbove [source]

```
@[simp]
lemma dropPair_update_succSuccAbove {n : ℕ} [inst : DecidableEq (Fin (n + 1 +1))]
    {c : Fin (n + 1 + 1) → C}
    (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (p : Pure S c)
    (m : Fin n)
    (x : S.FD.obj (Discrete.mk (c (succSuccAbove i j m)))) :
    dropPair i j hij (p.update (succSuccAbove i j m) x) =
    (dropPair i j hij p).update m x
```

lemma contrPCoeff_update_succSuccAbove [source]

```
@[simp]
lemma contrPCoeff_update_succSuccAbove {n : ℕ} [inst : DecidableEq (Fin (n + 1 +1))]
    {c : Fin (n + 1 + 1) → C}
    (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ S.τ (c i) = c j) (m : Fin n)
    (p : Pure S c) (x : S.FD.obj (Discrete.mk (c (succSuccAbove i j m)))) :
    contrPCoeff i j hij (p.update (succSuccAbove i j m) x) =
    contrPCoeff i j hij p
```

2.38 Physlib.Relativity.Tensors.Contraction.SuccSuccAbove

Physlib/Relativity/Tensors/Contraction/SuccSuccAbove.lean on GitHub.

def succSuccAbove [source]

The embedding of `Fin n` into `Fin (n + 1 + 1)` which leaves a hole at `i` and `j`.

```
def succSuccAbove (i j : Fin (n + 1 + 1)) (m : Fin n) : Fin (n + 1 + 1)
```

lemma succSuccAbove_val [source]

```
lemma succSuccAbove_val (i j : Fin (n + 1 + 1)) (m : Fin n) :
    (succSuccAbove i j m).val = if m.1 < i.1 ∧ m.1 < j.1 then m.1
    else if m.1 + 1 < i.1 ∧ j.1 ≤ m.1 then m.1 + 1
    else if i.1 ≤ m.1 ∧ m.1 + 1 < j.1 then m.1 + 1
    else m.1 + 2
```

lemma succSuccAbove_self_apply [source]

```
lemma succSuccAbove_self_apply (i : Fin (n + 1 + 1)) (m : Fin n) :
    succSuccAbove i i m = if m.1 < i.1 then ⟨m.1, by omega⟩ else ⟨m.1 + 2, by omega⟩
```

lemma succSuccAbove_eq_succAbove_succAbove [source]

```
lemma succSuccAbove_eq_succAbove_succAbove (i : Fin (n + 1 + 1)) (j : Fin (n + 1)) :
    succSuccAbove i (i.succAbove j) = i.succAbove ∘ j.succAbove
```

lemma succSuccAbove_eq_predAbove [source]

```
lemma succSuccAbove_eq_predAbove {i j : Fin (n + 1 + 1)} (hij : i ≠ j) :
    succSuccAbove i j = fun x => i.succAbove (((Fin.predAbove 0 i).predAbove j).succAbove x)
```

lemma succSuccAbove_injective [source]

```
lemma succSuccAbove_injective {n : ℕ}
    (i j : Fin (n + 1 + 1)) : Function.Injective (succSuccAbove i j)
```

lemma succSuccAbove_eq_iff_eq [source]

```
@[simp]
lemma succSuccAbove_eq_iff_eq {n : ℕ}
    (i j : Fin (n + 1 + 1)) (m1 m2 : Fin n) :
```

```
    succSuccAbove i j m1 = succSuccAbove i j m2 ↔ m1 = m2
```

lemma succSuccAbove_leq_iff_leq [source]

```
@[simp]
lemma succSuccAbove_leq_iff_leq {n : ℕ}
    (i j : Fin (n + 1 + 1)) (m1 m2 : Fin n) :
    succSuccAbove i j m1 ≤ succSuccAbove i j m2 ↔ m1 ≤ m2
```

lemma succSuccAbove_lt_iff_lt [source]

```
@[simp]
lemma succSuccAbove_lt_iff_lt {n : ℕ}
    (i j : Fin (n + 1 + 1)) (m1 m2 : Fin n) :
    succSuccAbove i j m1 < succSuccAbove i j m2 ↔ m1 < m2
```

lemma succSuccAbove_monotone [source]

```
@[simp]
lemma succSuccAbove_monotone {n : ℕ} (i j : Fin (n + 1 + 1)) :
    Monotone (succSuccAbove i j)
```

lemma succSuccAbove_strictMono [source]

```
lemma succSuccAbove_strictMono {n : ℕ} (i j : Fin (n + 1 + 1)) :
    StrictMono (succSuccAbove i j)
```

lemma succSuccAbove_range [source]

```
@[simp]
lemma succSuccAbove_range {i j : Fin (n + 1 + 1)} (hij : i ≠ j) :
    Set.range (succSuccAbove i j) = {i, j}ᶜ
```

lemma succSuccAbove_eq_orderEmbOfFin [source]

```
lemma succSuccAbove_eq_orderEmbOfFin {n : ℕ}
    (i j : Fin (n + 1 + 1)) (hij : i ≠ j) :
    succSuccAbove i j = Finset.orderEmbOfFin {i, j}ᶜ
    (by rw [Finset.card_compl]; simp [Finset.card_pair hij])
```

lemma succSuccAbove_symm [source]

```
lemma succSuccAbove_symm (i j : Fin (n + 1 + 1)) :
    succSuccAbove i j = succSuccAbove j i
```

lemma permCond_succSuccAbove_symm [source]

```
@[simp, nolint simpVarHead]
lemma permCond_succSuccAbove_symm {c : Fin (n + 1 + 1) → C} (i j : Fin (n + 1 + 1))
    (k : Fin n) : c (succSuccAbove i j k) = c (succSuccAbove j i k)
```

lemma succSuccAbove_apply_eq_orderIsoOfFin [source]

```
lemma succSuccAbove_apply_eq_orderIsoOfFin {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (m : Fin n) :
    (succSuccAbove i j) m = (Finset.orderIsoOfFin {i, j}ᶜ
      (by rw [Finset.card_compl]; simp [Finset.card_pair hij])) m
```

lemma succSuccAbove_image_compl [source]

```
lemma succSuccAbove_image_compl {i j : Fin (n + 1 + 1)} (hij : i ≠ j)
    (X : Set (Fin n)) :
    (succSuccAbove i j) '' Xᶜ = ({i, j} ∪ succSuccAbove i j '' X)ᶜ
```

lemma fst_ne_succSuccAbove_pre [source]

```
@[simp]
lemma fst_ne_succSuccAbove_pre (i j : Fin (n + 1 + 1)) (m : Fin n) :
    ¬ i = succSuccAbove i j m
```

lemma succSuccAbove_ne_fst [source]

```
@[simp]
lemma succSuccAbove_ne_fst (i j : Fin (n + 1 + 1)) (m : Fin n) :
    ¬ succSuccAbove i j m = i
```

lemma snd_ne_succSuccAbove_pre [source]

```
@[simp]
lemma snd_ne_succSuccAbove_pre (i j : Fin (n + 1 + 1)) (m : Fin n) :
    ¬ j = (succSuccAbove i j) m
```

lemma succSuccAbove_ne_snd [source]

```
@[simp]
lemma succSuccAbove_ne_snd (i j : Fin (n + 1 + 1)) (m : Fin n) :
    ¬ succSuccAbove i j m = j
```

lemma eq_or_exists_succSuccAbove [source]

```
lemma eq_or_exists_succSuccAbove(i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin (n + 1 + 1)) :
    m = i ∨ m = j ∨ ∃ m', m = succSuccAbove i j m'
```

lemma succSuccAbove_apply_lt_lt [source]

```
lemma succSuccAbove_apply_lt_lt {n : ℕ}
    (i j : Fin (n + 1 + 1)) (m : Fin n) (hi : m.val < i.val) (hj : m.val < j.val) :
    succSuccAbove i j m = m.castSucc.castSucc
```

lemma succSuccAbove_natAdd_apply_castAdd [source]

```
lemma succSuccAbove_natAdd_apply_castAdd {n n1 : ℕ}
    (i j : Fin (n + 1 + 1)) (m : Fin n1) :
    (succSuccAbove (n := n1 + n) (Fin.natAdd n1 i) (Fin.natAdd n1 j))
```

```
    (Fin.castAdd n m) = Fin.castAdd (n + 1 + 1) (m)
```

lemma succSuccAbove_natAdd_image_range_castAdd [source]

```
lemma succSuccAbove_natAdd_image_range_castAdd {n n1 : ℕ}
    (i j : Fin (n + 1 + 1)) :
    (succSuccAbove (n := n1 + n) (Fin.natAdd n1 i) (Fin.natAdd n1 j)) ''
    (Set.range (Fin.castAdd (m := n) (n := n1))) = {i | i.1 < n1}
```

lemma succSuccAbove_comm_natAdd [source]

```
lemma succSuccAbove_comm_natAdd {n n1 : ℕ}
    (i j : Fin (n + 1 + 1)) (m : Fin n) :
    succSuccAbove (n := n1 + n) (Fin.natAdd n1 i) (Fin.natAdd n1 j) (Fin.natAdd n1 m)
    = Fin.natAdd (n1) (succSuccAbove i j m)
```

def predPredAbove [source]

The preimage of `m` under `succSuccAbove i j hij` given that `m` is not equal to `i` or `j`.

```
def predPredAbove (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin (n + 1 + 1))
    (hm : m ≠ i ∧ m ≠ j) : Fin n
```

lemma predPredAbove_val [source]

```
lemma predPredAbove_val (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin (n + 1 + 1))
    (hm : m ≠ i ∧ m ≠ j) :
    (predPredAbove i j hij m hm).val = if m.1 < i.1 ∧ m.1 < j.1 then m.1
    else if m.1 - 1 < i.1 ∧ j.1 ≤ m.1 then m.1 - 1
    else if i.1 - 1 ≤ m.1 ∧ m.1 < j.1 then m.1 - 1
    else m.1 - 2
```

lemma succSuccAbove_predPredAbove [source]

```
@[simp]
lemma succSuccAbove_predPredAbove (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin (n + 1 + 1))
    (hm : m ≠ i ∧ m ≠ j) :
    succSuccAbove i j (predPredAbove i j hij m hm) = m
```

lemma predPredAbove_eq_orderIsoOfFin [source]

```
lemma predPredAbove_eq_orderIsoOfFin (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin (n + 1 + 1)
    )
    (hm : m ≠ i ∧ m ≠ j) :
    predPredAbove i j hij m hm =
    (Finset.orderIsoOfFin {i, j}ᶜ (by rw [Finset.card_compl]; simp [Finset.card_pair hij])).
    symm
    ⟨m, by simp [hm]⟩
```

lemma predPredAbove_injective [source]

```
@[simp]
lemma predPredAbove_injective (i j : Fin (n + 1 + 1)) (hij : i ≠ j)
```

```
    (m1 m2 : Fin (n + 1 + 1)) (hm1 : m1 ≠ i ∧ m1 ≠ j) (hm2 : m2 ≠ i ∧ m2 ≠ j) :
    predPredAbove i j hij m1 hm1 = predPredAbove i j hij m2 hm2 ↔ m1 = m2
```

lemma predPredAbove_surjective [source]

```
lemma predPredAbove_surjective (i j : Fin (n + 1 + 1)) (hij : i ≠ j)
    (m : Fin n) : ∃ m' : Fin (n + 1 + 1), ∃ (h : m' ≠ i ∧ m' ≠ j),
    predPredAbove i j hij m' h = m
```

lemma predPredAbove_succSuccAbove [source]

```
@[simp]
lemma predPredAbove_succSuccAbove (i j : Fin (n + 1 + 1)) (hij : i ≠ j)
    (m : Fin n) :
    predPredAbove i j hij (succSuccAbove i j m) (by simp) = m
```

lemma succSuccAbove_comm [source]

```
lemma succSuccAbove_comm (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))
    (hij1 : i1 ≠ j1) (hij2 : i2 ≠ j2) :
    let i2'
```

lemma succSuccAbove_comm_apply [source]

```
lemma succSuccAbove_comm_apply (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))
    (hij1 : i1 ≠ j1) (hij2 : i2 ≠ j2) (m : Fin n) :
    let i2'
```

def funPredPredAbove [source]

Given a bijection `Fin (n1 + 1 + 1) → Fin (n + 1 + 1))` and a pair `i j : Fin (n1 + 1 + 1)`, then `funPredPredAbove i j _ σ _ : Fin n1 → Fin n` corresponds to the induced bijection formed by dropping `i` and `j` in the source and their image in the target.

```
def funPredPredAbove {n n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j)
    (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : Function.Bijective σ)
    (m : Fin n1) : Fin n
```

lemma funPredPredAbove_injective [source]

```
lemma funPredPredAbove_injective {n n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j)
    (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : Function.Bijective σ) :
    Function.Injective (funPredPredAbove i j hij σ hσ)
```

lemma funPredPredAbove_surjective [source]

```
lemma funPredPredAbove_surjective {n n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j)
    (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : Function.Bijective σ) :
    Function.Surjective (funPredPredAbove i j hij σ hσ)
```

lemma funPredPredAbove_bijective [source]

```
lemma funPredPredAbove_bijective {n n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j)
    (σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : Function.Bijective σ) :
    Function.Bijective (funPredPredAbove i j hij σ hσ)
```

lemma funPredPredAbove_id [source]

```
@[simp]
lemma funPredPredAbove_id { n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j) :
    funPredPredAbove i j hij id (Function.bijective_id) = id
```

2.39 Physlib.Relativity.Tensors.Evaluation

Physlib/Relativity/Tensors/Evaluation.lean on GitHub.

lemma evalPCoeff_basisVector [source]

```
lemma evalPCoeff_basisVector (i : Fin (n + 1)) (b : basisIdx (c i)) (b' : ComponentIdx (S := S)
     c) :
    evalPCoeff i b (Pure.basisVector c b') = if b' i = b then (1 : k) else 0
```

lemma evalT_basis [source]

```
lemma evalT_basis {n : ℕ} {c : Fin (n + 1) → C} (i : Fin (n + 1))
    (b : ComponentIdx c) (x : basisIdx (c i)) :
    evalT i x (basis (S := S) c b) = if b i = x then basis (c ∘ i.succAbove)
      (fun j => b (i.succAbove j)) else 0
```

2.40 Physlib.Relativity.Tensors.RealTensor.Basic

Physlib/Relativity/Tensors/RealTensor/Basic.lean on GitHub.

lemma basisIdxCongr_eq_refl [source]

```
@[simp]
lemma basisIdxCongr_eq_refl {d : ℕ} {c1 c2 : realLorentzTensor.Color} (h : c1 = c2) :
    TensorSpecies.basisIdxCongr (basisIdx := fun _ => Fin 1 ⊕ Fin d) h = Equiv.refl _
```

lemma contrPCoeff_basis [source]

```
lemma contrPCoeff_basis {d n : ℕ} {c : Fin n → realLorentzTensor.Color} (i j : Fin n)
    (hij : i ≠ j ∧ (realLorentzTensor d).τ (c i) = c j)
    (b : ComponentIdx (S := realLorentzTensor d) c) :
    Pure.contrPCoeff i j hij (Pure.basisVector c b) = if b i = b j then 1 else 0
```

lemma contrT_eq_sum_evalT [source]

```
lemma contrT_eq_sum_evalT {n} {d} (c : Fin (n + 1 + 1) → Color) (i j : Fin (n + 1 + 1))
    (h : i ≠ j ∧ (realLorentzTensor d).τ (c i) = c j) (t : ℝT(d, c)) :
    contrT n i j h t =  ∑ (μ : Fin 1 ⊕ Fin d), permT id (by
      simp [Fin.succSuccAbove_eq_predAbove h.1])
     (evalT ((Fin.predAbove 0 i).predAbove j) μ (evalT i μ t))
```

lemma contrT_toField [source]

```
lemma contrT_toField {d} (c : Fin 2 → Color)
    (h : 0 ≠ 1 ∧ (realLorentzTensor d).τ (c 0) = c 1) (t : ℝT(d, c)) :
    (contrT 0 0 1 h t).toField = ∑ (μ : Fin 1 ⊕ Fin d), {t | [μ] [μ]}ᵀ.toField
```

2.41 Physlib.Relativity.Tensors.RealTensor.ToComplex

Physlib/Relativity/Tensors/RealTensor/ToComplex.lean on GitHub.

lemma complexify_comp_succSuccAbove [source]

`complexify` commutes with precomposition by `succSuccAbove`. We use `fun k => b (Fin.succSuccAbove i j k)` and direct application `(ComponentIdx.complexify b) (Fin.succSuccAbove i j m)` rather than composition so that dependent `ComponentIdx` types unify correctly (avoiding `Function.comp` type mismatch).

```
@[simp]
lemma ComponentIdx.complexify_comp_succSuccAbove
    {n : ℕ} {c : Fin (n + 1 + 1) → realLorentzTensor.Color}
    {i j : Fin (n + 1 + 1)} (b : ComponentIdx (S := realLorentzTensor) c) (m : Fin n) :
    (ComponentIdx.complexify (c := c ∘ Fin.succSuccAbove i j)
      (fun k => b (Fin.succSuccAbove i j k))) m =
    (ComponentIdx.complexify (c := c) b) (Fin.succSuccAbove i j m)
```

2.42 Physlib.Relativity.Tensors.RealTensor.Vector.Basic

Physlib/Relativity/Tensors/RealTensor/Vector/Basic.lean on GitHub.

lemma ext_of_apply [source]

```
lemma ext_of_apply {d} {v w : Vector d} (h : ∀ i, v i = w i) : v = w
```

def ofTemporalComponent [source]

The continuous linear map corresponding to the creation of a Lorentz Vector with only a non-zero temporal component.

```
def ofTemporalComponent {d : ℕ} : ℝ →L[ℝ] Vector d where
```

2.43 Physlib.Relativity.Tensors.Tensorial

Physlib/Relativity/Tensors/Tensorial.lean on GitHub.

lemma prod_basis_of_map_reindex [source]

```
lemma prod_basis_of_map_reindex {n2 : ℕ} {c2 : Fin n2 → C} {M₂ : Type}
    [Tensorial S c M] [AddCommMonoid M₂] [Module k M₂]
    [Tensorial S c2 M₂] {ι ι2 : Type}  {b : Module.Basis ι k M} {b2 : Module.Basis ι2 k M₂}
    {e : Tensor.ComponentIdx c ≃ ι} {e2 : Tensor.ComponentIdx c2 ≃ ι2}
    (h : b = ((Tensor.basis (S := S) c).map toTensor.symm).reindex e)
    (h2 : b2 = ((Tensor.basis (S := S) c2).map  toTensor.symm).reindex e2) :
    b.tensorProduct b2 = ((Tensor.basis (S := S) (Fin.append c c2)).map
    (toTensor (S := S) (M := M ⊗[k] M₂)).symm).reindex
    (ComponentIdx.prod.trans (e.prodCongr e2))
```

lemma prod_tensor_basis_eq_map_reindex [source]

```
lemma prod_tensor_basis_eq_map_reindex {n2 : ℕ} {c2 : Fin n2 → C} {M₂ : Type}
    [Tensorial S c M] [AddCommMonoid M₂] [Module k M₂]
    [Tensorial S c2 M₂] {ι ι2 : Type}  {b : Module.Basis ι k M} {b2 : Module.Basis ι2 k M₂}
    {e : Tensor.ComponentIdx c ≃ ι} {e2 : Tensor.ComponentIdx c2 ≃ ι2}
    (h : b = ((Tensor.basis (S := S) c).map toTensor.symm).reindex e)
    (h2 : b2 = ((Tensor.basis (S := S) c2).map  toTensor.symm).reindex e2) :
    Tensor.basis (S := S) (Fin.append c c2) =
    ((b.tensorProduct b2).map (toTensor (S := S) (M := M ⊗[k] M₂))).reindex
    (ComponentIdx.prod.trans (e.prodCongr e2)).symm
```

2.44 Physlib.Relativity.Tensors.TensorSpecies.Basic

Physlib/Relativity/Tensors/TensorSpecies/Basic.lean on GitHub.

def ofFunctions [source]

The creation of an element of `TensorSpecies` from functions rather than categorical objects.

```
def ofFunctions [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
```

2.45 Physlib.SpaceAndTime.Space.Derivatives.Curl

Physlib/SpaceAndTime/Space/Derivatives/Curl.lean on GitHub.

lemma eq_neg_curl_of_div_zero [source]

```
lemma eq_neg_curl_of_div_zero (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ 1 f)
    (hdiv : ∇ · f = 0) :
    f = - curl (fun x => ∫ (t : ℝ) in 0..1, (t • basis.repr x) ×₃ f (t • x) ∂(volume))
```

lemma eq_grad_integral_of_curl_zero [source]

A constructive form of the statement that if the curl of a function is zero, then it is equal to the grad of another function.

In the context of e.g. electromagnetism the potential given by this lemma corresponds to that defined by the work done to move a unit charge from the origin to the point `x` along a straight line.

```
lemma eq_grad_integral_of_curl_zero (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ 1 f
    )
    (hcurl : curl f = 0) :
    f = grad (fun x => ∫ t in (0 : ℝ)..1, ⟪f (t • x), basis.repr x⟫_ℝ ∂(volume))
```

2.46 Physlib.SpaceAndTime.Space.Derivatives.DerivativeIndex

Physlib/SpaceAndTime/Space/Derivatives/DerivativeIndex.lean on GitHub.

abbrev DerivativeIndex [source]

The multi-indices on `d` coordinates of order at most `k`.

```
abbrev DerivativeIndex (d k : ℕ)
```

instance Fintype (DerivativeIndex d k) [source]

```
noncomputable instance (d k : ℕ) : Fintype (DerivativeIndex d k)
```

instance DecidableEq (DerivativeIndex d k) [source]

```
instance (d k : ℕ) : DecidableEq (DerivativeIndex d k)
```

instance Zero (DerivativeIndex d k) [source]

```
instance (d k : ℕ) : Zero (DerivativeIndex d k) where
```

lemma coe_zero [source]

```
@[simp]
lemma coe_zero (d k : ℕ) :
    ((0 : DerivativeIndex d k) : MultiIndex d) = 0
```

lemma zero_apply [source]

```
@[simp]
lemma zero_apply (d k : ℕ) (i : Fin d) :
    ((0 : DerivativeIndex d k) : MultiIndex d) i = 0
```

2.47 Physlib.SpaceAndTime.Space.Derivatives.Grad

Physlib/SpaceAndTime/Space/Derivatives/Grad.lean on GitHub.

lemma grad_fun_add_const [source]

```
@[simp]
lemma grad_fun_add_const (f : Space d → ℝ) (c : ℝ) :
    ∇ (fun x => f x + c) = ∇ f
```

lemma grad_smul_inner_space [source]

```
lemma grad_smul_inner_space {d} (x : Space d) (f : Space d → ℝ) (hd : Differentiable ℝ f) (t :
    ℝ)
    (ht : 0 < t) :
    ⟪∇ f (t • x), basis.repr x⟫_ℝ =  _root_.deriv (fun r => f (r • x)) t
```

lemma eq_integral_grad [source]

```
lemma eq_integral_grad {f : Space → ℝ} (hf : ContDiff ℝ 1 f) :
    f = (fun x => ∫ t in (0 : ℝ)..1, ⟪∇ f (t • x), basis.repr x⟫_ℝ ∂(volume)
      + f 0)
```

2.48 Physlib.SpaceAndTime.Space.Derivatives.MatrixDiv

Physlib/SpaceAndTime/Space/Derivatives/MatrixDiv.lean on GitHub.

def matrixDiv [source]

The divergence of a matrix-valued spatial field.

For a field `T : Space d → Matrix (Fin d) (Fin d) ℝ`, `matrixDiv T` is the vector field whose `i`th component is

`∑ j, ∂[j] (fun x => T x i j) x`.

```
noncomputable def matrixDiv (d : ℕ) (T : Space d → Matrix (Fin d) (Fin d) ℝ) :
    Space d → EuclideanSpace ℝ (Fin d)
```

lemma matrixDiv_apply [source]

```
@[simp]
lemma matrixDiv_apply (d : ℕ) (T : Space d → Matrix (Fin d) (Fin d) ℝ)
    (x : Space d) (i : Fin d) :
    matrixDiv d T x i = ∑ j, ∂[j] (fun x => T x i j) x
```

lemma matrixDiv_zero [source]

```
@[simp]
lemma matrixDiv_zero (d : ℕ) :
    matrixDiv d (0 : Space d → Matrix (Fin d) (Fin d) ℝ) = 0
```

lemma matrixDiv_const [source]

```
@[simp]
lemma matrixDiv_const (d : ℕ) (T : Matrix (Fin d) (Fin d) ℝ) :
    matrixDiv d (fun _ : Space d => T) = 0
```

lemma matrixDiv_add [source]

```
lemma matrixDiv_add (d : ℕ) (T1 T2 : Space d → Matrix (Fin d) (Fin d) ℝ)
    (hT1 : Differentiable ℝ T1) (hT2 : Differentiable ℝ T2) :
    matrixDiv d (T1 + T2) = matrixDiv d T1 + matrixDiv d T2
```

lemma matrixDiv_smul [source]

```
lemma matrixDiv_smul (d : ℕ) (T : Space d → Matrix (Fin d) (Fin d) ℝ) (k : ℝ)
    (hT : Differentiable ℝ T) :
    matrixDiv d (k • T) = k • matrixDiv d T
```

2.49 Physlib.SpaceAndTime.Space.Norm

Physlib/SpaceAndTime/Space/Norm.lean on GitHub.

lemma distDiv_inv_pow_eq_dim' [source]

Auxiliary lemma with dimension defined as d.succ to handle `homeomorphUnitSpherePro`d. The dimension correct version is declared in `distDiv_inv_pow_eq_dim`.

```
private lemma distDiv_inv_pow_eq_dim' {d : ℕ} :
    distDiv (distOfFunction (fun x : Space d.succ => ‖x‖ ^ (- d.succ : ℤ) • basis.repr x)
      (IsDistBounded.zpow_smul_repr_self (- d.succ : ℤ) (by omega))) =
      (d.succ * (volume (α := Space d.succ)).real (Metric.ball 0 1)) • diracDelta ℝ 0
```

2.50 Physlib.SpaceAndTime.SpaceTime.TimeSlice

Physlib/SpaceAndTime/SpaceTime/TimeSlice.lean on GitHub.

lemma timeSlice_symm_contDiff [source]

```
@[fun_prop]
lemma timeSlice_symm_contDiff {d : ℕ} {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {n} (c : SpeedOfLight) (f : Time → Space d → M) (h : ContDiff ℝ n ↿f) :
    ContDiff ℝ n ((timeSlice c).symm f)
```

lemma timeSlice_symm_differentiable [source]

```
@[fun_prop]
lemma timeSlice_symm_differentiable {d : ℕ} {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (c : SpeedOfLight)
    (f : Time → Space d → M) (h : Differentiable ℝ ↿f) :
    Differentiable ℝ ↿((timeSlice c).symm f)
```

2.51 Physlib.StatisticalMechanics.MicroCanonicalEnsemble.IdealGas

Physlib/StatisticalMechanics/MicroCanonicalEnsemble/IdealGas.lean on GitHub.

lemma partitionZ_eq [source]

The partition function Z for an ideal gas.

```
lemma partitionZ_eq (hV : 0 < V) (hβ : 0 < β) :
    IdealGas.partitionZ (n,V) β = V ^ n * (2 * Real.pi / β) ^ (3 * n / 2 : ℝ)
```

lemma helmholtzA_eq [source]

The Helmholtz Free Energy A for an ideal gas.

```
lemma helmholtzA_eq (hV : 0 < V) (hT : 0 < T) : IdealGas.helmholtzA (n,V) T =
    -n * T * (Real.log V + (3/2) * Real.log (2 * Real.pi * T))
```

theorem ideal_gas_law [source]

The ideal gas law: $PV = nRT$. In our unitsless system, $R = 1$.

```
theorem ideal_gas_law (hV : 0 < V) (hT : 0 < T) :
    let P
```

2.52 Physlib.StatisticalMechanics.MicroCanonicalEnsemble.ThermoQuantities

Physlib/StatisticalMechanics/MicroCanonicalEnsemble/ThermoQuantities.lean on GitHub.

def partitionZ [source]

The partition function corresponding to a given MicroHamiltonian. This is a function taking a thermodynamic β, not a temperature. It also depends on the data D defining the system extrinsincs.

** Ideally this would be an NNReal, but $\int$ (NNReal) doesn't work right now, so it would just be a separate proof anyway*

```
def partitionZ (β : ℝ) : ℝ
```

def PartitionZComplex [source]

The complexified partition function, viewed as a Laplace transform.

```
def PartitionZComplex (z : ℂ) : ℂ
```

def ZComplexConvergenceDomain [source]

The complex convergence domain of the partition-function Laplace transform.

```
def ZComplexConvergenceDomain : Set ℂ
```

theorem partitionZ_eq_re_partitionZComplex [source]

```
private theorem partitionZ_eq_re_partitionZComplex {β : ℝ}
    (hβ : (β : ℂ) ∈ H.ZComplexConvergenceDomain d) :
    H.partitionZ d β = (H.PartitionZComplex d β).re
```

theorem contDiffAt_partitionZ_of_mem_interior_convergenceDomain [source]

```
theorem contDiffAt_partitionZ_of_mem_interior_convergenceDomain {β : ℝ}
    (hβ : (β : ℂ) ∈ interior (H.ZComplexConvergenceDomain d)) :
    ContDiffAt ℝ ⊤ (H.partitionZ d) β
```

def partitionZT [source]

The partition function as a function of temperature T instead of β.

```
def partitionZT (T : ℝ) : ℝ
```

def internalU [source]

The Internal Energy, U or E, defined as $-\partial(\ln Z)/\partial\beta$. Parameterized here with β.

```
def internalU (β : ℝ) : ℝ
```

def helmholtzA [source]

*The Helmholtz Free Energy, -T * ln Z. Also denoted F. Parameterized here with temperature T, not β.*

```
def helmholtzA (T : ℝ) : ℝ
```

def entropyS [source]

The entropy, defined as the -∂A/∂T. Function of T.

```
def entropyS (T : ℝ) : ℝ
```

def entropySβ [source]

*The entropy, defined as ln Z + β*U. Function of β.*

```
def entropySβ (β : ℝ) : ℝ
```

def pressure [source]

Pressure, as a function of T. Defined as the conjugate variable to volume.

```
def pressure (T : ℝ) : ℝ
```

2.53 QuantumInfo.Channels.CPTP

QuantumInfo/Channels/CPTP.lean on GitHub.

theorem prod_apply_prod [source]

Tensor products commute with channel application: `(Λ₁ ⊗^cp Λ₂) (ρ₁ ⊗ᴹ ρ₂) = Λ₁ ρ₁ ⊗ᴹ Λ₂ ρ₂`.

```
@[simp]
theorem prod_apply_prod (Λ₁ : CPTPMap dI₁ dO₁) (Λ₂ : CPTPMap dI₂ dO₂)
    (ρ₁ : MState dI₁) (ρ₂ : MState dI₂) :
    (Λ₁ ⊗^cp Λ₂) (ρ₁ ⊗ᴹ ρ₂) = (Λ₁ ρ₁) ⊗ᴹ (Λ₂ ρ₂)
```

theorem piProd_id [source]

```
@[simp]
theorem piProd_id :
    piProd (fun i ↦ (CPTPMap.id : CPTPMap (dI i) (dI i))) = CPTPMap.id
```

theorem prep_append_map_entry [source]

The Stinespring preparation `prep ∘ append` acts on a matrix entry by the Kronecker product with the fixed pure state `|default⟩⟨default|` on `dOut × dOut`.

```
theorem prep_append_map_entry (X : Matrix dIn dIn ℂ)
    (a₁ : dIn) (b₁c₁ : dOut × dOut) (a₂ : dIn) (b₂c₂ : dOut × dOut) :
    let τ
```

2.54 QuantumInfo.Channels.Dual

QuantumInfo/Channels/Dual.lean on GitHub.

theorem matrix_mem_span_kronecker [source]

```
private theorem matrix_mem_span_kronecker {A C : Type*} [Fintype A] [Fintype C]
    [DecidableEq A] [DecidableEq C] (X : Matrix (A × C) (A × C) 𝕜) :
    X ∈ Submodule.span 𝕜
      (Set.range (fun p : (Matrix A A 𝕜 × Matrix C C 𝕜) => p.1 ⊗ₖ p.2))
```

2.55 QuantumInfo.Channels.MatrixMap

QuantumInfo/Channels/MatrixMap.lean on GitHub.

theorem choi_matrix_piProd [source]

```
theorem choi_matrix_piProd (Λi : ∀ i, MatrixMap (dI i) (dO i) R) :
    (MatrixMap.piProd Λi).choi_matrix =
      Matrix.reindex
        (Equiv.arrowProdEquivProdArrow ι dO dI)
        (Equiv.arrowProdEquivProdArrow ι dO dI)
        (Matrix.piProd (fun i => (Λi i).choi_matrix))
```

theorem piProd_id [source]

```
@[simp]
theorem piProd_id :
    piProd (fun i ↦ (LinearMap.id : MatrixMap (dI i) (dI i) R)) = LinearMap.id
```

2.56 QuantumInfo.Channels.Unbundled

QuantumInfo/Channels/Unbundled.lean on GitHub.

theorem piProd [source]

The `MatrixMap.piProd` product of IsTracePreserving maps is also trace preserving.

```
theorem piProd {Λi : ∀ i, MatrixMap (dI i) (dO i) R} (h₁ : ∀ i, (Λi i).IsTracePreserving) :
    (MatrixMap.piProd Λi).IsTracePreserving
```

theorem cp_subunital_kadison_schwarz [source]

Kadison-Schwarz for completely positive subunital matrix maps.

```
theorem cp_subunital_kadison_schwarz {M : MatrixMap A B ℂ} [DecidableEq B]
    (hM : M.IsCompletelyPositive) (hM1 : M 1 ≤ (1 : Matrix B B ℂ))
    (X : Matrix A A ℂ) :
    (M X)^H * M X ≤ M (X^H * X)
```

theorem positive_subunital_norm_apply_le [source]

A positive subunital matrix map is contractive on positive inputs.

```
theorem positive_subunital_norm_apply_le {M : MatrixMap A B ℂ} [DecidableEq B]
    (hM : M.IsPositive) (hM1 : M 1 ≤ (1 : Matrix B B ℂ))
    {X : Matrix A A ℂ} (hX : 0 ≤ X) :
    ‖M X‖ ≤ ‖X‖
```

theorem cp_subunital_opNorm_le_one [source]

A completely positive subunital matrix map is contractive in operator norm.

```
theorem cp_subunital_opNorm_le_one {M : MatrixMap A B ℂ} [DecidableEq B]
    (hM : M.IsCompletelyPositive) (hM1 : M 1 ≤ (1 : Matrix B B ℂ))
    (X : Matrix A A ℂ) :
    ‖M X‖ ≤ ‖X‖
```

2.57 QuantumInfo.Entropy.DPI

QuantumInfo/Entropy/DPI.lean on GitHub.

def sandwichedTraceFunctional [source]

The sandwiched trace functional `Q̃_α(ρ‖σ) = Tr[(σ^γ ρ σ^γ)^α]` where `γ = (1-α)/(2α)`. This is the quantity underlying the sandwiched Rényi divergence: `D̃_α(ρ‖σ) = log(Q̃_α(ρ‖σ)) / (α - 1)`.

Note: the `conj` operation gives `A.conj B = B * A.mat * B†`, and since `σ^γ` is Hermitian (self-adjoint), `B† = B`, so `ρ.M.conj (σ.M ^ γ).mat = σ^γ * ρ * σ^γ`.

```
noncomputable def sandwichedTraceFunctional (α : ℝ) (ρ σ : MState d) : ℝ
```

theorem sandwichedRelRentropy_eq_log_traceFunctional [source]

```
theorem sandwichedRelRentropy_eq_log_traceFunctional (hα₀ : 0 < α) (hα₁ : α ≠ 1)
    (hker : σ.M.ker ≤ ρ.M.ker) :
    D̃ α(ρ‖σ) = ENNReal.ofReal (Real.log (Q̃ α(ρ‖σ)) / (α - 1))
```

theorem sandwichedTraceFunctional_nonneg [source]

```
theorem sandwichedTraceFunctional_nonneg (ρ σ : MState d) :
    0 ≤ Q̃ α(ρ‖σ)
```

theorem sandwichedTraceFunctional_pos [source]

The trace functional is strictly positive when the kernel condition holds. Under `σ.M.ker ≤ ρ.M.ker` (i.e. `supp(ρ) ⊆ supp(σ)`), the sandwich `σ^γ ρ σ^γ ≠ 0` because ρ has support inside σ's support.

```
theorem sandwichedTraceFunctional_pos
    (ρ σ : MState d) (hker : σ.M.ker ≤ ρ.M.ker) :
    0 < Q̃ α(ρ‖σ)
```

theorem sandwichedTraceFunctional_conj_unitary_hermitian [source]

```
theorem sandwichedTraceFunctional_conj_unitary_hermitian
    (U : Matrix.unitaryGroup d ℂ) (A B : HermitianMat d ℂ) :
    let γ
```

theorem sandwichedTraceFunctional_conj_unitary_MState [source]

The trace functional is invariant under joint unitary conjugation of MStates.

```
theorem sandwichedTraceFunctional_conj_unitary_MState
    (U : Matrix.unitaryGroup d ℂ) (ρ σ : MState d) :
    Q̃ α(ρ.U_conj U‖σ.U_conj U) = Q̃ α(ρ‖σ)
```

def f_alpha [source]

The variational function `f_α(H, ρ, σ) = α · ⟪ρ, H⟫ − (α−1) · Tr[(σ^{−γ} H σ^{−γ})^{α/(α−1)}]` where `γ = (1−α)/(2α)`. For fixed `H ≥ 0`, this is linear in `ρ` and convex in `σ`. Frank–Lieb show that `Q̃_α(ρ‖σ) = sup_{H ≥ 0} f_α(H, ρ, σ)` for `α > 1`.

```
noncomputable def f_alpha (α : ℝ) (H : HermitianMat d ℂ) (ρ σ : MState d) : ℝ
```

def H_hat [source]

The optimizer in the variational formula: `H_hat = σ^γ (σ^γ ρ σ^γ)^{α−1} σ^γ` where `γ = (1−α)/(2α)`.

```
noncomputable def H_hat (α : ℝ) (ρ σ : MState d) : HermitianMat d ℂ
```

theorem H_hat_nonneg [source]

```
theorem H_hat_nonneg (ρ σ : MState d) : 0 ≤ H_hat α ρ σ
```

lemma ker_sigma_le_ker_conj_rpow [source]

The kernel of σ is contained in the kernel of (ρ.conj (σ^γ))^{α-1} when $\gamma \neq 0$ and $\alpha > 1$.

```
private lemma ker_sigma_le_ker_conj_rpow (ρ σ : MState d) {γ : ℝ} (hγ : γ ≠ 0) (hα1 : α - 1 ≠
    0) :
    σ.M.ker ≤ ((ρ.M.conj (σ.M ^ γ).mat) ^ (α - 1)).ker
```

theorem H_hat_conj_sigma [source]

Sub-lemma for Step 1b: the conj of H_hat by $\sigma^{-\gamma}$ simplifies to (ρ.M.conj (σ^{γ}).mat)^{α−1}. This uses $\sigma^{-\gamma} \cdot \sigma^{\gamma}$ = identity (on support) to cancel the outer σ^{γ} factors.

```
theorem H_hat_conj_sigma (hα : 1 < α) (ρ σ : MState d) :
    let γ
```

theorem inner_rho_H_hat [source]

```
theorem inner_rho_H_hat (hα : 1 < α) (ρ σ : MState d) :
    let γ
```

theorem f_alpha_at_optimizer [source]

```
theorem f_alpha_at_optimizer (hα : 1 < α) (ρ σ : MState d) :
    f_alpha α (H_hat α ρ σ) ρ σ = Q̃_ α(ρ‖σ)
```

lemma rpow_mul_neg_rpow_eq_supportProj [source]

For PSD `A` and `γ ≠ 0`, the product `A^γ * A^{-γ}` equals the support projection of `A`. This is because `x^γ * x^{-γ} = if x = 0 then 0 else 1` for `x ≥ 0`.

```
lemma rpow_mul_neg_rpow_eq_supportProj {A : HermitianMat d ℂ}
    (hA : 0 ≤ A) (γ : ℝ) (hγ : γ ≠ 0) :
    (A ^ γ).mat * (A ^ (-γ)).mat = A.supportProj.mat
```

lemma supportProj_mul_of_ker_le [source]

The support projection of `A` acts as identity on `B` when `A.ker ≤ B.ker`. Since `A.supportProj` projects onto `ker(A)⊥` and `B` is zero on `ker(A)`, the projection preserves `B`.

```
lemma supportProj_mul_of_ker_le {A B : HermitianMat d ℂ}
    (hker : A.ker ≤ B.ker) :
    A.supportProj.mat * B.mat = B.mat
```

lemma inner_eq_inner_conj_of_ker_le [source]

Under the support condition `σ.M.ker ≤ ρ.M.ker` (i.e., supp(ρ) ⊆ supp(σ)), conjugation by `σ^γ` and `σ^{-γ}` preserves the inner product: `⟪ρ.M, H⟫ = ⟪σ^γ ρ σ^γ, σ^{-γ} H σ^{-γ}⟫`. This holds because the kernel condition ensures `ρ` is supported on `supp(σ)`, where `σ^γ σ^{-γ}` acts as the identity.

```
lemma inner_eq_inner_conj_of_ker_le (ρ σ : MState d)
    (H : HermitianMat d ℂ) (hker : σ.M.ker ≤ ρ.M.ker) (γ : ℝ) (hγ : γ ≠ 0) :
    ⟪ρ.M, H⟫_ℝ = ⟪ρ.M.conj (σ.M ^ γ).mat, H.conj (σ.M ^ (-γ)).mat⟫_ℝ
```

theorem f_alpha_le_at_optimizer [source]

***Step 1c**: `H_hat` is a maximizer: for all `H ≥ 0`, `f_α(H) ≤ f_α(H_hat)`. This uses the trace Young inequality: for PSD `A, B` and conjugate exponents `p, q > 1`, `⟪A, B⟫ ≤ Tr[A^p]/p + Tr[B^q]/q`. Applied with `A = σ^γ ρ σ^γ`, `B = σ^{-γ} H σ^{-γ}`, `p = α`, `q = α/(α-1)`, the inner product identity `⟪ρ, H⟫ = ⟪A, B⟫` (under the support condition) yields `f_α(H) ≤ Tr[A^α] = Q̃_α(ρ‖σ) = f_α(H_hat)`. Note: the support condition `σ.M.ker ≤ ρ.M.ker` (i.e., supp(ρ) ⊆ supp(σ)) is necessary. Without it, the theorem is false: taking ρ orthogonal to σ gives Q̃_α = 0 but `f_α(H) = α · Tr[ρH] > 0` for appropriate H.*

```
theorem f_alpha_le_at_optimizer (hα : 1 < α) (ρ σ : MState d)
    (H : HermitianMat d ℂ) (hH : 0 ≤ H) (hker : σ.M.ker ≤ ρ.M.ker) :
    f_alpha α H ρ σ ≤ f_alpha α (H_hat α ρ σ) ρ σ
```

theorem traceFunctional_eq_iSup_f_alpha [source]

***Step 1 (Variational formula)**: For `α > 1`, the trace functional equals the supremum of `f_α` over all PSD `H`: `Q̃_α(ρ‖σ) = ⨆ (H : HermitianMat d ℂ) (_ : 0 ≤ H), f_alpha α H ρ σ`. The optimizer is `H_hat = σ^γ (σ^γ ρ σ^γ)^{α-1} σ^γ`.*

```
theorem traceFunctional_eq_iSup_f_alpha (hα : 1 < α) (ρ σ : MState d) (hker : σ.M.ker ≤ ρ.M.ker
    ) :
    Q̃ α(ρ‖σ) = ⨆ (H : {H : HermitianMat d ℂ // 0 ≤ H}), f_alpha α H.1 ρ σ
```

theorem f_alpha_convex_in_sigma [source]

*(Convexity in σ): For fixed `H ≥ 0` and `ρ`, and `α > 1`, the map `σ ↦ f_alpha α H ρ σ` is convex. The key is that for `p = α/(α-1) > 1`: • `A ↦ Tr[A^p]` is convex on PSD matrices (trace function convexity, Theorem 2.10 of Carlen), • `σ ↦ σ^{-γ} H σ^{-γ}` is *concave* in `σ` by Lieb concavity (since `-γ = (α-1)/(2α) ∈ (0,½)`), • The composition of a convex non-decreasing function with a concave function is convex, but we actually need the sign: the second term has a factor `-(α-1) < 0` which flips concave → convex. More precisely: `σ ↦ Tr[(σ^{-γ} H σ^{-γ})^p]` is concave (by Lieb + trace function convexity), so `σ ↦ -(α-1) · Tr[(σ^{-γ} H σ^{-γ})^p]` is convex.*

```
theorem f_alpha_convex_in_sigma (hα : 1 < α) (H : HermitianMat d ℂ) (hH : 0 ≤ H)
    (ρ : MState d) {ι : Type*} [Fintype ι]
    (w : ι → ℝ) (hw_nonneg : ∀ i, 0 ≤ w i) (hw_sum : ∑ i, w i = 1)
    (σs : ι → MState d) (σ_mix : MState d)
    (hσ_mix : σ_mix.M = ∑ i, w i • (σs i).M) :
    f_alpha α H ρ σ_mix ≤ ∑ i, w i * f_alpha α H ρ (σs i)
```

theorem f_alpha_jointly_convex [source]

```
theorem f_alpha_jointly_convex (hα : 1 < α) (H : HermitianMat d ℂ) (hH : 0 ≤ H)
    {ι : Type*} [Fintype ι]
    (w : ι → ℝ) (hw_nonneg : ∀ i, 0 ≤ w i) (hw_sum : ∑ i, w i = 1)
    (ρs σs : ι → MState d) (ρ_mix σ_mix : MState d)
    (hρ_mix : ρ_mix.M = ∑ i, w i • (ρs i).M)
    (hσ_mix : σ_mix.M = ∑ i, w i • (σs i).M) :
    f_alpha α H ρ_mix σ_mix ≤ ∑ i, w i * f_alpha α H (ρs i) (σs i)
```

theorem f_alpha_bddAbove [source]

```
theorem f_alpha_bddAbove (hα : 1 < α) (ρ σ : MState d) (hker : σ.M.ker ≤ ρ.M.ker) :
    BddAbove (Set.range (fun H : {H : HermitianMat d ℂ // 0 ≤ H} => f_alpha α H.1 ρ σ))
```

theorem iSup_f_alpha_jointly_convex [source]

```
theorem iSup_f_alpha_jointly_convex (hα : 1 < α)
    {ι : Type*} [Fintype ι]
    (w : ι → ℝ) (hw_nonneg : ∀ i, 0 ≤ w i) (hw_sum : ∑ i, w i = 1)
    (ρs σs : ι → MState d) (ρ_mix σ_mix : MState d)
    (hρ_mix : ρ_mix.M = ∑ i, w i • (ρs i).M)
    (hσ_mix : σ_mix.M = ∑ i, w i • (σs i).M)
    (hker : ∀ i, (σs i).M.ker ≤ (ρs i).M.ker) :
    (⨆ (H : {H : HermitianMat d ℂ // 0 ≤ H}), f_alpha α H.1 ρ_mix σ_mix) ≤
      ∑ i, w i * (⨆ (H : {H : HermitianMat d ℂ // 0 ≤ H}), f_alpha α H.1 (ρs i) (σs i))
```

theorem ker_weighted_sum_le [source]

If for all i, ker(σs i) ≤ ker(ρs i), then ker(∑ w i • σs i) ≤ ker(∑ w i • ρs i), provided all weights are nonneg and all matrices are PSD.

```
theorem HermitianMat.ker_weighted_sum_le {ι : Type*} [Fintype ι]
    (w : ι → ℝ) (hw_nonneg : ∀ i, 0 ≤ w i)
    (ρs σs : ι → HermitianMat d ℂ)
    (hρs_nonneg : ∀ i, 0 ≤ ρs i)
    (hσs_nonneg : ∀ i, 0 ≤ σs i)
    (hker : ∀ i, (σs i).ker ≤ (ρs i).ker) :
    (∑ i, w i • σs i).ker ≤ (∑ i, w i • ρs i).ker
```

theorem sandwichedTraceFunctional_jointly_convex [source]

The trace functional `Q̃_α` is jointly convex for `α > 1`. This is Proposition 3 of the paper, originally from Frank–Lieb. The proof uses the variational formula: `Q̃_α(ρ‖σ) = sup_{H ≥ 0} f_α(H, ρ, σ)` where `f_α(H, ρ, σ) = α · Tr[ρ H] - (α-1) · Tr[(σ^{-γ} H σ^{-γ})^{α/(α-1)}]` is jointly convex in `(ρ, σ)` for fixed `H` (since the first term is linear and the second uses the convexity of trace functions). The supremum of jointly convex functions is jointly convex.

```
theorem sandwichedTraceFunctional_jointly_convex (hα : 1 < α) {ι : Type*} [Fintype ι]
    (w : ι → ℝ) (hw_nonneg : ∀ i, 0 ≤ w i) (hw_sum : ∑ i, w i = 1)
    (ρs σs : ι → MState d) (ρ_mix σ_mix : MState d)
    (hρ_mix : ρ_mix.M = ∑ i, w i • (ρs i).M)
    (hσ_mix : σ_mix.M = ∑ i, w i • (σs i).M)
    (hker : ∀ i, (σs i).M.ker ≤ (ρs i).M.ker) :
    Q̃ α(ρ_mix‖σ_mix) ≤ ∑ i, w i * Q̃ α(ρs i‖σs i)
```

lemma signDiag_mem_unitaryGroup [source]

```
private lemma signDiag_mem_unitaryGroup (f : dB → Bool) :
    Matrix.diagonal (fun i : dB => (if f i then (-1 : ℂ) else 1)) ∈ Matrix.unitaryGroup dB ℂ
```

lemma twirlingU_mem_unitaryGroup [source]

The product of a ±1 diagonal matrix and a permutation matrix is unitary.

```
private lemma twirlingU_mem_unitaryGroup (σ : Equiv.Perm dB) (f : dB → Bool) :
    Matrix.diagonal (fun i : dB => (if f i then (-1 : ℂ) else 1)) * σ.permMatrix ℂ ∈
      Matrix.unitaryGroup dB ℂ
```

def twirlingU [source]

The twirling unitary for a given permutation and sign function.

```
private def twirlingU (σ : Equiv.Perm dB) (f : dB → Bool) : Matrix.unitaryGroup dB ℂ
```

lemma twirlingU_conj_entry [source]

```
private lemma twirlingU_conj_entry (X : HermitianMat dB ℂ) (σ : Equiv.Perm dB) (f : dB → Bool)
    (p q : dB) :
    (X.conj (twirlingU σ f : Matrix dB dB ℂ)) p q =
      (if f p then (-1 : ℂ) else 1) * (if f q then (-1 : ℂ) else 1) * X (σ p) (σ q)
```

lemma sum_sign_prod [source]

```
private lemma sum_sign_prod (p q : dB) :
    ∑ f : dB → Bool, ((if f p then (-1 : ℂ) else 1) * (if f q then (-1 : ℂ) else 1)) =
      if p = q then (2 ^ Fintype.card dB : ℕ) else 0
```

lemma sum_perm_diag_entry [source]

```
private lemma sum_perm_diag_entry (X : HermitianMat dB ℂ) (p : dB) :
    ∑ σ : Equiv.Perm dB, X (σ p) (σ p) =
      ((Fintype.card dB - 1).factorial : ℂ) * ∑ k : dB, X k k
```

lemma twirling_sum_eq [source]

```
private lemma twirling_sum_eq [Nonempty dB] (X : HermitianMat dB ℂ) (p q : dB) :
    ∑ i : Equiv.Perm dB × (dB → Bool), (X.conj (twirlingU i.1 i.2 : Matrix dB dB ℂ)) p q =
      if p = q then ((Fintype.card dB - 1).factorial * 2 ^ Fintype.card dB : ℕ) * ∑ k, X k k
      else 0
```

lemma twirling_identity [source]

```
private lemma twirling_identity [Nonempty dB] (X : HermitianMat dB ℂ) :
    (Fintype.card (Equiv.Perm dB × (dB → Bool)) : ℝ)⁻¹ •
      ∑ i : Equiv.Perm dB × (dB → Bool), X.conj (twirlingU i.1 i.2 : Matrix dB dB ℂ) =
        (X.trace / Fintype.card dB) • (1 : HermitianMat dB ℂ)
```

lemma exists_twirling_unitaries [source]

A twirling set for the system `dB` exists: there is a finite collection of unitaries whose average conjugation action twirls any matrix to a multiple of the identity. Specifically, `(1/|κ|) Σ_i V_i X V_i† = (Tr X / dim dB) · I` for all Hermitian `X` on `dB`. The standard construction uses the discrete Heisenberg-Weyl group of order `|dB|²`.

```
private lemma exists_twirling_unitaries [Nonempty dB] :
    ∃ (κ : Type) (_ : Fintype κ) (_ : Nonempty κ) (V : κ → Matrix.unitaryGroup dB ℂ),
      ∀ (X : HermitianMat dB ℂ),
        (Fintype.card κ : ℝ)⁻¹ • ∑ i : κ, X.conj (V i : Matrix dB dB ℂ) =
          (X.trace / Fintype.card dB) • (1 : HermitianMat dB ℂ)
```

theorem sandwichedTraceFunctional_mul [source]

```
theorem sandwichedTraceFunctional_mul
    (ρ₁ σ₁ : MState dA) (ρ₂ σ₂ : MState dB) :
    Q̃ α(ρ₁ ⊗ᴹ ρ₂‖σ₁ ⊗ᴹ σ₂) = Q̃ α(ρ₁‖σ₁) * Q̃ α(ρ₂‖σ₂)
```

theorem sandwichedTraceFunctional_self [source]

```
theorem sandwichedTraceFunctional_self (hα : 0 < α) (ρ : MState d) :
    Q̃ α(ρ‖ρ) = 1
```

theorem sandwichedTraceFunctional_tensor_invariant [source]

The trace functional is invariant under tensoring with a fixed state. This follows from multiplicativity (`sandwichedTraceFunctional_mul`) and the self-trace-functional being 1 (`sandwichedTraceFunctional_self`).

```
theorem sandwichedTraceFunctional_tensor_invariant (hα : 0 < α)
    (ρ σ : MState dA) (τ : MState dB) :
    Q̃ α(ρ ⊗ᴹ τ‖σ ⊗ᴹ τ) = Q̃ α(ρ‖σ)
```

def conjTensorUnitary [source]

The MState obtained by conjugating a bipartite state by `1_A ⊗ V` where `V` is a unitary on the `B` system. This is `(1_A ⊗ V) ρ_AB (1_A ⊗ V)†`.

```
def MState.conjTensorUnitary (ρ : MState (dA × dB)) (V : Matrix.unitaryGroup dB ℂ) :
    MState (dA × dB)
```

theorem conjTensorUnitary_M [source]

The twirled MState: averaging conjugation by `1_A ⊗ V_i` over all elements of the twirling set gives `ρ_A ⊗ uniform_B`. We state the HermitianMat-level equality needed for the joint convexity argument.

```
theorem MState.conjTensorUnitary_M (ρ : MState (dA × dB)) (V : Matrix.unitaryGroup dB ℂ) :
    (ρ.conjTensorUnitary V).M = ρ.M.conj ((1 : Matrix.unitaryGroup dA ℂ) ⊗ᵤ V).val
```

theorem sandwichedTraceFunctional_conj_tensorUnitary [source]

The trace functional is invariant under `1_A ⊗ V` conjugation.

```
theorem sandwichedTraceFunctional_conj_tensorUnitary
    (ρ σ : MState (dA × dB)) (V : Matrix.unitaryGroup dB ℂ) :
    Q̃ α(ρ.conjTensorUnitary V‖σ.conjTensorUnitary V) = Q̃ α(ρ‖σ)
```

lemma conj_kron_one_entry [source]

```
lemma conj_kron_one_entry (M : Matrix (dA × dB) (dA × dB) ℂ)
    (V : Matrix dB dB ℂ) (a₁ a₂ : dA) (b₁ b₂ : dB) :
    (Matrix.kroneckerMap (· * ·) (1 : Matrix dA dA ℂ) V * M *
     (Matrix.kroneckerMap (· * ·) (1 : Matrix dA dA ℂ) V).conjTranspose) (a₁, b₁) (a₂, b₂) =
    (V * (Matrix.of fun b₁' b₂' => M (a₁, b₁') (a₂, b₂')) * V.conjTranspose) b₁ b₂
```

lemma twirling_hermitian_entry [source]

```
lemma twirling_hermitian_entry
    (κ : Type) [Fintype κ] (V : κ → Matrix.unitaryGroup dB ℂ)
    (hV : ∀ (X : HermitianMat dB ℂ),
      (Fintype.card κ : ℝ)⁻¹ • ∑ i : κ, X.conj (V i : Matrix dB dB ℂ) =
        (X.trace / Fintype.card dB) • (1 : HermitianMat dB ℂ))
    (X : HermitianMat dB ℂ) (b₁ b₂ : dB) :
    ∑ i : κ, ((V i).val * X.val * (V i).val.conjTranspose) b₁ b₂ =
    (X.val.trace / (Fintype.card dB : ℂ)) * (Fintype.card κ : ℂ) *
      (if b₁ = b₂ then 1 else 0)
```

lemma twirling_general_matrix [source]

```
lemma twirling_general_matrix
    (κ : Type) [Fintype κ] (V : κ → Matrix.unitaryGroup dB ℂ)
    (hV : ∀ (X : HermitianMat dB ℂ),
      (Fintype.card κ : ℝ)⁻¹ • ∑ i : κ, X.conj (V i : Matrix dB dB ℂ) =
        (X.trace / Fintype.card dB) • (1 : HermitianMat dB ℂ))
    (X : Matrix dB dB ℂ) (b₁ b₂ : dB) :
    ∑ i : κ, ((V i).val * X * (V i).val.conjTranspose) b₁ b₂ =
    (X.trace / (Fintype.card dB : ℂ)) * (Fintype.card κ : ℂ) *
      (if b₁ = b₂ then 1 else 0)
```

def conjTensorUnitary' [source]

The MState obtained by conjugating a bipartite state by `1_A ⊗ V`.

```
def MState.conjTensorUnitary' (ρ : MState (dA × dB)) (V : Matrix.unitaryGroup dB ℂ) :
    MState (dA × dB)
```

lemma conjTensorUnitary'_entry [source]

```
lemma conjTensorUnitary'_entry (ρ : MState (dA × dB)) (V : Matrix.unitaryGroup dB ℂ)
    (a₁ a₂ : dA) (b₁ b₂ : dB) :
    (ρ.conjTensorUnitary' V).M.val (a₁, b₁) (a₂, b₂) =
    ((V : Matrix dB dB ℂ) * (Matrix.of fun b₁' b₂' => ρ.M.val (a₁, b₁') (a₂, b₂')) *
     (V : Matrix dB dB ℂ).conjTranspose) b₁ b₂
```

lemma prod_traceRight_uniform_entry [source]

```
lemma prod_traceRight_uniform_entry [Nonempty dB] (ρ : MState (dA × dB))
    (a₁ a₂ : dA) (b₁ b₂ : dB) :
    (ρ.traceRight ⊗ᴹ MState.uniform).M.val (a₁, b₁) (a₂, b₂) =
    ρ.M.val.traceRight a₁ a₂ * ((Fintype.card dB : ℂ)⁻¹ * if b₁ = b₂ then 1 else 0)
```

theorem twirling_average_eq [source]

```
theorem twirling_average_eq [Nonempty dB]
    (κ : Type) [Fintype κ] (V : κ → Matrix.unitaryGroup dB ℂ)
    (hV : ∀ (X : HermitianMat dB ℂ),
      (Fintype.card κ : ℝ)⁻¹ • ∑ i : κ, X.conj (V i : Matrix dB dB ℂ) =
        (X.trace / Fintype.card dB) • (1 : HermitianMat dB ℂ))
    (ρ : MState (dA × dB)) :
    ∑ i : κ, ((Fintype.card κ : ℝ)⁻¹ • (ρ.conjTensorUnitary' (V i)).M) =
      (ρ.traceRight ⊗ᴹ MState.uniform).M
```

lemma ker_conj_le_of_ker_le [source]

If `σ.M.ker ≤ ρ.M.ker`, then `(σ.conj B).ker ≤ (ρ.conj B).ker` for any matrix `B`. This follows from `ker_conj` (which expresses `(A.conj B).ker` as a `comap`) and `Submodule.comap_mono`.

```
lemma ker_conj_le_of_ker_le {n : Type*} [Fintype n] [DecidableEq n]
    {A B : HermitianMat n ℂ} (hA : 0 ≤ A) (hB : 0 ≤ B) (h : A.ker ≤ B.ker)
    (C : Matrix n n ℂ) : (A.conj C).ker ≤ (B.conj C).ker
```

lemma ker_conjTensorUnitary_le [source]

Unitary conjugation preserves the kernel ordering between MStates. If `σ.M.ker ≤ ρ.M.ker`, then `(σ.conjTensorUnitary V).M.ker ≤ (ρ.conjTensorUnitary V).M.ker`.

```
lemma MState.ker_conjTensorUnitary_le {dA dB : Type*} [Fintype dA] [Fintype dB]
    [DecidableEq dA] [DecidableEq dB]
    (ρ σ : MState (dA × dB)) (V : Matrix.unitaryGroup dB ℂ)
    (hker : σ.M.ker ≤ ρ.M.ker) :
    (σ.conjTensorUnitary V).M.ker ≤ (ρ.conjTensorUnitary V).M.ker
```

theorem sandwichedTraceFunctional_mono_traceRight [source]

Monotonicity of the trace functional under partial trace for `α > 1`. Equation (2.8) of the paper (second line).

```
theorem sandwichedTraceFunctional_mono_traceRight [Nonempty dB]
    (hα : 1 < α) (ρ σ : MState (dA × dB)) (hker : σ.M.ker ≤ ρ.M.ker) :
    Q̃ α(ρ.traceRight‖σ.traceRight) ≤ Q̃ α(ρ‖σ)
```

def vecTensorBasis [source]

The "tensor product" of a vector v with basis vector e_b: (v ⊗ e_b)(a, b') = v(a) if b' = b, else 0

```
private def vecTensorBasis (v : dA → ℂ) (b : dB) : (dA × dB) → ℂ
```

lemma inner_traceRight_eq_sum_inner_vecTensorBasis [source]

Key identity: ⟨v, (Tr_B A)v⟩ = ∑_b ⟨v⊗e_b, A(v⊗e_b)⟩

```
private lemma inner_traceRight_eq_sum_inner_vecTensorBasis
    (A : Matrix (dA × dB) (dA × dB) ℂ) (v : dA → ℂ) :
    star v ⬝ᵥ A.traceRight *ᵥ v =
    ∑ b : dB, star (vecTensorBasis v b) ⬝ᵥ A *ᵥ (vecTensorBasis v b)
```

lemma traceRight_mulVec_zero_of_vecTensorBasis_zero [source]

*If A.mulVec(v⊗e_b) = 0 for all b, then (Tr_B A) *ᵥ v = 0*

```
private lemma traceRight_mulVec_zero_of_vecTensorBasis_zero
    (A : Matrix (dA × dB) (dA × dB) ℂ) (v : dA → ℂ)
    (h : ∀ b : dB, A *ᵥ (vecTensorBasis v b) = 0) :
    A.traceRight *ᵥ v = 0
```

theorem ker_le_traceRight [source]

The kernel condition `σ.M.ker ≤ ρ.M.ker` is preserved under partial trace. This follows because `supp(ρ) ⊆ supp(σ)` implies `supp(Tr_B ρ) ⊆ supp(Tr_B σ)`: if `v ∈ supp(Tr_B ρ)`, then `⟨v, (Tr_B ρ) v⟩ > 0`, so for some basis vector `e_b` we have `v ⊗ e_b ∈ supp(ρ) ⊆ supp(σ)`, hence `⟨v, (Tr_B σ) v⟩ ≥ ⟨v ⊗ e_b, σ (v ⊗ e_b)⟩ > 0`.

```
theorem ker_le_traceRight {ρ σ : MState (dA × dB)}
    (hker : σ.M.ker ≤ ρ.M.ker) :
    σ.traceRight.M.ker ≤ ρ.traceRight.M.ker
```

theorem sandwichedRenyiEntropy_mono_traceRight [source]

The sandwiched Rényi divergence is monotone under partial trace for `α > 1`. This follows from monotonicity of the trace functional together with the fact that `D̃_α = log(Q̃_α) / (α - 1)` and both `log` and `1/(α-1)` are order-preserving for $\alpha > 1$.

```
theorem sandwichedRenyiEntropy_mono_traceRight [Nonempty dB]
    (hα : 1 < α) (ρ σ : MState (dA × dB))
    (hker : σ.M.ker ≤ ρ.M.ker) :
    D̃ α(ρ.traceRight‖σ.traceRight) ≤ D̃ α(ρ‖σ)
```

theorem sandwichedRenyiEntropy_conj_unitary [source]

```
theorem sandwichedRenyiEntropy_conj_unitary (hα : 0 < α) (ρ σ : MState d)
    (U : Matrix.unitaryGroup d ℂ) :
    D̃ α(ρ.U_conj U‖σ.U_conj U) = D̃ α(ρ‖σ)
```

theorem sandwichedRenyiEntropy_tensor_pure [source]

```
theorem sandwichedRenyiEntropy_tensor_pure (hα : 0 < α) (ρ σ : MState d₁) (ψ : Ket d₂) :
    D̃ α(ρ ⊗ᴹ MState.pure ψ‖σ ⊗ᴹ MState.pure ψ) = D̃ α(ρ‖σ)
```

theorem sandwichedRenyiEntropy_SWAP [source]

The sandwiched Rényi divergence is invariant under SWAP.

```
@[simp]
theorem sandwichedRenyiEntropy_SWAP (ρ σ : MState (dA × dB)) :
    D̃ α(ρ.SWAP‖σ.SWAP) = D̃ α(ρ‖σ)
```

theorem sandwichedRenyiEntropy_mono_traceRight' [source]

```
theorem sandwichedRenyiEntropy_mono_traceRight' [Nonempty dB]
    (hα : 1 < α) (ρ σ : MState (dA × dB)) :
    D̃ α(ρ.traceRight‖σ.traceRight) ≤ D̃ α(ρ‖σ)
```

theorem sandwichedRenyiEntropy_mono_traceLeft [source]

Monotonicity of the sandwiched Rényi divergence under `traceLeft` for `α > 1`. Follows from `sandwichedRenyiEntropy_mono_traceRight'` + SWAP invariance.

```
theorem sandwichedRenyiEntropy_mono_traceLeft [Nonempty dA]
    (hα : 1 < α) (ρ σ : MState (dA × dB)) :
    D̃ α(ρ.traceLeft‖σ.traceLeft) ≤ D̃ α(ρ‖σ)
```

theorem prep_append_eq_tensor_pure [source]

Helper: The Stinespring preparation `prep ∘ append` equals tensoring with a fixed pure state. `append = ofEquiv (Equiv.prodPUnit d₁).symm`. TODO: PULLOUT to a more reasonable place.

```
theorem prep_append_eq_tensor_pure [Inhabited d₂] (ρ : MState d₁) :
    let ψ₀ : Ket (d₂ × d₂)
```

theorem sandwichedRenyiEntropy_DPI_gt_one [source]

The Data Processing Inequality for the Sandwiched Rényi relative entropy ($\alpha > 1$). Every CPTP map Φ satisfies $\tilde{D}_\alpha(\Phi\rho\|\Phi\sigma) \le \tilde{D}_\alpha(\rho\|\sigma)$.

The proof uses the Stinespring representation (see `CPTPMap.exists_purify`): every CPTP map can be written as ancilla preparation + unitary conjugation + partial trace. Since the sandwiched Rényi divergence is invariant under the first two operations (by additivity and relabel invariance) and monotone under partial trace (by `sandwichedRenyiEntropy_mono_traceRight`), the DPI follows.

```
theorem sandwichedRenyiEntropy_DPI_gt_one (hα : 1 < α) (ρ σ : MState d₁) (Φ : CPTPMap d₁ d₂) :
    D̃ α(Φ ρ‖Φ σ) ≤ D̃ α(ρ‖σ)
```

theorem sandwichedRenyiEntropy_DPI_eq_one [source]

```
theorem sandwichedRenyiEntropy_DPI_eq_one (ρ σ : MState d₁) (Φ : CPTPMap d₁ d₂) :
    D̃ 1(Φ ρ‖Φ σ) ≤ D̃ 1(ρ‖σ)
```

2.58 QuantumInfo.Entropy.Relative

QuantumInfo/Entropy/Relative.lean on GitHub.

lemma fixed_support_kron_right [source]

```
private lemma fixed_support_kron_right (ρ : MState (dA × dB))
    {x : EuclideanSpace ℂ (dA × dB)} (hx : x ∈ ρ.M.support) :
    (ρ.traceRight.M.supportProj ⊗ₖ (1 : HermitianMat dB ℂ)).lin x = x
```

lemma fixed_support_kron_left [source]

```
private lemma fixed_support_kron_left (ρ : MState (dA × dB))
    {x : EuclideanSpace ℂ (dA × dB)} (hx : x ∈ ρ.M.support) :
    ((1 : HermitianMat dA ℂ) ⊗ₖ ρ.traceLeft.M.supportProj).lin x = x
```

2.59 QuantumInfo.ForMathlib.ComplexLaplaceTransform

QuantumInfo/ForMathlib/ComplexLaplaceTransform.lean on GitHub.

def ComplexLaplaceIntegrand [source]

The integrand of the complex Laplace transform of a possibly infinite-valued energy function.

```
def ComplexLaplaceIntegrand {α : Type*} (E : α → WithTop ℝ) (z : ℂ) (x : α) : ℂ
```

def ComplexLaplaceTransform [source]

The complex Laplace transform of a possibly infinite-valued energy function.

```
def ComplexLaplaceTransform {α : Type*} [MeasurableSpace α] [MeasureTheory.MeasureSpace α]
    (E : α → WithTop ℝ) (z : ℂ) : ℂ
```

def ComplexLaplaceConvergenceDomain [source]

The complex convergence domain of a Laplace transform.

```
def ComplexLaplaceConvergenceDomain {α : Type*} [MeasurableSpace α] [MeasureTheory.MeasureSpace
    α]
    (E : α → WithTop ℝ) : Set ℂ
```

def ComplexLaplaceEnvelope [source]

A two-sided exponential envelope controlling the Laplace integrand near `z`.

```
private def ComplexLaplaceEnvelope {α : Type*} (E : α → WithTop ℝ) (z : ℂ) (δ : ℝ)
    (x : α) : ℝ
```

def ComplexLaplaceEndpointEnvelope [source]

```
private def ComplexLaplaceEndpointEnvelope {α : Type*} (E : α → WithTop ℝ) (z w : ℂ)
    (x : α) : ℝ
```

theorem norm_complexLaplaceIntegrand_le_envelope [source]

```
private theorem norm_complexLaplaceIntegrand_le_envelope
    {α : Type*} {E : α → WithTop ℝ} {z w : ℂ} {δ : ℝ}
    (hw : w ∈ Metric.closedBall z δ) (x : α) :
    ‖ComplexLaplaceIntegrand E w x‖ ≤ ComplexLaplaceEnvelope E z δ x
```

theorem norm_complexLaplaceIntegrand_horizontal_le_endpointEnvelope [source]

```
private theorem norm_complexLaplaceIntegrand_horizontal_le_endpointEnvelope
    {α : Type*} {E : α → WithTop ℝ} {a b : ℂ} {t : ℝ}
    (ht : t ∈ Set.uIcc a.re b.re) (x : α) :
    ‖ComplexLaplaceIntegrand E (t + a.im * Complex.I) x‖ ≤
      ComplexLaplaceEndpointEnvelope E a (b.re + a.im * Complex.I) x
```

theorem norm_complexLaplaceIntegrand_vertical_le_endpointEnvelope [source]

```
private theorem norm_complexLaplaceIntegrand_vertical_le_endpointEnvelope
    {α : Type*} {E : α → WithTop ℝ} {a b : ℂ} {t : ℝ}
    (x : α) :
    ‖ComplexLaplaceIntegrand E (b.re + t * Complex.I) x‖ ≤
      ComplexLaplaceEndpointEnvelope E b (b.re + a.im * Complex.I) x
```

theorem analyticAt_complexLaplaceIntegrand [source]

For each configuration, the complex Laplace integrand is analytic in the parameter.

```
theorem analyticAt_complexLaplaceIntegrand
    {α : Type*} (E : α → WithTop ℝ) (x : α) (z : ℂ) :
    AnalyticAt ℂ (fun w : ℂ => ComplexLaplaceIntegrand E w x) z
```

theorem measurable_complexLaplaceIntegrand [source]

```
theorem measurable_complexLaplaceIntegrand
    {α : Type*} [MeasurableSpace α] {E : α → WithTop ℝ} (hE : Measurable E) (z : ℂ) :
    Measurable (ComplexLaplaceIntegrand E z)
```

theorem eventually_integrable_complexLaplaceIntegrand_of_mem_interior_convergenceDomain [source]

Interior convergence gives integrability throughout a neighborhood of the parameter.

```
theorem eventually_integrable_complexLaplaceIntegrand_of_mem_interior_convergenceDomain
    {α : Type*} [MeasurableSpace α] [MeasureTheory.MeasureSpace α] {E : α → WithTop ℝ} {z : ℂ}
    (hz : z ∈ interior (ComplexLaplaceConvergenceDomain E)) :
    ∀ᶠ w in nhds z,
      MeasureTheory.Integrable (μ := MeasureTheory.volume) (ComplexLaplaceIntegrand E w)
```

theorem continuousAt_complexLaplaceTransform_of_mem_interior_convergenceDomain [source]

```
theorem continuousAt_complexLaplaceTransform_of_mem_interior_convergenceDomain
    {α : Type*} [MeasurableSpace α] [MeasureTheory.MeasureSpace α] {E : α → WithTop ℝ} {z : ℂ}
    (hz : z ∈ interior (ComplexLaplaceConvergenceDomain E)) :
    ContinuousAt (ComplexLaplaceTransform E) z
```

theorem continuousOn_complexLaplaceTransform_interior_convergenceDomain [source]

```
theorem continuousOn_complexLaplaceTransform_interior_convergenceDomain
    {α : Type*} [MeasurableSpace α] [MeasureTheory.MeasureSpace α] {E : α → WithTop ℝ} :
    ContinuousOn (ComplexLaplaceTransform E) (interior (ComplexLaplaceConvergenceDomain E))
```

theorem integrable_uncurry_complexLaplaceIntegrand_horizontal [source]

```
private theorem integrable_uncurry_complexLaplaceIntegrand_horizontal
    {α : Type*} [MeasureTheory.MeasureSpace α]
    [MeasureTheory.SFinite (MeasureTheory.volume : MeasureTheory.Measure α)] {E : α → WithTop
    ℝ}
    {a b : ℂ}
    (hE : Measurable E)
    (ha : MeasureTheory.Integrable (μ := MeasureTheory.volume) (ComplexLaplaceIntegrand E a))
    (hb : MeasureTheory.Integrable (μ := MeasureTheory.volume)
      (ComplexLaplaceIntegrand E (b.re + a.im * Complex.I))) :
    MeasureTheory.Integrable
      (Function.uncurry fun t : ℝ => fun x : α =>
        ComplexLaplaceIntegrand E (t + a.im * Complex.I) x)
      ((MeasureTheory.volume.restrict (Set.uIoc a.re b.re)).prod MeasureTheory.volume)
```

theorem integrable_uncurry_complexLaplaceIntegrand_vertical [source]

```
private theorem integrable_uncurry_complexLaplaceIntegrand_vertical
    {α : Type*} [MeasureTheory.MeasureSpace α]
    [MeasureTheory.SFinite (MeasureTheory.volume : MeasureTheory.Measure α)] {E : α → WithTop
    ℝ}
    {a b : ℂ}
    (hE : Measurable E)
    (hb : MeasureTheory.Integrable (μ := MeasureTheory.volume) (ComplexLaplaceIntegrand E b))
    (hba : MeasureTheory.Integrable (μ := MeasureTheory.volume)
```

```
    (ComplexLaplaceIntegrand E (b.re + a.im * Complex.I))) :
  MeasureTheory.Integrable
    (Function.uncurry fun t : ℝ => fun x : α =>
      ComplexLaplaceIntegrand E (b.re + t * Complex.I) x)
    ((MeasureTheory.volume.restrict (Set.uIoc a.im b.im)).prod MeasureTheory.volume)
```

theorem wedgeIntegral_complexLaplaceTransform_eq_integral [source]

```
private theorem wedgeIntegral_complexLaplaceTransform_eq_integral
    {α : Type*} [MeasureTheory.MeasureSpace α]
    [MeasureTheory.SFinite (MeasureTheory.volume : MeasureTheory.Measure α)] {E : α → WithTop
    ℝ}
    {a b : ℂ}
    (hE : Measurable E)
    (hab : Complex.Rectangle a b ⊆ interior (ComplexLaplaceConvergenceDomain E)) :
    Complex.wedgeIntegral a b (ComplexLaplaceTransform E) =
      ∫ x, Complex.wedgeIntegral a b (fun w : ℂ => ComplexLaplaceIntegrand E w x)
```

theorem analyticAt_complexLaplaceTransform_of_mem_interior_convergenceDomain [source]

A complex Laplace transform is analytic on the interior of its convergence domain.

```
theorem analyticAt_complexLaplaceTransform_of_mem_interior_convergenceDomain
    {α : Type*} [MeasureTheory.MeasureSpace α]
    [MeasureTheory.SFinite (MeasureTheory.volume : MeasureTheory.Measure α)] {E : α → WithTop
    ℝ} {z : ℂ}
    (hE : Measurable E)
    (hz : z ∈ interior (ComplexLaplaceConvergenceDomain E)) :
    AnalyticAt ℂ (ComplexLaplaceTransform E) z
```

2.60 QuantumInfo.ForMathlib.HayataGroup.TraceInequality.BlockDiagonal

QuantumInfo/ForMathlib/HayataGroup/TraceInequality/BlockDiagonal.lean on GitHub.

abbrev HSum [source]

A two-fold Hilbert sum used for the block-operator proof of Jensen's inequality.

```
abbrev HSum (ℋ : Type u) [NormedAddCommGroup ℋ] [InnerProductSpace ℂ ℋ] : Type u
```

def hsumEquiv [source]

```
noncomputable def hsumEquiv (ℋ : Type u) [NormedAddCommGroup ℋ] [InnerProductSpace ℂ ℋ] :
    HSum ℋ ≃L[ℂ] (Fin 2 → ℋ)
```

def hsumProj [source]

```
noncomputable def hsumProj (ℋ : Type u) [NormedAddCommGroup ℋ] [InnerProductSpace ℂ ℋ]
    (i : Fin 2) : HSum ℋ →L[ℂ] ℋ
```

def hsumIncl [source]

```
noncomputable def hsumIncl (ℋ : Type u) [NormedAddCommGroup ℋ] [InnerProductSpace ℂ ℋ]
    (i : Fin 2) : ℋ →L[ℂ] HSum ℋ
```

def blockDiagonal [source]

The block diagonal operator `diag(A, B)` on the two-fold Hilbert sum.

```
noncomputable def blockDiagonal (A B : L ℋ) : L (HSum ℋ)
```

def blockOp [source]

A general `2 × 2` block operator on `HSum ℋ`.

```
noncomputable def blockOp (A00 A01 A10 A11 : L ℋ) : L (HSum ℋ)
```

theorem blockOp_ext [source]

```
theorem blockOp_ext {T S : L (HSum ℋ)}
    (h0 : ∀ z : HSum ℋ, hsumProj ℋ 0 (T z) = hsumProj ℋ 0 (S z))
    (h1 : ∀ z : HSum ℋ, hsumProj ℋ 1 (T z) = hsumProj ℋ 1 (S z)) :
    T = S
```

def blockDiagonalHom [source]

```
noncomputable def blockDiagonalHom : (L ℋ × L ℋ) →⋆ₐ[ℝ] L (HSum ℋ) where
```

theorem blockDiagonal_nonneg [source]

```
theorem blockDiagonal_nonneg {A B : L ℋ} (hA : 0 ≤ A) (hB : 0 ≤ B) :
    0 ≤ blockDiagonal (ℋ := ℋ) A B
```

2.61 QuantumInfo.ForMathlib.HayataGroup.TraceInequality.GeneralizedPerspectiveFunction

QuantumInfo/ForMathlib/HayataGroup/TraceInequality/GeneralizedPerspectiveFunction.lean on GitHub.

def JointlyConvexOn [source]

Joint convexity of a two-variable map on prescribed domains in each argument.

```
def JointlyConvexOn (s : Set E) (t : Set F) (Φ : E → F → G) : Prop
```

def JointlyConvex [source]

Joint convexity of a two-variable map without domain restrictions.

```
def JointlyConvex (Φ : E → F → G) : Prop
```

def JointlyConcaveOn [source]

Joint concavity of a two-variable map on prescribed domains in each argument.

```
def JointlyConcaveOn (s : Set E) (t : Set F) (Φ : E → F → G) : Prop
```

def JointlyConcave [source]

Joint concavity of a two-variable map without domain restrictions.

```
def JointlyConcave (Φ : E → F → G) : Prop
```

def hSqrt [source]

The operator `h(B)^(1/2)` defined by real continuous functional calculus.

```
noncomputable def hSqrt (h : ℝ → ℝ) (B : L ℋ) : L ℋ
```

def hInvSqrt [source]

The operator `h(B)^(-1/2)` defined by real continuous functional calculus.

```
noncomputable def hInvSqrt (h : ℝ → ℝ) (B : L ℋ) : L ℋ
```

def GeneralizedPerspective [source]

The generalized perspective function `(fΔh)(A, B) = h(B)^(1/2) f(h(B)^(-1/2) A h(B)^(-1/2)) h(B)^(1/2)`.

This definition is intended to be used when `A` is Hermitian and `h(B)` is positive/invertible.

```
noncomputable def GeneralizedPerspective (f h : ℝ → ℝ) (A B : L ℋ) : L ℋ
```

def psdSet [source]

Positive semidefinite operators.

```
def psdSet : Set (L ℋ)
```

def pdSet [source]

Strictly positive operators.

```
def pdSet : Set (L ℋ)
```

lemma spectrum_convexCombo_Ioi [source]

```
private lemma spectrum_convexCombo_Ioi {A B : L ℋ} {t : ℝ}
    (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B) (ht0 : 0 ≤ t) (ht1 : t ≤ 1)
    (As : spectrum ℝ A ⊆ Set.Ioi (0 : ℝ)) (Bs : spectrum ℝ B ⊆ Set.Ioi (0 : ℝ)) :
    spectrum ℝ ((1 - t) • A + t • B) ⊆ Set.Ioi (0 : ℝ)
```

lemma cfcR_sq_eq [source]

```
private lemma cfcR_sq_eq {g k : ℝ → ℝ} {A : L ℋ}
    (hA : IsSelfAdjoint A)
    (hg : ContinuousOn g (spectrum ℝ A))
    (hk : ContinuousOn k (spectrum ℝ A))
    (hmul : ∀ x ∈ spectrum ℝ A, g x * k x = 1) :
    cfcR (ℋ := ℋ) g A * cfcR (ℋ := ℋ) k A = (1 : L ℋ)
```

lemma cfcR_mul_eq [source]

```
private lemma cfcR_mul_eq {g k m : ℝ → ℝ} {A : L ℋ}
    (hg : ContinuousOn g (spectrum ℝ A))
    (hk : ContinuousOn k (spectrum ℝ A))
    (hmul : ∀ x ∈ spectrum ℝ A, g x * k x = m x) :
    cfcR (ℋ := ℋ) g A * cfcR (ℋ := ℋ) k A = cfcR (ℋ := ℋ) m A
```

lemma hpow_continuousOn [source]

```
private lemma hpow_continuousOn
    (h : ℝ → ℝ) (p : ℝ)
    (hcont : ContinuousOn h (Set.Ioi (0 : ℝ)))
    (hpos : ∀ x ∈ Set.Ioi (0 : ℝ), 0 < h x) :
    ContinuousOn (fun x : ℝ ↦ (h x) ^ p) (Set.Ioi (0 : ℝ))
```

lemma hSqrt_selfAdjoint [source]

```
private lemma hSqrt_selfAdjoint (h : ℝ → ℝ) (B : L ℋ) :
    IsSelfAdjoint (hSqrt (ℋ := ℋ) h B)
```

lemma hInvSqrt_selfAdjoint [source]

```
private lemma hInvSqrt_selfAdjoint (h : ℝ → ℝ) (B : L ℋ) :
    IsSelfAdjoint (hInvSqrt (ℋ := ℋ) h B)
```

lemma hSqrt_mul_hInvSqrt_eq_one [source]

```
private lemma hSqrt_mul_hInvSqrt_eq_one
    {h : ℝ → ℝ} {B : L ℋ}
    (hB : IsSelfAdjoint B) (Bs : spectrum ℝ B ⊆ Set.Ioi (0 : ℝ))
    (hcont : ContinuousOn h (Set.Ioi (0 : ℝ)))
    (hpos : ∀ x ∈ Set.Ioi (0 : ℝ), 0 < h x) :
    hSqrt (ℋ := ℋ) h B * hInvSqrt (ℋ := ℋ) h B = (1 : L ℋ)
```

lemma hInvSqrt_mul_hSqrt_eq_one [source]

```
private lemma hInvSqrt_mul_hSqrt_eq_one
    {h : ℝ → ℝ} {B : L ℋ}
    (hB : IsSelfAdjoint B) (Bs : spectrum ℝ B ⊆ Set.Ioi (0 : ℝ))
    (hcont : ContinuousOn h (Set.Ioi (0 : ℝ)))
    (hpos : ∀ x ∈ Set.Ioi (0 : ℝ), 0 < h x) :
    hInvSqrt (ℋ := ℋ) h B * hSqrt (ℋ := ℋ) h B = (1 : L ℋ)
```

lemma hSqrt_mul_hSqrt_eq [source]

```
private lemma hSqrt_mul_hSqrt_eq
    {h : ℝ → ℝ} {B : L ℋ}
    (Bs : spectrum ℝ B ⊆ Set.Ioi (0 : ℝ))
    (hcont : ContinuousOn h (Set.Ioi (0 : ℝ)))
    (hpos : ∀ x ∈ Set.Ioi (0 : ℝ), 0 < h x) :
    hSqrt (ℋ := ℋ) h B * hSqrt (ℋ := ℋ) h B = cfcR (ℋ := ℋ) h B
```

lemma conj_le_conj [source]

```
private lemma conj_le_conj {X Y T : L ℋ} (hXY : X ≤ Y) (hT : IsSelfAdjoint T) :
    T * X * T ≤ T * Y * T
```

theorem theorem_2_5_forward_jointlyConvexOn_psd_pd_of_condV [source]

```
private theorem theorem_2_5_forward_jointlyConvexOn_psd_pd_of_condV
    {f h : ℝ → ℝ}
    (hcoreV : CondV (ℋ := ℋ) f)
    (hconc : OperatorConcaveOn (ℋ := ℋ) (Set.Ioi (0 : ℝ)) h)
    (hcont : ContinuousOn h (Set.Ioi (0 : ℝ)))
    (hpos : ∀ x ∈ Set.Ioi (0 : ℝ), 0 < h x) :
    JointlyConvexOn (psdSet (ℋ := ℋ)) (pdSet (ℋ := ℋ)) (fun A B ↦ (f Δ h) A B)
```

theorem theorem_2_5_forward_jointlyConvexOn_psd_pd [source]

```
theorem theorem_2_5_forward_jointlyConvexOn_psd_pd
    {f h : ℝ → ℝ}
    (hf : CondIAll.{u} f)
    (hconc : OperatorConcaveOn (ℋ := ℋ) (Set.Ioi (0 : ℝ)) h)
    (hcont : ContinuousOn h (Set.Ioi (0 : ℝ)))
    (hpos : ∀ x ∈ Set.Ioi (0 : ℝ), 0 < h x) :
    JointlyConvexOn (psdSet (ℋ := ℋ)) (pdSet (ℋ := ℋ)) (fun A B ↦ (f Δ h) A B)
```

theorem theorem_2_5_forward_jointlyConvexOn_psd_pd_Ici [source]

```
theorem theorem_2_5_forward_jointlyConvexOn_psd_pd_Ici
    {f h : ℝ → ℝ}
    (hf : CondIciAll.{u} f)
    (hconc : OperatorConcaveOn (ℋ := ℋ) (Set.Ioi (0 : ℝ)) h)
    (hcont : ContinuousOn h (Set.Ioi (0 : ℝ)))
    (hpos : ∀ x ∈ Set.Ioi (0 : ℝ), 0 < h x) :
    JointlyConvexOn (psdSet (ℋ := ℋ)) (pdSet (ℋ := ℋ)) (fun A B ↦ (f Δ h) A B)
```

lemma generalizedPerspective_neg [source]

```
private lemma generalizedPerspective_neg
    (f h : ℝ → ℝ) (A B : L ℋ) :
    ((fun x : ℝ ↦ -f x) Δ h) A B = -((f Δ h) A B)
```

lemma jointlyConvexOn_neg [source]

```
private lemma jointlyConvexOn_neg
    {s : Set (L ℋ)} {t : Set (L ℋ)} {Φ : L ℋ → L ℋ → L ℋ}
    (hΦ : JointlyConvexOn s t (fun A B ↦ -Φ A B)) :
    JointlyConcaveOn s t Φ
```

theorem theorem_2_6_forward_jointlyConcaveOn_psd_pd [source]

Restricted forward form of Corollary 2.6 on the positive cone.

```
theorem theorem_2_6_forward_jointlyConcaveOn_psd_pd
    {f h : ℝ → ℝ}
```

```
    (hfconc : OperatorConcaveAll.{u} f)
    (hf0 : 0 ≤ f 0)
    (hconc : OperatorConcaveOn (ℋ := ℋ) (Set.Ioi (0 : ℝ)) h)
    (hcont : ContinuousOn h (Set.Ioi (0 : ℝ)))
    (hpos : ∀ x ∈ Set.Ioi (0 : ℝ), 0 < h x) :
    JointlyConcaveOn (psdSet (ℋ := ℋ)) (pdSet (ℋ := ℋ)) (fun A B ↦ (f Δ h) A B)
```

theorem theorem_2_6_forward_jointlyConcaveOn_psd_pd_Ici [source]

Restricted localized forward form of Corollary 2.6 on the positive cone.

```
theorem theorem_2_6_forward_jointlyConcaveOn_psd_pd_Ici
    {f h : ℝ → ℝ}
    (hfconc : OperatorConcaveOnAll.{u} (Set.Ici (0 : ℝ)) f)
    (hfcont : ContinuousOn f (Set.Ici (0 : ℝ)))
    (hf0 : 0 ≤ f 0)
    (hconc : OperatorConcaveOn (ℋ := ℋ) (Set.Ioi (0 : ℝ)) h)
    (hcont : ContinuousOn h (Set.Ioi (0 : ℝ)))
    (hpos : ∀ x ∈ Set.Ioi (0 : ℝ), 0 < h x) :
    JointlyConcaveOn (psdSet (ℋ := ℋ)) (pdSet (ℋ := ℋ)) (fun A B ↦ (f Δ h) A B)
```

2.62 QuantumInfo.ForMathlib.HayataGroup.TraceInequality.HilbertSchmidtOperatorSpace

QuantumInfo/ForMathlib/HayataGroup/TraceInequality/HilbertSchmidtOperatorSpace.lean on GitHub.

abbrev HSIndex [source]

The canonical finite index used for Hilbert-Schmidt coordinates.

```
abbrev HSIndex (ℋ : Type u) [NormedAddCommGroup ℋ] [InnerProductSpace ℂ ℋ]
    [FiniteDimensional ℂ ℋ]
```

abbrev hsOrthonormalBasis [source]

The standard orthonormal basis on a finite-dimensional Hilbert space.

```
noncomputable abbrev hsOrthonormalBasis :
    OrthonormalBasis (HSIndex ℋ) ℂ ℋ
```

abbrev HSCoords [source]

Coordinate space for Hilbert-Schmidt operators.

```
abbrev HSCoords (ℋ : Type u) [NormedAddCommGroup ℋ] [InnerProductSpace ℂ ℋ]
    [FiniteDimensional ℂ ℋ]
```

abbrev HSCoordFun [source]

The underlying coordinate function space for Hilbert-Schmidt operators.

```
abbrev HSCoordFun (ℋ : Type u) [NormedAddCommGroup ℋ] [InnerProductSpace ℂ ℋ]
    [FiniteDimensional ℂ ℋ]
```

def HSOp [source]

The operator space `L` $\mathcal{H}$, viewed later with the Hilbert-Schmidt structure.

```
def HSOp (ℋ : Type u) [NormedAddCommGroup ℋ] [InnerProductSpace ℂ ℋ] : Type u
```

instance AddCommGroup (HSOp $\mathcal{H}$) [source]

```
instance : AddCommGroup (HSOp ℋ)
```

instance Module $\mathbb{C}$ (HSOp $\mathcal{H}$) [source]

```
instance : Module ℂ (HSOp ℋ)
```

instance Star (HSOp $\mathcal{H}$) [source]

```
instance : Star (HSOp ℋ)
```

instance Mul (HSOp $\mathcal{H}$) [source]

```
instance : Mul (HSOp ℋ)
```

instance One (HSOp $\mathcal{H}$) [source]

```
instance : One (HSOp ℋ)
```

instance Inhabited (HSOp $\mathcal{H}$) [source]

```
instance : Inhabited (HSOp ℋ)
```

instance Zero (HSOp $\mathcal{H}$) [source]

```
instance : Zero (HSOp ℋ)
```

instance FiniteDimensional $\mathbb{C}$ (HSOp $\mathcal{H}$) [source]

```
instance : FiniteDimensional ℂ (HSOp ℋ)
```

def hsCoordsLinearEquiv [source]

```
noncomputable def hsCoordsLinearEquiv :
    HSCoords ℋ ≃ₗ[ℂ] (HSCoordFun ℋ)
```

def matrixToFun [source]

```
def matrixToFun :
    Matrix (HSIndex ℋ) (HSIndex ℋ) ℂ ≃ₗ[ℂ] HSCoordFun ℋ where
```

def matrixToCoords [source]

```
noncomputable def matrixToCoords :
    Matrix (HSIndex ℋ) (HSIndex ℋ) ℂ ≃ₗ[ℂ] HSCoords ℋ
```

def hsLinearMapEquiv [source]

Forget continuity and identify continuous linear operators with linear endomorphisms.

```
noncomputable def hsLinearMapEquiv :
    HSOp ℋ ≃ₗ[ℂ] (ℋ →ₗ[ℂ] ℋ)
```

def toHSCoordsLinearEquiv [source]

Hilbert-Schmidt coordinates on `L ℋ`.

```
noncomputable def toHSCoordsLinearEquiv :
    HSOp ℋ ≃ₗ[ℂ] HSCoords ℋ
```

instance NormedAddCommGroup (HSOp ℋ) [source]

```
instance : NormedAddCommGroup (HSOp ℋ)
```

instance NormedSpace ℂ (HSOp ℋ) [source]

```
instance : NormedSpace ℂ (HSOp ℋ)
```

instance Inner ℂ (HSOp ℋ) [source]

```
instance : Inner ℂ (HSOp ℋ) where
```

instance InnerProductSpace ℂ (HSOp ℋ) [source]

```
instance : InnerProductSpace ℂ (HSOp ℋ) where
```

def toHSCoordsLinearIsometryEquiv [source]

Hilbert-Schmidt coordinates as a linear isometry.

```
noncomputable def toHSCoordsLinearIsometryEquiv :
    HSOp ℋ ≃ₗᵢ[ℂ] HSCoords ℋ
```

instance CompleteSpace (HSOp ℋ) [source]

```
instance : CompleteSpace (HSOp ℋ)
```

instance ContinuousSMul ℂ (HSOp ℋ) [source]

```
instance : ContinuousSMul ℂ (HSOp ℋ)
```

abbrev ofOp [source]

Reinterpret an operator as an element of the Hilbert-Schmidt operator space.

```
abbrev ofOp (T : L ℋ) : HSOp ℋ
```

abbrev toOp [source]

Forget the Hilbert-Schmidt structure and recover the underlying operator.

```
abbrev toOp (T : HSOp ℋ) : L ℋ
```

def leftMulHS [source]

Left multiplication on the Hilbert-Schmidt operator space.

```
noncomputable def leftMulHS (A : L ℋ) : HSOp ℋ →L[ℂ] HSOp ℋ
```

def rightMulHS [source]

Right multiplication on the Hilbert-Schmidt operator space.

```
noncomputable def rightMulHS (B : L ℋ) : HSOp ℋ →L[ℂ] HSOp ℋ
```

lemma leftMulHS_rightMulHS_commute [source]

```
lemma leftMulHS_rightMulHS_commute (A B : L ℋ) :
    Commute (leftMulHS (ℋ := ℋ) A) (rightMulHS (ℋ := ℋ) B)
```

lemma hsInner_eq_pairSum [source]

```
private lemma hsInner_eq_pairSum (X Y : L ℋ) :
    inner ℂ (ofOp X) (ofOp Y) =
      ∑ p : HSIndex ℋ × HSIndex ℋ,
        LinearMap.toMatrix (hsOrthonormalBasis (ℋ := ℋ)).toBasis
          (hsOrthonormalBasis (ℋ := ℋ)).toBasis Y.toLinearMap p.1 p.2 *
        star
          (LinearMap.toMatrix (hsOrthonormalBasis (ℋ := ℋ)).toBasis
            (hsOrthonormalBasis (ℋ := ℋ)).toBasis X.toLinearMap p.1 p.2)
```

lemma trace_star_mul_eq_pairSum [source]

```
private lemma trace_star_mul_eq_pairSum (X Y : L ℋ) :
    LinearMap.trace ℂ ℋ ((star X * Y).toLinearMap) =
      ∑ p : HSIndex ℋ × HSIndex ℋ,
        LinearMap.toMatrix (hsOrthonormalBasis (ℋ := ℋ)).toBasis
          (hsOrthonormalBasis (ℋ := ℋ)).toBasis Y.toLinearMap p.1 p.2 *
        star
          (LinearMap.toMatrix (hsOrthonormalBasis (ℋ := ℋ)).toBasis
            (hsOrthonormalBasis (ℋ := ℋ)).toBasis X.toLinearMap p.1 p.2)
```

lemma hsInner_eq_trace [source]

```
lemma hsInner_eq_trace (X Y : L ℋ) :
    inner ℂ (ofOp X) (ofOp Y) = LinearMap.trace ℂ ℋ ((star X * Y).toLinearMap)
```

lemma re_hsInner_eq_traceRe [source]

```
lemma re_hsInner_eq_traceRe (X Y : L ℋ) :
    Complex.re (inner ℂ (ofOp X) (ofOp Y)) =
      Complex.re (LinearMap.trace ℂ ℋ ((star X * Y).toLinearMap))
```

lemma leftMulHS_nonneg [source]

```
lemma leftMulHS_nonneg {A : L ℋ} (hA0 : 0 ≤ A) :
    0 ≤ leftMulHS (ℋ := ℋ) A
```

lemma leftMulHS_le_leftMulHS [source]

```
lemma leftMulHS_le_leftMulHS {A B : L ℋ} (hAB : A ≤ B) :
    leftMulHS (ℋ := ℋ) A ≤ leftMulHS (ℋ := ℋ) B
```

lemma leftMulHS_pdSet [source]

```
lemma leftMulHS_pdSet [ContinuousFunctionalCalculus ℝ (L ℋ) IsSelfAdjoint] [Nontrivial (L ℋ)]
    {A : L ℋ} (hA : A ∈ GeneralizedPerspectiveFunction.pdSet (ℋ := ℋ)) :
    leftMulHS (ℋ := ℋ) A ∈ GeneralizedPerspectiveFunction.pdSet (ℋ := HSOp ℋ)
```

def leftMulHSStarAlgHom [source]

Left multiplication as a real ⋆-algebra homomorphism on the Hilbert-Schmidt operator space.

```
noncomputable def leftMulHSStarAlgHom : L ℋ →⋆ₐ[ℝ] L (HSOp ℋ) where
```

def rightMulHSStarAlgHom [source]

Right multiplication as a real ⋆-algebra homomorphism out of the opposite algebra.

```
noncomputable def rightMulHSStarAlgHom : (L ℋ)ᵐᵒᵖ →⋆ₐ[ℝ] L (HSOp ℋ) where
```

def opStarHSStarAlgHom [source]

The ⋆-algebra hom sending `A` to `op (star A)`. On selfadjoint operators this is just `op`.

```
noncomputable def opStarHSStarAlgHom : L ℋ →⋆ₐ[ℝ] (L ℋ)ᵐᵒᵖ where
```

def opStarHSAlgEquiv [source]

```
private noncomputable def opStarHSAlgEquiv : L ℋ ≃ₐ[ℝ] (L ℋ)ᵐᵒᵖ where
```

lemma op_isSelfAdjoint [source]

```
lemma op_isSelfAdjoint (A : L ℋ) (hA : IsSelfAdjoint A) :
    IsSelfAdjoint (MulOpposite.op A : (L ℋ)ᵐᵒᵖ)
```

def opStarHSLinearMap [source]

```
private noncomputable def opStarHSLinearMap : L ℋ →ₗ[ℝ] (L ℋ)ᵐᵒᵖ where
```

def leftMulHSLinearMap [source]

```
private noncomputable def leftMulHSLinearMap : L ℋ →ₗ[ℝ] L (HSOp ℋ) where
```

def rightMulHSLinearMap [source]

```
private noncomputable def rightMulHSLinearMap : (L ℋ)ᵐᵒᵖ →ₗ[ℝ] L (HSOp ℋ) where
```

lemma spectrum_op_eq [source]

```
lemma spectrum_op_eq [ContinuousFunctionalCalculus ℝ (L ℋ) IsSelfAdjoint] [Nontrivial (L ℋ)]
    (A : L ℋ) (hA : IsSelfAdjoint A) :
    spectrum ℝ (MulOpposite.op A : (L ℋ)ᵐᵒᵖ) = spectrum ℝ A
```

lemma cfc_op_eq_op_cfc [source]

```
lemma cfc_op_eq_op_cfc [ContinuousFunctionalCalculus ℝ (L ℋ) IsSelfAdjoint]
    [ContinuousFunctionalCalculus ℝ ((L ℋ)ᵐᵒᵖ) IsSelfAdjoint] [Nontrivial (L ℋ)]
    (f : ℝ → ℝ) (A : L ℋ) (hA : IsSelfAdjoint A)
    (hf : ContinuousOn f (spectrum ℝ A)) :
    cfc (R := ℝ) (A := (L ℋ)ᵐᵒᵖ) (p := IsSelfAdjoint) f (MulOpposite.op A) =
      MulOpposite.op (cfcR f A)
```

lemma leftMulHS_cfcR [source]

```
lemma leftMulHS_cfcR [ContinuousFunctionalCalculus ℝ (L ℋ) IsSelfAdjoint] [Nontrivial (L ℋ)]
    (f : ℝ → ℝ) (A : L ℋ) (hA : IsSelfAdjoint A)
    (hf : ContinuousOn f (spectrum ℝ A)) :
    leftMulHS (ℋ := ℋ) (cfcR f A) =
      cfcR (ℋ := HSOp ℋ) f (leftMulHS (ℋ := ℋ) A)
```

lemma rightMulHS_cfcR [source]

```
lemma rightMulHS_cfcR [ContinuousFunctionalCalculus ℝ (L ℋ) IsSelfAdjoint]
    [ContinuousFunctionalCalculus ℝ ((L ℋ)ᵐᵒᵖ) IsSelfAdjoint] [Nontrivial (L ℋ)]
    (f : ℝ → ℝ) (A : L ℋ) (hA : IsSelfAdjoint A)
    (hf : ContinuousOn f (spectrum ℝ A)) :
    rightMulHS (ℋ := ℋ) (cfcR f A) =
      cfcR (ℋ := HSOp ℋ) f (rightMulHS (ℋ := ℋ) A)
```

2.63 QuantumInfo.ForMathlib.HayataGroup.TraceInequality.JensenOperatorInequality

QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperatorInequality.lean on GitHub.

instance ContinuousFunctionalCalculus ℂ (L ℋ × L ℋ) IsStarNormal [source]

```
noncomputable local instance : ContinuousFunctionalCalculus ℂ (L ℋ × L ℋ) IsStarNormal
```

instance ContinuousFunctionalCalculus ℝ (L ℋ × L ℋ) IsSelfAdjoint [source]

```
noncomputable local instance : ContinuousFunctionalCalculus ℝ (L ℋ × L ℋ) IsSelfAdjoint
```

instance NonnegSpectrumClass ℝ (L (HSum ℋ)) [source]

```
noncomputable local instance : NonnegSpectrumClass ℝ (L (HSum ℋ))
```

instance Module ℝ (L (HSum ℋ)) [source]

```
noncomputable local instance : Module ℝ (L (HSum ℋ))
```

def CondIVAll [source]

Uniform version of Condition (iv), with the Hilbert space arbitrary in the same universe. This is the theorem-level uniform counterpart to the operator-level `...All` predicates.

```
def CondIVAll (f : ℝ → ℝ) : Prop
```

theorem nontrivial_hsumL_wrap [source]

`L (HSum ℋ)` is nontrivial once `L ℋ` is.

```
private theorem nontrivial_hsumL_wrap [Nontrivial ℋ] : Nontrivial (L (HSum ℋ))
```

lemma blockDiagonal_selfAdjoint_wrap [source]

```
private lemma blockDiagonal_selfAdjoint_wrap {A B : L ℋ}
    (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B) :
    IsSelfAdjoint (blockDiagonal (ℋ := ℋ) A B)
```

lemma blockDiagonal_eq_blockOp_wrap [source]

```
private lemma blockDiagonal_eq_blockOp_wrap (A B : L ℋ) :
    blockDiagonal (ℋ := ℋ) A B = blockOp (ℋ := ℋ) A 0 0 B
```

lemma blockOp_mul_wrap [source]

```
private lemma blockOp_mul_wrap (A00 A01 A10 A11 B00 B01 B10 B11 : L ℋ) :
    blockOp (ℋ := ℋ) A00 A01 A10 A11 * blockOp (ℋ := ℋ) B00 B01 B10 B11 =
      blockOp (ℋ := ℋ)
        (A00 * B00 + A01 * B10)
        (A00 * B01 + A01 * B11)
        (A10 * B00 + A11 * B10)
        (A10 * B01 + A11 * B11)
```

lemma cfcR_blockDiagonal_wrap [source]

```
private lemma cfcR_blockDiagonal_wrap (f : ℝ → ℝ)
    (A B : L ℋ) (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B)
    (hcont : ContinuousOn f (spectrum ℝ A ∪ spectrum ℝ B)) :
    cfcR (ℋ := HSum ℋ) f (blockDiagonal (ℋ := ℋ) A B) =
      blockDiagonal (ℋ := ℋ) (cfcR (ℋ := ℋ) f A) (cfcR (ℋ := ℋ) f B)
```

lemma continuousOn_union_of_subset_Ici_wrap [source]

```
private lemma continuousOn_union_of_subset_Ici_wrap {f : ℝ → ℝ}
    (hcont : ContinuousOn f (Set.Ici (0 : ℝ))) {s t : Set ℝ}
    (hs : s ⊆ Set.Ici (0 : ℝ)) (ht : t ⊆ Set.Ici (0 : ℝ)) :
    ContinuousOn f (s ∪ t)
```

lemma spectrum_Ici_of_nonneg_wrap [source]

```
private lemma spectrum_Ici_of_nonneg_wrap {A : L ℋ} (hA0 : (0 : L ℋ) ≤ A) :
    spectrum ℝ A ⊆ Set.Ici (0 : ℝ)
```

lemma spectrum_zero_subset_Ici_wrap [source]

```
private lemma spectrum_zero_subset_Ici_wrap :
    spectrum ℝ (0 : L ℋ) ⊆ Set.Ici (0 : ℝ)
```

lemma blockDiagonal_le_left_wrap [source]

```
private lemma blockDiagonal_le_left_wrap {A0 A1 B0 B1 : L ℋ}
    (h : blockDiagonal (ℋ := ℋ) A0 A1 ≤ blockDiagonal (ℋ := ℋ) B0 B1) :
    A0 ≤ B0
```

theorem theorem_2_5_2_iv_imp_v [source]

```
theorem theorem_2_5_2_iv_imp_v {f : ℝ → ℝ} (hiv : CondIVAll.{u} f)
    (hcont : ContinuousOn f Set.univ) :
    CondV (ℋ := ℋ) f
```

theorem theorem_2_5_2_i_all_imp_v [source]

Uniform consequence of Theorem 2.5.2: `(i)` → `(v)` via `(iv)`.

```
theorem theorem_2_5_2_i_all_imp_v {f : ℝ → ℝ} (hf : CondIAll.{u} f) :
    CondV (ℋ := ℋ) f
```

theorem theorem_2_5_2_i_ici_all_imp_v [source]

```
theorem theorem_2_5_2_i_ici_all_imp_v {f : ℝ → ℝ}
    (hf : CondIciAll.{u} f) :
    CondV (ℋ := ℋ) f
```

2.64 QuantumInfo.ForMathlib.HayataGroup.TraceInequality.JensenOperatorInequalityIImpIV

`QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperatorInequalityIImpIV.lean` on GitHub.

instance ContinuousFunctionalCalculus ℂ (L ℋ × L ℋ) IsStarNormal [source]

```
noncomputable local instance : ContinuousFunctionalCalculus ℂ (L ℋ × L ℋ) IsStarNormal
```

instance ContinuousFunctionalCalculus ℝ (L ℋ × L ℋ) IsSelfAdjoint [source]

```
noncomputable local instance : ContinuousFunctionalCalculus ℝ (L ℋ × L ℋ) IsSelfAdjoint
```

instance NonnegSpectrumClass ℝ (L (HSum ℋ)) [source]

```
noncomputable local instance : NonnegSpectrumClass ℝ (L (HSum ℋ))
```

def CondIV [source]

Condition (iv) in Theorem 2.5.2.

```
def CondIV (f : ℝ → ℝ) : Prop
```

def CondI [source]

Condition (i) in Theorem 2.5.2 on the fixed Hilbert space ℋ.

```
def CondI (f : ℝ → ℝ) : Prop
```

def CondIAll [source]

Uniform version of Condition (i), packaged as `OperatorConvexAll` together with `f 0 ≤ 0`.

```
def CondIAll (f : ℝ → ℝ) : Prop
```

def CondIciAll [source]

Uniform localized version of Condition (i), packaged as `OperatorConvexOnAll (Set.Ici 0)` together with continuity and `f 0 ≤ 0`.

```
def CondIciAll (f : ℝ → ℝ) : Prop
```

def CondV [source]

Condition (v) in Theorem 2.5.2.

```
def CondV (f : ℝ → ℝ) : Prop
```

def blockSwap [source]

Selfadjoint `2 × 2` block built from a contraction candidate `X`.

```
private noncomputable def blockSwap (X : L ℋ) : L (HSum ℋ)
```

lemma blockSwap_star [source]

```
private lemma blockSwap_star (X : L ℋ) :
    star (blockSwap (ℋ := ℋ) X) = blockSwap (ℋ := ℋ) X
```

lemma blockSwap_sq [source]

```
private lemma blockSwap_sq (X : L ℋ) :
    blockSwap (ℋ := ℋ) X * blockSwap (ℋ := ℋ) X =
      blockDiagonal (ℋ := ℋ) (star X * X) (X * star X)
```

lemma blockDiagonal_eq_blockOp [source]

```
private lemma blockDiagonal_eq_blockOp (A B : L ℋ) :
    blockDiagonal (ℋ := ℋ) A B = blockOp (ℋ := ℋ) A 0 0 B
```

lemma blockOp_mul [source]

```
private lemma blockOp_mul (A00 A01 A10 A11 B00 B01 B10 B11 : L ℋ) :
    blockOp (ℋ := ℋ) A00 A01 A10 A11 * blockOp (ℋ := ℋ) B00 B01 B10 B11 =
      blockOp (ℋ := ℋ)
        (A00 * B00 + A01 * B10)
        (A00 * B01 + A01 * B11)
        (A10 * B00 + A11 * B10)
        (A10 * B01 + A11 * B11)
```

lemma blockOp_add [source]

```
private lemma blockOp_add
    (A00 A01 A10 A11 B00 B01 B10 B11 : L ℋ) :
    blockOp (ℋ := ℋ) A00 A01 A10 A11 + blockOp (ℋ := ℋ) B00 B01 B10 B11 =
      blockOp (ℋ := ℋ) (A00 + B00) (A01 + B01) (A10 + B10) (A11 + B11)
```

lemma blockOp_smulR [source]

```
private lemma blockOp_smulR
    (r : ℝ) (A00 A01 A10 A11 : L ℋ) :
    r • blockOp (ℋ := ℋ) A00 A01 A10 A11 =
      blockOp (ℋ := ℋ) (r • A00) (r • A01) (r • A10) (r • A11)
```

lemma blockSwap_add_I_smul_blockDiagonal [source]

```
private lemma blockSwap_add_I_smul_blockDiagonal (X R0 R1 : L ℋ) :
    blockSwap (ℋ := ℋ) X + Complex.I • blockDiagonal (ℋ := ℋ) R0 R1 =
      blockOp (ℋ := ℋ) (Complex.I • R0) (star X) X (Complex.I • R1)
```

lemma blockOp_mul_blockDiagonal_zero_right [source]

```
private lemma blockOp_mul_blockDiagonal_zero_right
    (P00 P01 P10 P11 A Q00 Q01 Q10 Q11 : L ℋ) :
    blockOp (ℋ := ℋ) P00 P01 P10 P11 * blockDiagonal (ℋ := ℋ) 0 A *
        blockOp (ℋ := ℋ) Q00 Q01 Q10 Q11 =
      blockOp (ℋ := ℋ)
```

```
        (P01 * A * Q10)
        (P01 * A * Q11)
        (P11 * A * Q10)
        (P11 * A * Q11)
```

lemma cfcR_blockDiagonal [source]

```
private lemma cfcR_blockDiagonal (f : ℝ → ℝ)
    (A B : L ℋ) (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B)
    (hcont : ContinuousOn f (spectrum ℝ A ∪ spectrum ℝ B)) :
    cfcR (ℋ := HSum ℋ) f (blockDiagonal (ℋ := ℋ) A B) =
      blockDiagonal (ℋ := ℋ) (cfcR (ℋ := ℋ) f A) (cfcR (ℋ := ℋ) f B)
```

lemma blockDiagonal_le_left [source]

```
private lemma blockDiagonal_le_left {A0 A1 B0 B1 : L ℋ}
    (h : blockDiagonal (ℋ := ℋ) A0 A1 ≤ blockDiagonal (ℋ := ℋ) B0 B1) :
    A0 ≤ B0
```

lemma blockDiagonal_selfAdjoint [source]

```
private lemma blockDiagonal_selfAdjoint {A B : L ℋ}
    (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B) :
    IsSelfAdjoint (blockDiagonal (ℋ := ℋ) A B)
```

lemma cfcR_zero [source]

```
private lemma cfcR_zero (f : ℝ → ℝ) :
    cfcR (ℋ := ℋ) f (0 : L ℋ) = algebraMap ℝ (L ℋ) (f 0)
```

lemma cfcR_conj_unitary [source]

```
private lemma cfcR_conj_unitary (f : ℝ → ℝ) (hcont : ContinuousOn f Set.univ)
    (u : unitary (L ℋ)) (A : L ℋ) (hA : IsSelfAdjoint A) :
    cfcR (ℋ := ℋ) f (star u * A * u) = star u * cfcR (ℋ := ℋ) f A * u
```

lemma cfcR_conj_unitary_on [source]

```
private lemma cfcR_conj_unitary_on (s : Set ℝ) (f : ℝ → ℝ) (hcont : ContinuousOn f s)
    {A : L ℋ} (hAs : spectrum ℝ A ⊆ s)
    (u : unitary (L ℋ)) (hA : IsSelfAdjoint A) :
    cfcR (ℋ := ℋ) f (star u * A * u) = star u * cfcR (ℋ := ℋ) f A * u
```

lemma cfcR_real_sqrt_eq_sqrt [source]

```
private lemma cfcR_real_sqrt_eq_sqrt {A : L ℋ} (hA : (0 : L ℋ) ≤ A) :
    cfcR (ℋ := ℋ) Real.sqrt A = CFC.sqrt A
```

theorem nontrivial_hsumL [source]

```
private theorem nontrivial_hsumL [Nontrivial ℋ] : Nontrivial (L (HSum ℋ))
```

lemma sqrt_blockDiagonal_of_nonneg [source]

```
private lemma sqrt_blockDiagonal_of_nonneg
    [Nontrivial ℋ]
    {A B : L ℋ} (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B)
    (hA_nonneg : (0 : L ℋ) ≤ A) (hB_nonneg : (0 : L ℋ) ≤ B) :
    CFC.sqrt (blockDiagonal (ℋ := ℋ) A B) =
      blockDiagonal (ℋ := ℋ) (CFC.sqrt A) (CFC.sqrt B)
```

lemma complex_I_smul_real_I_smul_invTwo [source]

```
private lemma complex_I_smul_real_I_smul_invTwo (r : ℝ) (T : L ℋ) :
    Complex.I • r • Complex.I • (2⁻¹ : ℝ) • T =
      -((2⁻¹ : ℝ) * r) • T
```

lemma real_smul_complex_I_real_smul_complex_I_comm [source]

```
private lemma real_smul_complex_I_real_smul_complex_I_comm (s r : ℝ) (T : L ℋ) :
    (s : ℝ) • Complex.I • r • Complex.I • T =
      Complex.I • r • Complex.I • (s : ℝ) • T
```

lemma half_add_half_eq [source]

```
private lemma half_add_half_eq (T : L ℋ) :
    (2⁻¹ : ℝ) • T + (2⁻¹ : ℝ) • T = T
```

lemma half_mul_real_add_half_mul_real_eq [source]

```
private lemma half_mul_real_add_half_mul_real_eq (r : ℝ) (T : L ℋ) :
    ((2⁻¹ : ℝ) * r) • T + ((2⁻¹ : ℝ) * r) • T = r • T
```

lemma rightEval_topLeft_scalar [source]

```
private lemma rightEval_topLeft_scalar
    (r : ℝ) (R0 X T : L ℋ) :
    (2⁻¹ : ℝ) • (star X * (T * X)) +
        ((2⁻¹ : ℝ) • (star X * (T * X)) +
          (-((2⁻¹ : ℝ) • Complex.I • r • Complex.I • (R0 * R0)) +
            -((2⁻¹ : ℝ) • Complex.I • r • Complex.I • (R0 * R0)))) =
      star X * (T * X) + r • (R0 * R0)
```

lemma rightEval_bottomRight_scalar [source]

```
private lemma rightEval_bottomRight_scalar
    (r : ℝ) (R1 X T : L ℋ) :
    (2⁻¹ * r) • (X * star X) +
        ((2⁻¹ * r) • (X * star X) +
          ((2⁻¹ : ℝ) • (R1 * (T * R1)) +
            (2⁻¹ : ℝ) • (R1 * (T * R1)))) =
      (R1 * T * R1) + r • (X * star X)
```

lemma star_mul_le_one [source]

```
private lemma star_mul_le_one [Nontrivial ℋ] (X : L ℋ) (hX : ‖X‖ ≤ 1) :
    (star X * X : L ℋ) ≤ 1
```

lemma mul_star_le_one [source]

```
private lemma mul_star_le_one [Nontrivial ℋ] (X : L ℋ) (hX : ‖X‖ ≤ 1) :
    (X * star X : L ℋ) ≤ 1
```

lemma blockSwap_norm_le_one [source]

```
private lemma blockSwap_norm_le_one [Nontrivial ℋ] (X : L ℋ) (hX : ‖X‖ ≤ 1) :
    ‖blockSwap (ℋ := ℋ) X‖ ≤ 1
```

lemma continuousOn_union_of_subset_Ici [source]

```
private lemma continuousOn_union_of_subset_Ici {f : ℝ → ℝ}
    (hcont : ContinuousOn f (Set.Ici (0 : ℝ))) {s t : Set ℝ}
    (hs : s ⊆ Set.Ici (0 : ℝ)) (ht : t ⊆ Set.Ici (0 : ℝ)) :
    ContinuousOn f (s ∪ t)
```

lemma spectrum_Ici_of_nonneg [source]

```
private lemma spectrum_Ici_of_nonneg {A : L ℋ} (hA0 : (0 : L ℋ) ≤ A) :
    spectrum ℝ A ⊆ Set.Ici (0 : ℝ)
```

lemma spectrum_zero_subset_Ici [source]

```
private lemma spectrum_zero_subset_Ici :
    spectrum ℝ (0 : L ℋ) ⊆ Set.Ici (0 : ℝ)
```

theorem theorem_2_5_2_i_ici_all_imp_iv [source]

```
theorem theorem_2_5_2_i_ici_all_imp_iv {f : ℝ → ℝ} (hf : CondIciAll.{u} f) :
    CondIV (ℋ := ℋ) f
```

theorem theorem_2_5_2_i_all_imp_iv [source]

```
theorem theorem_2_5_2_i_all_imp_iv {f : ℝ → ℝ} (hf : CondIAll.{u} f) :
    CondIV (ℋ := ℋ) f
```

2.65 QuantumInfo.ForMathlib.HayataGroup.TraceInequality.JensenOperatorInequalityIVtoV

QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperatorInequalityIVtoV.lean on GitHub.

instance ContinuousFunctionalCalculus ℂ (L ℋ × L ℋ) IsStarNormal [source]

```
noncomputable local instance : ContinuousFunctionalCalculus ℂ (L ℋ × L ℋ) IsStarNormal
```

instance ContinuousFunctionalCalculus ℝ (L $\mathcal{H}$ × L $\mathcal{H}$) IsSelfAdjoint [source]

```
noncomputable local instance : ContinuousFunctionalCalculus ℝ (L ℋ × L ℋ) IsSelfAdjoint
```

instance NonnegSpectrumClass ℝ (L (HSum $\mathcal{H}$)) [source]

```
noncomputable local instance : NonnegSpectrumClass ℝ (L (HSum ℋ))
```

def CondIV [source]

Local copy of condition (iv) for fast iteration on `(iv) → (v)`.

```
def CondIV (f : ℝ → ℝ) : Prop
```

def CondIVAll [source]

Uniform version of Condition (iv), with the Hilbert space arbitrary in the same universe. This scratch copy follows the same `...All` naming convention as the main Jensen file.

```
def CondIVAll (f : ℝ → ℝ) : Prop
```

def CondV [source]

Local copy of condition (v) for fast iteration on `(iv) → (v)`.

```
def CondV (f : ℝ → ℝ) : Prop
```

theorem nontrivial_hsumL [source]

`L (HSum ℋ)` is nontrivial once `L ℋ` is.

```
private theorem nontrivial_hsumL : Nontrivial (L (HSum ℋ))
```

lemma blockDiagonal_selfAdjoint [source]

```
private lemma blockDiagonal_selfAdjoint {A B : L ℋ}
    (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B) :
    IsSelfAdjoint (blockDiagonal (ℋ := ℋ) A B)
```

lemma blockDiagonal_eq_blockOp [source]

```
private lemma blockDiagonal_eq_blockOp (A B : L ℋ) :
    blockDiagonal (ℋ := ℋ) A B = blockOp (ℋ := ℋ) A 0 0 B
```

lemma blockOp_mul [source]

```
private lemma blockOp_mul (A00 A01 A10 A11 B00 B01 B10 B11 : L ℋ) :
    blockOp (ℋ := ℋ) A00 A01 A10 A11 * blockOp (ℋ := ℋ) B00 B01 B10 B11 =
      blockOp (ℋ := ℋ)
        (A00 * B00 + A01 * B10)
        (A00 * B01 + A01 * B11)
        (A10 * B00 + A11 * B10)
```

```
        (A10 * B01 + A11 * B11)
```

lemma cfcR_blockDiagonal [source]

```
private lemma cfcR_blockDiagonal (f : ℝ → ℝ)
    (A B : L ℋ) (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B)
    (hcont : ContinuousOn f (spectrum ℝ A ∪ spectrum ℝ B)) :
    cfcR (ℋ := HSum ℋ) f (blockDiagonal (ℋ := ℋ) A B) =
      blockDiagonal (ℋ := ℋ) (cfcR (ℋ := ℋ) f A) (cfcR (ℋ := ℋ) f B)
```

lemma blockDiagonal_le_left [source]

```
private lemma blockDiagonal_le_left {A0 A1 B0 B1 : L ℋ}
    (h : blockDiagonal (ℋ := ℋ) A0 A1 ≤ blockDiagonal (ℋ := ℋ) B0 B1) :
    A0 ≤ B0
```

theorem theorem_2_5_2_iv_imp_v [source]

```
theorem theorem_2_5_2_iv_imp_v {f : ℝ → ℝ} (hiv : CondIVAll.{u} f)
    (hcont : ContinuousOn f Set.univ) :
    CondV (ℋ := ℋ) f
```

2.66 QuantumInfo.ForMathlib.HayataGroup.TraceInequality.LiebAndoTrace

QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LiebAndoTrace.lean on GitHub.

instance NonnegSpectrumClass ℝ ((L ℋ)ᵐᵒᵖ) [source]

```
noncomputable local instance : NonnegSpectrumClass ℝ ((L ℋ)ᵐᵒᵖ)
```

instance IsometricContinuousFunctionalCalculus ℂ ((L ℋ)ᵐᵒᵖ) IsStarNormal [source]

```
noncomputable local instance :
    IsometricContinuousFunctionalCalculus ℂ ((L ℋ)ᵐᵒᵖ) IsStarNormal
```

instance instCFCRealSelfAdjointMop [source]

```
noncomputable instance instCFCRealSelfAdjointMop :
    ContinuousFunctionalCalculus ℝ ((L ℋ)ᵐᵒᵖ) IsSelfAdjoint
```

def traceRe [source]

The real part of the finite-dimensional trace on bounded operators.

```
noncomputable def traceRe (T : L ℋ) : ℝ
```

def liebTraceMap [source]

Trace functional appearing in Lieb's concavity theorem.

```
noncomputable def liebTraceMap (s : ℝ) (K : L ℋ) (A B : L ℋ) : ℝ
```

def liebExtensionTraceMap [source]

Trace functional appearing in Lieb's extension theorem.

```
noncomputable def liebExtensionTraceMap (q p : ℝ) (K : L ℋ) (A B : L ℋ) : ℝ
```

def liebCorollaryTraceMap [source]

Trace functional appearing in Corollary 1.3.

```
noncomputable def liebCorollaryTraceMap (q r : ℝ) (K : L ℋ) (A B : L ℋ) : ℝ
```

def andoTraceMap [source]

Trace functional appearing in Ando's convexity theorem.

```
noncomputable def andoTraceMap (q r : ℝ) (K : L ℋ) (A B : L ℋ) : ℝ
```

lemma rightMulHS_real_smul_one [source]

```
private lemma rightMulHS_real_smul_one (r : ℝ) :
    rightMulHS (ℋ := ℋ) (r • (1 : L ℋ)) = r • (1 : L (HSOp ℋ))
```

lemma rightMulHS_nonneg [source]

```
private lemma rightMulHS_nonneg {A : L ℋ} (hA0 : 0 ≤ A) :
    0 ≤ rightMulHS (ℋ := ℋ) A
```

lemma rightMulHS_le_rightMulHS [source]

```
private lemma rightMulHS_le_rightMulHS {A B : L ℋ} (hAB : A ≤ B) :
    rightMulHS (ℋ := ℋ) A ≤ rightMulHS (ℋ := ℋ) B
```

lemma rightMulHS_pdSet [source]

```
private lemma rightMulHS_pdSet {A : L ℋ} (hA : A ∈ pdSet (ℋ := ℋ)) :
    rightMulHS (ℋ := ℋ) A ∈ pdSet (ℋ := HSOp ℋ)
```

def phiK [source]

```
private noncomputable def phiK (K : L ℋ) (T : L (HSOp ℋ)) : ℝ
```

lemma phiK_nonneg [source]

```
private lemma phiK_nonneg (K : L ℋ) {T : L (HSOp ℋ)} (hT : 0 ≤ T) :
    0 ≤ phiK (ℋ := ℋ) K T
```

lemma phiK_add [source]

```
private lemma phiK_add (K : L ℋ) (T S : L (HSOp ℋ)) :
    phiK (ℋ := ℋ) K (T + S) = phiK (ℋ := ℋ) K T + phiK (ℋ := ℋ) K S
```

lemma phiK_smul [source]

```
private lemma phiK_smul (K : L ℋ) (r : ℝ) (T : L (HSOp ℋ)) :
    phiK (ℋ := ℋ) K (r • T) = r * phiK (ℋ := ℋ) K T
```

lemma phiK_mono [source]

```
private lemma phiK_mono (K : L ℋ) {T S : L (HSOp ℋ)} (hTS : T ≤ S) :
    phiK (ℋ := ℋ) K T ≤ phiK (ℋ := ℋ) K S
```

lemma leftMulHS_rankOne [source]

```
private lemma leftMulHS_rankOne (A : L ℋ) (x y : ℋ) :
    leftMulHS (ℋ := ℋ) A (ofOp (InnerProductSpace.rankOne ℂ x y)) =
      ofOp (InnerProductSpace.rankOne ℂ (A x) y)
```

lemma rightMulHS_rankOne [source]

```
private lemma rightMulHS_rankOne (B : L ℋ) (x y : ℋ) :
    rightMulHS (ℋ := ℋ) B (ofOp (InnerProductSpace.rankOne ℂ x y)) =
      ofOp (InnerProductSpace.rankOne ℂ x ((star B) y))
```

lemma re_inner_nonneg_of_nonneg [source]

```
private lemma re_inner_nonneg_of_nonneg
    {𝒦 : Type*} [NormedAddCommGroup 𝒦] [InnerProductSpace ℂ 𝒦]
    {T : 𝒦 →L[ℂ] 𝒦} (hT : 0 ≤ T) :
    ∀ x : 𝒦, 0 ≤ Complex.re (inner ℂ x (T x))
```

lemma aeval_apply_of_mem_eigenspace_realpoly [source]

```
private lemma aeval_apply_of_mem_eigenspace_realpoly
    {𝒦 : Type*} [NormedAddCommGroup 𝒦] [InnerProductSpace ℂ 𝒦]
    {T : 𝒦 →L[ℂ] 𝒦} {r : ℝ} {x : 𝒦}
    (hx : x ∈ eigenspace T.toLinearMap (r : ℂ)) (p : ℝ[X]) :
    Polynomial.aeval T (p.map (algebraMap ℝ ℂ)) x =
      ((p.map (algebraMap ℝ ℂ)).eval (r : ℂ)) • x
```

lemma cfcR_apply_of_mem_eigenspace_real [source]

```
private lemma cfcR_apply_of_mem_eigenspace_real
    {𝒦 : Type*} [NormedAddCommGroup 𝒦] [InnerProductSpace ℂ 𝒦] [CompleteSpace 𝒦]
    [FiniteDimensional ℂ 𝒦]
    [ContinuousFunctionalCalculus ℝ (L 𝒦) IsSelfAdjoint]
    (f : ℝ → ℝ) {T : L 𝒦} (hT : IsSelfAdjoint T) {r : ℝ} {x : 𝒦}
    (hx : x ∈ eigenspace T.toLinearMap (r : ℂ)) :
    cfcR (ℋ := 𝒦) f T x = (f r : ℂ) • x
```

lemma hmiddle_leftMul_rightMul [source]

```
private lemma hmiddle_leftMul_rightMul
    {s : ℝ} {A B : L ℋ}
    (hA : A ∈ pdSet (ℋ := ℋ)) (hB : B ∈ pdSet (ℋ := ℋ)) :
```

```
    cfcR (ℋ := HSOp ℋ) (fun x : ℝ ↦ x ^ s)
        (cfcR (ℋ := HSOp ℋ) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (rightMulHS (ℋ := ℋ) B) *
          leftMulHS (ℋ := ℋ) A *
          cfcR (ℋ := HSOp ℋ) (fun x : ℝ ↦ x ^ ((-1 : ℝ) / 2)) (rightMulHS (ℋ := ℋ) B)) =
      leftMulHS (ℋ := ℋ) (A ^ s) * rightMulHS (ℋ := ℋ) (B ^ (-s))
```

lemma phiK_operatorPowerMean_eq_liebTraceMap [source]

```
private lemma phiK_operatorPowerMean_eq_liebTraceMap
    {s : ℝ} (K A B : L ℋ) (hA : A ∈ pdSet (ℋ := ℋ)) (hB : B ∈ pdSet (ℋ := ℋ)) :
    phiK (ℋ := ℋ) K
        (operatorPowerMean (ℋ := HSOp ℋ) s 1
          (leftMulHS (ℋ := ℋ) A) (rightMulHS (ℋ := ℋ) B)) =
      liebTraceMap (ℋ := ℋ) s K A B
```

lemma pdSet_convexCombo [source]

Convex combinations preserve `pdSet` (strict positivity).

```
lemma pdSet_convexCombo {A B : L ℋ} {t : ℝ}
    (hA : A ∈ pdSet (ℋ := ℋ)) (hB : B ∈ pdSet (ℋ := ℋ))
    (ht0 : 0 ≤ t) (ht1 : t ≤ 1) :
    ((1 - t) • A + t • B) ∈ pdSet (ℋ := ℋ)
```

lemma phiK_leftMul_rightMul_eq_traceRe [source]

```
private lemma phiK_leftMul_rightMul_eq_traceRe (K C D : L ℋ) :
    phiK (ℋ := ℋ) K
        (leftMulHS (ℋ := ℋ) C * rightMulHS (ℋ := ℋ) D) =
      traceRe (ℋ := ℋ) (C * star K * D * K)
```

lemma pdSet_rpow_of_mem_Icc_zero_one [source]

```
private lemma pdSet_rpow_of_mem_Icc_zero_one
    {p : ℝ} (hp : p ∈ Set.Icc (0 : ℝ) 1) {A : L ℋ} (hA : A ∈ pdSet (ℋ := ℋ)) :
    A ^ p ∈ pdSet (ℋ := ℋ)
```

lemma liebTraceMap_mono_right [source]

```
private lemma liebTraceMap_mono_right
    {s : ℝ} (hs : 1 - s ∈ Set.Icc (0 : ℝ) 1)
    (K A B₁ B₂ : L ℋ)
    (hA : A ∈ pdSet (ℋ := ℋ)) (hB₁ : B₁ ∈ pdSet (ℋ := ℋ)) (hB₂ : B₂ ∈ pdSet (ℋ := ℋ))
    (hB : B₁ ≤ B₂) :
    liebTraceMap (ℋ := ℋ) s K A B₁ ≤ liebTraceMap (ℋ := ℋ) s K A B₂
```

lemma liebTraceMap_antitone_right [source]

```
private lemma liebTraceMap_antitone_right
    {s : ℝ} (hs : 1 - s ∈ Set.Icc (-1 : ℝ) 0)
    (K A B₁ B₂ : L ℋ)
    (hA : A ∈ pdSet (ℋ := ℋ)) (hB₁ : B₁ ∈ pdSet (ℋ := ℋ)) (hB₂ : B₂ ∈ pdSet (ℋ := ℋ))
    (hB : B₁ ≤ B₂) :
    liebTraceMap (ℋ := ℋ) s K A B₂ ≤ liebTraceMap (ℋ := ℋ) s K A B₁
```

lemma phiK_weightedSum_operatorPowerMean_eq [source]

```
private lemma phiK_weightedSum_operatorPowerMean_eq
    {s θ : ℝ} (K A₁ A₂ B₁ B₂ : L ℋ)
    (hA₁ : A₁ ∈ pdSet (ℋ := ℋ)) (hA₂ : A₂ ∈ pdSet (ℋ := ℋ))
    (hB₁ : B₁ ∈ pdSet (ℋ := ℋ)) (hB₂ : B₂ ∈ pdSet (ℋ := ℋ)) :
    phiK (ℋ := ℋ) K
        ((1 - θ) • operatorPowerMean (ℋ := HSOp ℋ) s 1
            (leftMulHS (ℋ := ℋ) A₁) (rightMulHS (ℋ := ℋ) B₁) +
          θ • operatorPowerMean (ℋ := HSOp ℋ) s 1
            (leftMulHS (ℋ := ℋ) A₂) (rightMulHS (ℋ := ℋ) B₂)) =
      (1 - θ) • liebTraceMap (ℋ := ℋ) s K A₁ B₁ +
        θ • liebTraceMap (ℋ := ℋ) s K A₂ B₂
```

theorem liebTrace_jointlyConcaveOn_pdSet [source]

```
theorem liebTrace_jointlyConcaveOn_pdSet
    {s : ℝ} (hs0 : 0 < s) (hs1 : s < 1) (K : L ℋ) :
    JointlyConcaveOn (pdSet (ℋ := ℋ)) (pdSet (ℋ := ℋ))
      (liebTraceMap (ℋ := ℋ) s K)
```

theorem liebTrace_jointlyConvexOn_pdSet [source]

```
theorem liebTrace_jointlyConvexOn_pdSet
    {s : ℝ} (hs1 : 1 ≤ s) (hs2 : s ≤ 2) (K : L ℋ) :
    JointlyConvexOn (pdSet (ℋ := ℋ)) (pdSet (ℋ := ℋ))
      (liebTraceMap (ℋ := ℋ) s K)
```

theorem liebExtensionTrace_jointlyConcaveOn_pdSet [source]

```
theorem liebExtensionTrace_jointlyConcaveOn_pdSet
    {p q : ℝ} (hp : 0 < p) (hq : 0 < q) (hpq : p + q ≤ 1) (K : L ℋ) :
    JointlyConcaveOn (pdSet (ℋ := ℋ)) (pdSet (ℋ := ℋ))
      (liebExtensionTraceMap (ℋ := ℋ) q p K)
```

theorem andoTrace_jointlyConvexOn_pdSet [source]

```
theorem andoTrace_jointlyConvexOn_pdSet
    {q r : ℝ} (hq1 : 1 ≤ q) (hq2 : q ≤ 2) (hr0 : 0 ≤ r) (hr1 : r ≤ 1)
    (hqr : 1 ≤ q - r) (K : L ℋ) :
    JointlyConvexOn (pdSet (ℋ := ℋ)) (pdSet (ℋ := ℋ))
      (andoTraceMap (ℋ := ℋ) q r K)
```

theorem liebCorollaryTrace_jointlyConvexOn_pdSet [source]

```
theorem liebCorollaryTrace_jointlyConvexOn_pdSet
    {q r : ℝ} (hr1 : 1 < r) (hrq : r ≤ q) (hq2 : q ≤ 2) (K : L ℋ) :
    JointlyConvexOn (pdSet (ℋ := ℋ)) (pdSet (ℋ := ℋ))
      (liebCorollaryTraceMap (ℋ := ℋ) q r K)
```

2.67 QuantumInfo.ForMathlib.HayataGroup.TraceInequality.LownerHeinzCore

QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LownerHeinzCore.lean on GitHub.

abbrev cfcR [source]

```
noncomputable abbrev cfcR (f : ℝ → ℝ) (A : 𝓐) : 𝓐
```

def OperatorMonotone [source]

Fixed-space operator monotonicity on the ambient algebra $\mathcal{A}$.

```
def OperatorMonotone (f : ℝ → ℝ) : Prop
```

def OperatorMonotoneOn [source]

Fixed-space operator monotonicity on `s` for the ambient algebra $\mathcal{A}$.

```
def OperatorMonotoneOn (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorAntitone [source]

Fixed-space operator antitonicity on the ambient algebra $\mathcal{A}$.

```
def OperatorAntitone (f : ℝ → ℝ) : Prop
```

def OperatorAntitoneOn [source]

Fixed-space operator antitonicity on `s` for the ambient algebra $\mathcal{A}$.

```
def OperatorAntitoneOn (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorConvex [source]

Fixed-space operator convexity on the ambient algebra $\mathcal{A}$.

```
def OperatorConvex (f : ℝ → ℝ) : Prop
```

def OperatorConvexOn [source]

Fixed-space operator convexity on `s` for the ambient algebra $\mathcal{A}$.

```
def OperatorConvexOn (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorConcave [source]

Fixed-space operator concavity on the ambient algebra $\mathcal{A}$.

```
def OperatorConcave (f : ℝ → ℝ) : Prop
```

def OperatorConcaveOn [source]

Fixed-space operator concavity on `s` for the ambient algebra $\mathcal{A}$.

```
def OperatorConcaveOn (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorMonotoneAll [source]

Uniform operator monotonicity over all ambient algebras in universe `u`.

```
def OperatorMonotoneAll (f : ℝ → ℝ) : Prop
```

def OperatorMonotoneOnAll [source]

Uniform operator monotonicity on `s` over all ambient algebras in universe `u`.

```
def OperatorMonotoneOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorAntitoneAll [source]

Uniform operator antitonicity over all ambient algebras in universe `u`.

```
def OperatorAntitoneAll (f : ℝ → ℝ) : Prop
```

def OperatorAntitoneOnAll [source]

Uniform operator antitonicity on `s` over all ambient algebras in universe `u`.

```
def OperatorAntitoneOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorConvexAll [source]

Uniform operator convexity over all ambient algebras in universe `u`.

```
def OperatorConvexAll (f : ℝ → ℝ) : Prop
```

def OperatorConvexOnAll [source]

Uniform operator convexity on `s` over all ambient algebras in universe `u`.

```
def OperatorConvexOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorConcaveAll [source]

Uniform operator concavity over all ambient algebras in universe `u`.

```
def OperatorConcaveAll (f : ℝ → ℝ) : Prop
```

def OperatorConcaveOnAll [source]

Uniform operator concavity on `s` over all ambient algebras in universe `u`.

```
def OperatorConcaveOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

lemma conjugate_isPositive [source]

```
lemma conjugate_isPositive {X T : 𝒜} (hX : 0 ≤ X) (hT : IsSelfAdjoint T) :
    0 ≤ T * X * T
```

theorem LownerHeinzCore.one_div_operatorAntitoneOn_Ioi [source]

```
theorem one_div_operatorAntitoneOn_Ioi :
    OperatorAntitoneOn (𝒜 := 𝒜) (Set.Ioi (0 : ℝ)) (fun x : ℝ ↦ 1 / x)
```

lemma spectrum_convexCombo_Ioi [source]

```
private lemma spectrum_convexCombo_Ioi {A B : 𝒜} {t : ℝ}
    (hA : IsSelfAdjoint A) (hB : IsSelfAdjoint B) (ht0 : 0 ≤ t) (ht1 : t ≤ 1)
    (As : spectrum ℝ A ⊆ Set.Ioi (0 : ℝ)) (Bs : spectrum ℝ B ⊆ Set.Ioi (0 : ℝ)) :
    spectrum ℝ ((1 - t) • A + t • B) ⊆ Set.Ioi (0 : ℝ)
```

lemma posSemidef_block_one_inv [source]

```
private lemma posSemidef_block_one_inv {A : 𝒜} (hA : IsSelfAdjoint A)
    (As : spectrum ℝ A ⊆ Set.Ioi (0 : ℝ)) :
    Matrix.PosSemidef
      (!![A, 1; 1, cfcR (fun x : ℝ ↦ x⁻¹) A] : Matrix (Fin 2) (Fin 2) 𝒜)
```

lemma schur_conj_eq_diagonal [source]

```
private lemma schur_conj_eq_diagonal {C D invC : 𝒜} (hInvC_sa : IsSelfAdjoint invC)
    (invC_mul_C : invC * C = (1 : 𝒜)) (C_mul_invC : C * invC = (1 : 𝒜)) :
    star (!![(1 : 𝒜), -invC; 0, 1] : Matrix (Fin 2) (Fin 2) (𝒜))
        * (!![C, 1; 1, D] : Matrix (Fin 2) (Fin 2) (𝒜))
        * (!![(1 : 𝒜), -invC; 0, 1] : Matrix (Fin 2) (Fin 2) (𝒜))
      = Matrix.diagonal (fun i : Fin 2 => if i = 0 then C else D - invC)
```

theorem LownerHeinzCore.one_div_operatorConvexOn_Ioi [source]

```
theorem one_div_operatorConvexOn_Ioi :
  OperatorConvexOn (𝒜 := 𝒜) (Set.Ioi (0 : ℝ)) (fun x : ℝ ↦ 1 / x)
```

theorem LownerHeinzCore.one_div_add_t_operatorAntitoneOn_Ici [source]

```
theorem one_div_add_t_operatorAntitoneOn_Ici : ∀ (t : ℝ), 0 < t →
  OperatorAntitoneOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ 1 / (x + t))
```

theorem LownerHeinzCore.one_div_add_t_operatorConvexOn_Ici [source]

```
theorem one_div_add_t_operatorConvexOn_Ici : ∀ (t : ℝ), 0 < t →
  OperatorConvexOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ 1 / (x + t))
```

theorem LownerHeinzCore.ratio_add_t_operatorMonotoneOn_Ici [source]

```
theorem ratio_add_t_operatorMonotoneOn_Ici : ∀ (t : ℝ), 0 < t →
  OperatorMonotoneOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ x / (x + t))
```

theorem LownerHeinzCore.ratio_add_t_operatorConcaveOn_Ici [source]

```
theorem ratio_add_t_operatorConcaveOn_Ici : ∀ (t : ℝ), 0 < t →
  OperatorConcaveOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ x / (x + t))
```

theorem LownerHeinzCore.power_Icc_zero_one_operatorMonotoneOn_Ici [source]

```
theorem power_Icc_zero_one_operatorMonotoneOn_Ici : ∀ p ∈ Set.Icc (0 : ℝ) 1,
  OperatorMonotoneOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p)
```

lemma cfcₙ_rpowIntegrand₀₁_eq_smul_cfcR_ratio [source]

```
private lemma cfcₙ_rpowIntegrand₀₁_eq_smul_cfcR_ratio {q : NNReal} (hq : q ∈ Set.Ioo (0 :
    NNReal) 1)
    {t : ℝ} (htpos : 0 < t) (X : 𝒜) (hX0 : 0 ≤ X) :
    cfcₙ (Real.rpowIntegrand₀₁ (q : ℝ) t) X =
      (t ^ ((q : ℝ) - 1)) • cfcR (fun x : ℝ => x / (x + t)) X
```

lemma cfcR_ratio_weighted_le [source]

```
private lemma cfcR_ratio_weighted_le {t : ℝ} (htpos : 0 < t) {A B : 𝒜} (hA0 : 0 ≤ A) (hB0 : 0
    ≤ B)
    {a b : ℝ} (ha : 0 ≤ a) (hb : 0 ≤ b) (hab : a + b = 1) :
    a • cfcR (fun x : ℝ => x / (x + t)) A + b • cfcR (fun x : ℝ => x / (x + t)) B
      ≤ cfcR (fun x : ℝ => x / (x + t)) (a • A + b • B)
```

lemma concaveOn_cfcₙ_rpowIntegrand₀₁ [source]

```
private lemma concaveOn_cfcₙ_rpowIntegrand₀₁ {q : NNReal} (hq : q ∈ Set.Ioo (0 : NNReal) 1)
    {t : ℝ} (htpos : 0 < t) :
    ConcaveOn ℝ (Set.Ici (0 : 𝒜)) (fun A : 𝒜 => cfcₙ (Real.rpowIntegrand₀₁ (q : ℝ) t) A)
```

lemma concaveOn_nnrpow_Ioo [source]

```
private lemma concaveOn_nnrpow_Ioo {q : NNReal} (hq : q ∈ Set.Ioo (0 : NNReal) 1) :
    ConcaveOn ℝ (Set.Ici (0 : 𝒜)) (fun A : 𝒜 ↦ A ^ q)
```

lemma concaveOn_rpow_Ioo [source]

```
private lemma concaveOn_rpow_Ioo {p : ℝ} (hp : p ∈ Set.Ioo (0 : ℝ) 1) :
    ConcaveOn ℝ (Set.Ici (0 : 𝒜)) (fun A : 𝒜 ↦ A ^ p)
```

theorem LownerHeinzCore.power_Icc_zero_one_operatorConcaveOn_Ici [source]

```
theorem power_Icc_zero_one_operatorConcaveOn_Ici : ∀ p ∈ Set.Icc (0 : ℝ) 1,
  OperatorConcaveOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p)
```

lemma sq_mul_div_add [source]

```
private lemma sq_mul_div_add (x t : ℝ) (hxt : x + t ≠ 0) :
    (x * x) / (x + t) = x - t + (t * t) / (x + t)
```

lemma convexOn_cfcR_one_div_add_t [source]

```
private lemma convexOn_cfcR_one_div_add_t (t : ℝ) (htpos : 0 < t) :
    ConvexOn ℝ (Set.Ici (0 : 𝒜)) (fun X : 𝒜 ↦ cfcR (fun x : ℝ ↦ 1 / (x + t)) X)
```

lemma G_eqOn_rpowIntegrand₀₁_mul [source]

```
private lemma G_eqOn_rpowIntegrand₀₁_mul {q : NNReal} (hq_real : (q : ℝ) ∈ Set.Ioo (0 : ℝ) 1)
    (t : ℝ) (htpos : 0 < t) :
    (Set.Ici (0 : 𝒜)).EqOn
      (fun X : 𝒜 ↦ cfcₙ (fun x : ℝ ↦ x * Real.rpowIntegrand₀₁ (q : ℝ) t x) X)
      (fun X : 𝒜 ↦ (t ^ ((q : ℝ) - 1)) •
        (X - algebraMap ℝ (𝒜) t + (t ^ (2 : ℕ)) • cfcR (fun x : ℝ ↦ 1 / (x + t)) X))
```

lemma convexOn_G_rpowIntegrand₀₁_mul [source]

```
private lemma convexOn_G_rpowIntegrand₀₁_mul {q : NNReal} (hq_real : (q : ℝ) ∈ Set.Ioo (0 : ℝ)
    1)
    (t : ℝ) (htpos : 0 < t) :
    ConvexOn ℝ (Set.Ici (0 : 𝒜))
      (fun X : 𝒜 ↦ cfcₙ (fun x : ℝ ↦ x * Real.rpowIntegrand₀₁ (q : ℝ) t x) X)
```

lemma ae_cfcₙ_mul_id_rpowIntegrand₀₁_restrict_Ioi [source]

```
private lemma ae_cfcₙ_mul_id_rpowIntegrand₀₁_restrict_Ioi {q : NNReal} (hq_real : (q : ℝ) ∈ Set.
    Ioo (0 : ℝ) 1)
    (μ : MeasureTheory.Measure ℝ) (A : 𝒜) (hA0 : 0 ≤ A) :
    ∀ᵐ t ∂(μ.restrict (Set.Ioi (0 : ℝ))),
      cfcₙ (fun x : ℝ ↦ x * Real.rpowIntegrand₀₁ (q : ℝ) t x) A =
        A * cfcₙ (Real.rpowIntegrand₀₁ (q : ℝ) t) A
```

lemma convexOn_nnrpow_Ioo_one_add [source]

```
private lemma convexOn_nnrpow_Ioo_one_add {q : NNReal} (hq : q ∈ Set.Ioo (0 : NNReal) 1) :
    ConvexOn ℝ (Set.Ici (0 : 𝒜)) (fun A : 𝒜 ↦ A ^ ((1 : NNReal) + q))
```

lemma convexOn_rpow_Ioo_one_two [source]

```
private lemma convexOn_rpow_Ioo_one_two {p : ℝ} (hp : p ∈ Set.Ioo (1 : ℝ) 2) :
    ConvexOn ℝ (Set.Ici (0 : 𝒜)) (fun A : 𝒜 ↦ A ^ p)
```

lemma cfcR_mul_self [source]

```
private lemma cfcR_mul_self (T : 𝒜) (hT : IsSelfAdjoint T) :
    cfcR (fun x : ℝ ↦ x * x) T = T * T
```

lemma sub_mul_sub [source]

```
private lemma sub_mul_sub (A B : 𝒜) :
    (A - B) * (A - B) = A * A - A * B - B * A + B * B
```

lemma smul_sub_mul_sub [source]

```
private lemma smul_sub_mul_sub (α : ℝ) (A B : 𝒜) :
    α • (A * A - A * B - B * A + B * B) =
      α • (A * A) - α • (A * B) - α • (B * A) + α • (B * B)
```

lemma square_convexity_diff_rhs [source]

```
private lemma square_convexity_diff_rhs (A B : 𝒜) (u : ℝ) :
    (u * (1 - u)) • ((A - B) * (A - B)) =
      (u * (1 - u)) • (A * A) - (u * (1 - u)) • (A * B) - (u * (1 - u)) • (B * A)
        + (u * (1 - u)) • (B * B)
```

lemma square_convexity_diff_hL [source]

```
private lemma square_convexity_diff_hL (A B : 𝒜) (u : ℝ) :
    (1 - u) • (A * A) + u • (B * B) -
        (((1 - u) * (1 - u)) • (A * A) + ((1 - u) * u) • (A * B)
          + (u * (1 - u)) • (B * A) + (u * u) • (B * B)) =
      (u * (1 - u)) • (A * A) - (u * (1 - u)) • (A * B) - (u * (1 - u)) • (B * A)
        + (u * (1 - u)) • (B * B)
```

lemma square_convexity_diff_hCC_sum [source]

```
private lemma square_convexity_diff_hCC_sum (A B : 𝒜) (u : ℝ) :
    ((1 - u) • A) * ((1 - u) • A)
      + ((1 - u) • A) * (u • B)
      + (u • B) * ((1 - u) • A)
      + (u • B) * (u • B) =
      ((1 - u) * (1 - u)) • (A * A) + ((1 - u) * u) • (A * B)
        + (u * (1 - u)) • (B * A) + (u * u) • (B * B)
```

lemma square_convexity_diff_hCC [source]

```
private lemma square_convexity_diff_hCC (A B : 𝒜) (u : ℝ) :
    ((1 - u) • A + u • B) * ((1 - u) • A + u • B) =
      ((1 - u) * (1 - u)) • (A * A) + ((1 - u) * u) • (A * B)
        + (u * (1 - u)) • (B * A) + (u * u) • (B * B)
```

lemma square_convexity_diff [source]

```
private lemma square_convexity_diff (A B : 𝒜) (u : ℝ) :
    (1 - u) • (A * A) + u • (B * B)
        - ((1 - u) • A + u • B) * ((1 - u) • A + u • B)
      =
      (u * (1 - u)) • ((A - B) * (A - B))
```

lemma operatorConvexOn_pow_two_Ici [source]

```
private lemma operatorConvexOn_pow_two_Ici :
    OperatorConvexOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ x ^ (2 : ℝ))
```

theorem LownerHeinzCore.power_Icc_one_two_operatorConvexOn_Ici [source]

```
theorem power_Icc_one_two_operatorConvexOn_Ici : ∀ p ∈ Set.Icc (1 : ℝ) 2,
  OperatorConvexOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p)
```

theorem LownerHeinzCore.power_Icc_neg_one_zero_neg_operatorMonotoneOn_Ioi [source]

```
theorem power_Icc_neg_one_zero_neg_operatorMonotoneOn_Ioi : ∀ p ∈ Set.Icc (-1 : ℝ) 0,
    OperatorMonotoneOn (𝒜 := 𝒜) (Set.Ioi (0 : ℝ)) (fun x ↦ -(x ^ p))
```

theorem LownerHeinzCore.power_Icc_neg_one_zero_neg_operatorConcaveOn_Ioi [source]

```
theorem power_Icc_neg_one_zero_neg_operatorConcaveOn_Ioi : ∀ p ∈ Set.Icc (-1 : ℝ) 0,
    OperatorConcaveOn (𝒜 := 𝒜) (Set.Ioi (0 : ℝ)) (fun x ↦ -(x ^ p))
```

instance PartialOrder $\mathcal{A}$ [source]

```
noncomputable local instance : PartialOrder 𝒜
```

instance StarOrderedRing $\mathcal{A}$ [source]

```
noncomputable local instance : StarOrderedRing 𝒜
```

instance NonnegSpectrumClass $\mathbb{R}$ $\mathcal{A}$ [source]

```
noncomputable local instance : NonnegSpectrumClass ℝ 𝒜
```

theorem Spectral.one_div_operatorAntitoneOn_Ioi [source]

```
theorem one_div_operatorAntitoneOn_Ioi :
    OperatorAntitoneOn (𝒜 := 𝒜) (Set.Ioi (0 : ℝ)) (fun x : ℝ ↦ 1 / x)
```

theorem Spectral.one_div_operatorConvexOn_Ioi [source]

```
theorem one_div_operatorConvexOn_Ioi :
    OperatorConvexOn (𝒜 := 𝒜) (Set.Ioi (0 : ℝ)) (fun x : ℝ ↦ 1 / x)
```

theorem Spectral.one_div_add_t_operatorAntitoneOn_Ici [source]

```
theorem one_div_add_t_operatorAntitoneOn_Ici : ∀ (t : ℝ), 0 < t →
    OperatorAntitoneOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ 1 / (x + t))
```

theorem Spectral.one_div_add_t_operatorConvexOn_Ici [source]

```
theorem one_div_add_t_operatorConvexOn_Ici : ∀ (t : ℝ), 0 < t →
    OperatorConvexOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ 1 / (x + t))
```

theorem Spectral.ratio_add_t_operatorMonotoneOn_Ici [source]

```
theorem ratio_add_t_operatorMonotoneOn_Ici : ∀ (t : ℝ), 0 < t →
    OperatorMonotoneOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ x / (x + t))
```

theorem Spectral.ratio_add_t_operatorConcaveOn_Ici [source]

```
theorem ratio_add_t_operatorConcaveOn_Ici : ∀ (t : ℝ), 0 < t →
    OperatorConcaveOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ x / (x + t))
```

theorem Spectral.power_Icc_zero_one_operatorMonotoneOn_Ici [source]

```
theorem power_Icc_zero_one_operatorMonotoneOn_Ici : ∀ p ∈ Set.Icc (0 : ℝ) 1,
    OperatorMonotoneOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p)
```

theorem Spectral.power_Icc_zero_one_operatorConcaveOn_Ici [source]

```
theorem power_Icc_zero_one_operatorConcaveOn_Ici : ∀ p ∈ Set.Icc (0 : ℝ) 1,
    OperatorConcaveOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p)
```

theorem Spectral.power_Icc_one_two_operatorConvexOn_Ici [source]

```
theorem power_Icc_one_two_operatorConvexOn_Ici : ∀ p ∈ Set.Icc (1 : ℝ) 2,
    OperatorConvexOn (𝒜 := 𝒜) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p)
```

theorem Spectral.power_Icc_neg_one_zero_neg_operatorMonotoneOn_Ioi [source]

```
theorem power_Icc_neg_one_zero_neg_operatorMonotoneOn_Ioi : ∀ p ∈ Set.Icc (-1 : ℝ) 0,
    OperatorMonotoneOn (𝒜 := 𝒜) (Set.Ioi (0 : ℝ)) (fun x ↦ -(x ^ p))
```

theorem Spectral.power_Icc_neg_one_zero_neg_operatorConcaveOn_Ioi [source]

```
theorem power_Icc_neg_one_zero_neg_operatorConcaveOn_Ioi : ∀ p ∈ Set.Icc (-1 : ℝ) 0,
    OperatorConcaveOn (𝒜 := 𝒜) (Set.Ioi (0 : ℝ)) (fun x ↦ -(x ^ p))
```

2.68 QuantumInfo.ForMathlib.HayataGroup.TraceInequality.LownerHeinzTheorem

QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LownerHeinzTheorem.lean on GitHub.

abbrev L [source]

```
abbrev L (ℋ : Type u) [NormedAddCommGroup ℋ] [InnerProductSpace ℂ ℋ] : Type u
```

instance instNontrivialL [source]

```
noncomputable instance instNontrivialL : Nontrivial (L ℋ)
```

instance NonnegSpectrumClass ℝ (L ℋ) [source]

```
noncomputable local instance : NonnegSpectrumClass ℝ (L ℋ)
```

instance instCFCRealSelfAdjoint [source]

```
noncomputable instance instCFCRealSelfAdjoint :
    ContinuousFunctionalCalculus ℝ (L ℋ) IsSelfAdjoint
```

abbrev cfcR [source]

```
noncomputable abbrev cfcR (f : ℝ → ℝ) (A : L ℋ) : L ℋ
```

def OperatorMonotone [source]

Fixed-space operator monotonicity on the Hilbert space $\mathcal{H}$.

```
def OperatorMonotone (f : ℝ → ℝ) : Prop
```

def OperatorMonotoneOn [source]

Fixed-space operator monotonicity on `s` for the Hilbert space $\mathcal{H}$.

```
def OperatorMonotoneOn (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorAntitone [source]

Fixed-space operator antitonicity on the Hilbert space $\mathcal{H}$.

```
def OperatorAntitone (f : ℝ → ℝ) : Prop
```

def OperatorAntitoneOn [source]

Fixed-space operator antitonicity on `s` for the Hilbert space $\mathcal{H}$.

```
def OperatorAntitoneOn (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorConvex [source]

Fixed-space operator convexity on the Hilbert space $\mathcal{H}$.

```
def OperatorConvex (f : ℝ → ℝ) : Prop
```

def OperatorConvexOn [source]

Fixed-space operator convexity on `s` for the Hilbert space $\mathcal{H}$.

```
def OperatorConvexOn (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorConcave [source]

Fixed-space operator concavity on the Hilbert space $\mathcal{H}$.

```
def OperatorConcave (f : ℝ → ℝ) : Prop
```

def OperatorConcaveOn [source]

Fixed-space operator concavity on `s` for the Hilbert space $\mathcal{H}$.

```
def OperatorConcaveOn (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorMonotoneAll [source]

Uniform operator monotonicity over all Hilbert spaces in universe `u`.

```
def OperatorMonotoneAll (f : ℝ → ℝ) : Prop
```

def OperatorMonotoneOnAll [source]

Uniform operator monotonicity on `s` over all Hilbert spaces in universe `u`.

```
def OperatorMonotoneOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorAntitoneAll [source]

Uniform operator antitonicity over all Hilbert spaces in universe `u`.

```
def OperatorAntitoneAll (f : ℝ → ℝ) : Prop
```

def OperatorAntitoneOnAll [source]

Uniform operator antitonicity on `s` over all Hilbert spaces in universe `u`.

```
def OperatorAntitoneOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorConvexAll [source]

Uniform operator convexity over all Hilbert spaces in universe `u`.

```
def OperatorConvexAll (f : ℝ → ℝ) : Prop
```

def OperatorConvexOnAll [source]

Uniform operator convexity on `s` over all Hilbert spaces in universe `u`.

```
def OperatorConvexOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

def OperatorConcaveAll [source]

Uniform operator concavity over all Hilbert spaces in universe `u`.

```
def OperatorConcaveAll (f : ℝ → ℝ) : Prop
```

def OperatorConcaveOnAll [source]

Uniform operator concavity on `s` over all Hilbert spaces in universe `u`.

```
def OperatorConcaveOnAll (s : Set ℝ) (f : ℝ → ℝ) : Prop
```

theorem operatorConvex_convexOn_univ [source]

```
theorem operatorConvex_convexOn_univ {f : ℝ → ℝ} (hf : OperatorConvex (ℋ := ℋ) f) :
    ConvexOn ℝ Set.univ f
```

theorem operatorConvex_continuousOn_univ [source]

```
theorem operatorConvex_continuousOn_univ {f : ℝ → ℝ} (hf : OperatorConvex (ℋ := ℋ) f) :
    ContinuousOn f Set.univ
```

theorem operatorConvex_continuousOn_spectrum_union [source]

```
theorem operatorConvex_continuousOn_spectrum_union {f : ℝ → ℝ}
    (hf : OperatorConvex (ℋ := ℋ) f) (A B : L ℋ) :
    ContinuousOn f (spectrum ℝ A ∪ spectrum ℝ B)
```

theorem one_div_operatorAntitoneOn_Ioi [source]

```
theorem one_div_operatorAntitoneOn_Ioi :
    OperatorAntitoneOn (ℋ := ℋ) (Set.Ioi (0 : ℝ)) (fun x : ℝ ↦ 1 / x)
```

theorem one_div_operatorConvexOn_Ioi [source]

```
theorem one_div_operatorConvexOn_Ioi :
    OperatorConvexOn (ℋ := ℋ) (Set.Ioi (0 : ℝ)) (fun x : ℝ ↦ 1 / x)
```

theorem one_div_add_t_operatorAntitoneOn_Ici [source]

```
theorem one_div_add_t_operatorAntitoneOn_Ici : ∀ (t : ℝ), 0 < t →
    OperatorAntitoneOn (ℋ := ℋ) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ 1 / (x + t))
```

theorem one_div_add_t_operatorConvexOn_Ici [source]

```
theorem one_div_add_t_operatorConvexOn_Ici : ∀ (t : ℝ), 0 < t →
    OperatorConvexOn (ℋ := ℋ) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ 1 / (x + t))
```

theorem ratio_add_t_operatorMonotoneOn_Ici [source]

```
theorem ratio_add_t_operatorMonotoneOn_Ici : ∀ (t : ℝ), 0 < t →
    OperatorMonotoneOn (ℋ := ℋ) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ x / (x + t))
```

theorem ratio_add_t_operatorConcaveOn_Ici [source]

```
theorem ratio_add_t_operatorConcaveOn_Ici : ∀ (t : ℝ), 0 < t →
    OperatorConcaveOn (ℋ := ℋ) (Set.Ici (0 : ℝ)) (fun x : ℝ ↦ x / (x + t))
```

theorem power_Icc_zero_one_operatorMonotoneOn_Ici [source]

```
theorem power_Icc_zero_one_operatorMonotoneOn_Ici : ∀ p ∈ Set.Icc (0 : ℝ) 1,
    OperatorMonotoneOn (ℋ := ℋ) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p)
```

theorem power_Icc_zero_one_operatorConcaveOn_Ici [source]

```
theorem power_Icc_zero_one_operatorConcaveOn_Ici : ∀ p ∈ Set.Icc (0 : ℝ) 1,
    OperatorConcaveOn (ℋ := ℋ) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p)
```

theorem power_Icc_one_two_operatorConvexOn_Ici [source]

```
theorem power_Icc_one_two_operatorConvexOn_Ici : ∀ p ∈ Set.Icc (1 : ℝ) 2,
    OperatorConvexOn (ℋ := ℋ) (Set.Ici (0 : ℝ)) (fun x ↦ x ^ p)
```

theorem power_Icc_neg_one_zero_neg_operatorMonotoneOn_Ioi [source]

```
theorem power_Icc_neg_one_zero_neg_operatorMonotoneOn_Ioi : ∀ p ∈ Set.Icc (-1 : ℝ) 0,
    OperatorMonotoneOn (ℋ := ℋ) (Set.Ioi (0 : ℝ)) (fun x ↦ -(x ^ p))
```

theorem power_Icc_neg_one_zero_neg_operatorConcaveOn_Ioi [source]

```
theorem power_Icc_neg_one_zero_neg_operatorConcaveOn_Ioi : ∀ p ∈ Set.Icc (-1 : ℝ) 0,
    OperatorConcaveOn (ℋ := ℋ) (Set.Ioi (0 : ℝ)) (fun x ↦ -(x ^ p))
```

2.69 QuantumInfo.ForMathlib.HayataGroup.TraceInequality.OperatorGeometricMean

QuantumInfo/ForMathlib/HayataGroup/TraceInequality/OperatorGeometricMean.lean on GitHub.

def operatorPowerMean [source]

The operator `(α, β)`-power mean, realized as a generalized perspective.

```
noncomputable def operatorPowerMean (α β : ℝ) (A B : L ℋ) : L ℋ
```

lemma rpow_continuousOn_Ici [source]

```
private lemma rpow_continuousOn_Ici (p : ℝ) (hp : 0 ≤ p) :
    ContinuousOn (fun x : ℝ ↦ x ^ p) (Set.Ici (0 : ℝ))
```

lemma operatorConcaveOn_Ioi_of_Ici [source]

```
private lemma operatorConcaveOn_Ioi_of_Ici {f : ℝ → ℝ}
    (h : OperatorConcaveOn (ℋ := ℋ) (Set.Ici (0 : ℝ)) f) :
    OperatorConcaveOn (ℋ := ℋ) (Set.Ioi (0 : ℝ)) f
```

lemma pdSet_subset_psdSet [source]

```
private lemma pdSet_subset_psdSet : pdSet (ℋ := ℋ) ⊆ psdSet (ℋ := ℋ)
```

lemma rpow_pos_on_Ioi [source]

```
private lemma rpow_pos_on_Ioi (p : ℝ) :
    ∀ x ∈ Set.Ioi (0 : ℝ), 0 < x ^ p
```

theorem operatorPowerMean_jointlyConcaveOn_pdSet [source]

Theorem 1.1, concave range: the operator `(α, β)`-power mean is jointly concave on strictly positive operators for `0 ≤ α, β ≤ 1`.

```
theorem operatorPowerMean_jointlyConcaveOn_pdSet
    {α β : ℝ}
    (hα : α ∈ Set.Icc (0 : ℝ) 1)
```

```
    (hβ : β ∈ Set.Icc (0 : ℝ) 1) :
    JointlyConcaveOn (pdSet (ℋ := ℋ)) (pdSet (ℋ := ℋ))
      (operatorPowerMean (ℋ := ℋ) α β)
```

theorem operatorPowerMean_jointlyConvexOn_pdSet [source]

Theorem 1.1, convex range: the operator `(α, β)`-power mean is jointly convex on strictly positive operators for `1 ≤ α ≤ 2` and `0 ≤ β ≤ 1`.

```
theorem operatorPowerMean_jointlyConvexOn_pdSet
    {α β : ℝ}
    (hα : α ∈ Set.Icc (1 : ℝ) 2)
    (hβ : β ∈ Set.Icc (0 : ℝ) 1) :
    JointlyConvexOn (pdSet (ℋ := ℋ)) (pdSet (ℋ := ℋ))
      (operatorPowerMean (ℋ := ℋ) α β)
```

2.70 QuantumInfo.ForMathlib.HermitianMat.CFC

QuantumInfo/ForMathlib/HermitianMat/CFC.lean on GitHub.

theorem cfc_monoOn_pos_of_monoOn_posDef [source]

```
theorem cfc_monoOn_pos_of_monoOn_posDef {d : Type*} [Fintype d] [DecidableEq d]
  {f : ℝ → ℝ} (hf_is_operator_convex : False) :
    MonotoneOn (HermitianMat.cfc · f) { A : HermitianMat d ℂ | A.mat.PosDef }
```

2.71 QuantumInfo.ForMathlib.HermitianMat.CompoundMatrix

QuantumInfo/ForMathlib/HermitianMat/CompoundMatrix.lean on GitHub.

def compoundHermitian [source]

The `k`-th compound matrix of a Hermitian matrix, packaged as a `HermitianMat`.

```
noncomputable def compoundHermitian (A : HermitianMat d ℂ) (k : ℕ) :
    HermitianMat {S : Finset d // S.card = k} ℂ
```

lemma compoundHermitian_eigenvalues [source]

The eigenvalues of `compoundHermitian A k` are the products of eigenvalues of `A` over `k`-subsets, up to an index permutation.

```
lemma compoundHermitian_eigenvalues (A : HermitianMat d ℂ) (k : ℕ) :
    ∃ σ : {S : Finset d // S.card = k} ≃ {S : Finset d // S.card = k},
      (compoundHermitian A k).H.eigenvalues ∘ σ =
        fun S => ∏ i : Fin k, A.H.eigenvalues (S.1.orderEmbOfFin S.2 i)
```

lemma compoundHermitian_nonneg [source]

`compoundHermitian` preserves positive semidefiniteness.

```
lemma compoundHermitian_nonneg (A : HermitianMat d ℂ) (hA : 0 ≤ A) (k : ℕ) :
    0 ≤ compoundHermitian A k
```

lemma compoundHermitian_conj [source]

`compoundHermitian` distributes over `HermitianMat.conj`: the compound of `A.conj B` equals the conjugation of `compoundHermitian A k` by `compoundMatrix B k`.

```
lemma compoundHermitian_conj (A : HermitianMat d ℂ) (B : Matrix d d ℂ) (k : ℕ) :
    compoundHermitian (A.conj B) k =
      (compoundHermitian A k).conj (compoundMatrix B k)
```

2.72 QuantumInfo.ForMathlib.HermitianMat.Jordan

QuantumInfo/ForMathlib/HermitianMat/Jordan.lean on GitHub.

instance NonAssocCommRing (HermitianMat d 𝕜) [source]

```
scoped instance : NonAssocCommRing (HermitianMat d 𝕜) where
```

2.73 QuantumInfo.ForMathlib.HermitianMat.LiebConcavity

QuantumInfo/ForMathlib/HermitianMat/LiebConcavity.lean on GitHub.

abbrev Φ [source]

Abbreviation for the star algebra isomorphism.

```
noncomputable abbrev Φ : Matrix d d ℂ ≃⋆ₐ[ℂ] (EuclideanSpace ℂ d →L[ℂ] EuclideanSpace ℂ d)
```

lemma Φ_continuous [source]

`Φ` is continuous (as a linear map between finite-dimensional spaces).

```
lemma Φ_continuous : Continuous (⇑Φ : Matrix d d ℂ → _)
```

lemma Φ_isSelfAdjoint [source]

`Φ` maps Hermitian matrices to self-adjoint operators.

```
lemma Φ_isSelfAdjoint (A : HermitianMat d ℂ) :
    IsSelfAdjoint (Φ A.mat)
```

lemma Φ_nonneg [source]

```
lemma Φ_nonneg (A : HermitianMat d ℂ) (hA : 0 ≤ A) :
    (0 : EuclideanSpace ℂ d →L[ℂ] EuclideanSpace ℂ d) ≤ Φ A.mat
```

lemma Φ_mem_pdSet [source]

`Φ` maps PosDef HermitianMat to pdSet.

```
lemma Φ_mem_pdSet [Nonempty d] (A : HermitianMat d ℂ) (hA : A.mat.PosDef) :
    Φ A.mat ∈ pdSet (ℋ := EuclideanSpace ℂ d)
```

lemma Φ_cfc [source]

Φ commutes with CFC for Hermitian matrices.

```
lemma Φ_cfc (A : HermitianMat d ℂ) (f : ℝ → ℝ) :
    Φ (cfc f A.mat) = cfc f (Φ A.mat)
```

lemma Φ_rpow [source]

Φ commutes with rpow for PSD matrices.

```
lemma Φ_rpow (A : HermitianMat d ℂ) (hA : 0 ≤ A) (r : ℝ) :
    Φ (A ^ r).mat = (Φ A.mat) ^ r
```

lemma trace_Φ_eq [source]

General trace bridge: the operator trace of $\Phi(M)$ equals the matrix trace of M, for any matrix M (not just Hermitian).

```
lemma trace_Φ_eq (M : Matrix d d ℂ) :
    (LinearMap.trace ℂ (EuclideanSpace ℂ d)) (Φ M).toLinearMap = M.trace
```

lemma traceRe_Φ_general [source]

traceRe(Φ(M)) = re(Tr[M]) for any matrix M.

```
lemma traceRe_Φ_general (M : Matrix d d ℂ) :
    traceRe (Φ M) = Complex.re M.trace
```

lemma psd_convex [source]

The PSD cone is convex.

```
private lemma psd_convex : Convex ℝ {σ : HermitianMat d ℂ | 0 ≤ σ}
```

lemma trace_conj_rpow_continuous [source]

The trace of rpow applied to a congruence is continuous in the base matrix.

```
private lemma trace_conj_rpow_continuous {s p : ℝ} (hs : 0 ≤ s) (hp : 0 ≤ p)
    (H : HermitianMat d ℂ) :
    Continuous (fun σ : HermitianMat d ℂ ↦
      ((H.conj (σ ^ s).mat) ^ p).trace)
```

lemma psd_add_eps_posdef [source]

```
private lemma psd_add_eps_posdef [Nonempty d] (σ : HermitianMat d ℂ) (hσ : 0 ≤ σ)
    (ε : ℝ) (hε : 0 < ε) : (σ + ε • (1 : HermitianMat d ℂ)).mat.PosDef
```

lemma tendsto_add_eps [source]

$\sigma + \varepsilon I \to \sigma$ *as* $\varepsilon \to 0+$.

```
private lemma tendsto_add_eps (σ : HermitianMat d ℂ) :
    Filter.Tendsto (fun ε : ℝ ↦ σ + ε • (1 : HermitianMat d ℂ))
      (nhdsWithin 0 (Set.Ioi 0)) (nhds σ)
```

lemma trace_rpow_conjTranspose_mul_comm [source]

***AB/BA trace identity for rpow**: `Tr[(C^*C)^p] = Tr[(CC^*)^p]` for any square C.*

```
private lemma trace_rpow_conjTranspose_mul_comm [Nonempty d]
    (C : Matrix d d ℂ) (p : ℝ) :
    let M₁ : HermitianMat d ℂ
```

lemma variational_lower_bound [source]

```
private lemma variational_lower_bound
    (X Z : HermitianMat d ℂ) (hX : 0 ≤ X) (hZ : 0 ≤ Z)
    {p : ℝ} (hp : 1 < p) :
    p * ⟪X, Z ^ ((p-1)/p)⟫_ℝ - (p - 1) * Z.trace ≤ (X ^ p).trace
```

lemma variational_eq_optimizer [source]

```
private lemma variational_eq_optimizer
    (X : HermitianMat d ℂ) (hX : 0 ≤ X)
    {p : ℝ} (hp : 1 < p) :
    p * ⟪X, (X ^ p) ^ ((p-1)/p)⟫_ℝ - (p - 1) * (X ^ p).trace = (X ^ p).trace
```

lemma liebExtension_bridge [source]

```
private lemma liebExtension_bridge [Nonempty d]
    {q r : ℝ} (hq : 0 < q) (hr : 0 < r) (hqr : q + r ≤ 1)
    (K : HermitianMat d ℂ)
    (σ₁ σ₂ Z₁ Z₂ : HermitianMat d ℂ)
    (hσ₁ : σ₁.mat.PosDef) (hσ₂ : σ₂.mat.PosDef)
    (hZ₁ : Z₁.mat.PosDef) (hZ₂ : Z₂.mat.PosDef)
    (θ : ℝ) (hθ₀ : 0 ≤ θ) (hθ₁ : θ ≤ 1) :
    (1 - θ) * ⟪(σ₁ ^ q).conj K, Z₁ ^ r⟫_ℝ + θ * ⟪(σ₂ ^ q).conj K, Z₂ ^ r⟫_ℝ ≤
    ⟪(((1 - θ) • σ₁ + θ • σ₂) ^ q).conj K, ((1 - θ) • Z₁ + θ • Z₂) ^ r⟫_ℝ
```

lemma trace_conj_rpow_eq_conj_sqrt [source]

```
private lemma trace_conj_rpow_eq_conj_sqrt [Nonempty d]
    (σ H : HermitianMat d ℂ) (hσ : 0 ≤ σ) (hH : 0 ≤ H) (s p : ℝ) (hs : 0 < s) :
    ((H.conj (σ ^ s).mat) ^ p).trace =
    (((σ ^ (2 * s)).conj (H ^ (1/2 : ℝ)).mat) ^ p).trace
```

lemma liebExtension_bridge_psd [source]

```
private lemma liebExtension_bridge_psd [Nonempty d]
    {q r : ℝ} (hq : 0 < q) (hr : 0 < r) (hqr : q + r ≤ 1)
    (K σ₁ σ₂ Z₁ Z₂ : HermitianMat d ℂ)
    (hσ₁ : σ₁.mat.PosDef) (hσ₂ : σ₂.mat.PosDef)
    (hZ₁ : 0 ≤ Z₁) (hZ₂ : 0 ≤ Z₂)
```

```
    (θ : ℝ) (hθ₀ : 0 ≤ θ) (hθ₁ : θ ≤ 1) :
    (1 - θ) * ⟪(σ₁ ^ q).conj K, Z₁ ^ r⟫_ℝ + θ * ⟪(σ₂ ^ q).conj K, Z₂ ^ r⟫_ℝ ≤
    ⟪(((1 - θ) • σ₁ + θ • σ₂) ^ q).conj K, ((1 - θ) • Z₁ + θ • Z₂) ^ r⟫_ℝ
```

lemma trace_conj_rpow_concave_pd [source]

Core concavity inequality on positive definite matrices.

```
private lemma trace_conj_rpow_concave_pd [Nonempty d] {α : ℝ} (hα : 1 < α)
    (H : HermitianMat d ℂ) (hH : 0 ≤ H)
    (σ₁ σ₂ : HermitianMat d ℂ) (hσ₁ : σ₁.mat.PosDef) (hσ₂ : σ₂.mat.PosDef)
    (a b : ℝ) (ha : 0 ≤ a) (hb : 0 ≤ b) (hab : a + b = 1) :
    let s
```

theorem trace_conj_rpow_concave [source]

```
theorem trace_conj_rpow_concave {α : ℝ} (hα : 1 < α)
    (H : HermitianMat d ℂ) (hH : 0 ≤ H) :
    ConcaveOn ℝ {σ : HermitianMat d ℂ | 0 ≤ σ}
      (fun σ ↦ ((H.conj (σ ^ ((α - 1) / (2 * α))).mat) ^ (α / (α - 1))).trace)
```

2.74 QuantumInfo.ForMathlib.HermitianMat.Order

QuantumInfo/ForMathlib/HermitianMat/Order.lean on GitHub.

theorem kronecker_self_mono [source]

The self-Kronecker map `A ↦ A ⊗ₖ A` is monotone on nonnegative Hermitian matrices.

```
theorem kronecker_self_mono (hA : 0 ≤ A) (hB : 0 ≤ B) (hAB : A ≤ B) :
    A ⊗ₖ A ≤ B ⊗ₖ B
```

2.75 QuantumInfo.ForMathlib.HermitianMat.Peierls

QuantumInfo/ForMathlib/HermitianMat/Peierls.lean on GitHub.

lemma trace_cfc_conj_unitary [source]

The trace of cfc(g, A) is invariant under unitary conjugation of A. Follows from `cfc_conj_unitary` and `trace_conj_unitary`.

```
lemma trace_cfc_conj_unitary (A : HermitianMat d ℂ) (g : ℝ → ℝ) (U : 𝐔[d]) :
    ((A.conj U.val).cfc g).trace = (A.cfc g).trace
```

theorem peierls_inequality [source]

Peierls inequality: for a convex function g, the sum of g applied to the diagonal entries of a Hermitian matrix is at most the trace of g(A). This follows from Jensen's inequality applied to the spectral decomposition.

```
theorem peierls_inequality (A : HermitianMat d ℂ) (g : ℝ → ℝ) (hg : ConvexOn ℝ Set.univ g) :
    ∑ i, g ((A.mat i i).re) ≤ (A.cfc g).trace
```

theorem peierls_inequality_ici [source]

```
theorem peierls_inequality_ici (A : HermitianMat d ℂ) (g : ℝ → ℝ) (hg : ConvexOn ℝ (Set.Ici 0)
    g)
  (hA : 0 ≤ A) :
    ∑ i, g ((A.mat i i).re) ≤ (A.cfc g).trace
```

theorem trace_function_convex_univ [source]

Joint convexity of the trace functional: for a convex function g, the map $A \mapsto tr(g(A))$ is convex on the space of Hermitian matrices.

```
theorem trace_function_convex_univ (g : ℝ → ℝ) (hg : ConvexOn ℝ Set.univ g) :
    ConvexOn ℝ Set.univ (fun A : HermitianMat d ℂ => (A.cfc g).trace)
```

theorem trace_function_convex_ici [source]

Convexity of trace functions: if g is convex on $\mathbb{R}_+$, then $A \mapsto Tr[g(A)]$ is convex on PSD matrices.

```
theorem trace_function_convex_ici {g : ℝ → ℝ} (hg : ConvexOn ℝ (Set.Ici 0) g) :
    ConvexOn ℝ {A : HermitianMat d ℂ | 0 ≤ A} (fun A => (A.cfc g).trace)
```

2.76 QuantumInfo.ForMathlib.HermitianMat.Proj

QuantumInfo/ForMathlib/HermitianMat/Proj.lean on GitHub.

theorem projector_nonneg [source]

```
theorem projector_nonneg : 0 ≤ projector S
```

theorem projector_ker [source]

```
@[simp]
theorem projector_ker : (projector S).ker = S^⊥
```

theorem supportProj_ker [source]

```
@[simp]
theorem supportProj_ker : A.supportProj.ker = A.ker
```

theorem kerProj_ker [source]

```
@[simp]
theorem kerProj_ker : A.kerProj.ker = A.support
```

theorem posPart_eq_zero_iff [source]

```
theorem posPart_eq_zero_iff : A⁺ = 0 ↔ A ≤ 0
```

2.77 QuantumInfo.ForMathlib.HermitianMat.Rpow

QuantumInfo/ForMathlib/HermitianMat/Rpow.lean on GitHub.

lemma compoundHermitian_rpow [source]

```
private lemma compoundHermitian_rpow (A : HermitianMat d ℂ) (hA : 0 ≤ A)
    (k : ℕ) (r : ℝ) :
    compoundHermitian (A ^ r) k = (compoundHermitian A k) ^ r
```

lemma conj_rpow_le_one_of_conj_le_one_posDef [source]

```
private lemma conj_rpow_le_one_of_conj_le_one_posDef
    {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : B.mat.PosDef)
    {r : ℝ} (hr0 : 0 < r) (hr1 : r ≤ 1)
    (hAB : A.conj B.mat ≤ 1) :
    (A ^ r).conj (B ^ r).mat ≤ 1
```

lemma conj_smul_right [source]

```
private lemma conj_smul_right (A B : HermitianMat d ℂ) (c : ℝ) :
    A.conj (↑(c • B).mat) = c ^ 2 • A.conj B.mat
```

lemma conjTranspose_half_mul_eq_conj [source]

```
private lemma conjTranspose_half_mul_eq_conj
    {A B : HermitianMat d ℂ} (hA : 0 ≤ A) :
    ((A ^ (1/2 : ℝ)).mat * B.mat).conjTranspose * ((A ^ (1/2 : ℝ)).mat * B.mat)
      = (A.conj B.mat).mat
```

lemma top_singular_le_of_self_mul_le_smul_one [source]

```
private lemma top_singular_le_of_self_mul_le_smul_one
    {e : Type*} [Fintype e] [DecidableEq e] (X : Matrix e e ℂ)
    {α : ℝ} (_ : 0 ≤ α) (hX : X.conjTranspose * X ≤ α • (1 : Matrix e e ℂ))
    (hcard : 0 < Fintype.card e) :
    singularValuesSorted X ⟨0, hcard⟩ ≤ Real.sqrt α
```

lemma compound_top_singular_le_posDef [source]

```
private lemma compound_top_singular_le_posDef
    {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : B.mat.PosDef)
    {r : ℝ} (hr0 : 0 < r) (hr1 : r ≤ 1)
    (k : ℕ) (hk : k ≤ Fintype.card d) :
    singularValuesSorted (compoundMatrix ((A ^ (r / 2 : ℝ)).mat * (B ^ r).mat) k) (compoundZero
     k hk) ≤
      singularValuesSorted (compoundMatrix ((A ^ (1 / 2 : ℝ)).mat * B.mat) k) (compoundZero k
    hk) ^ r
```

lemma trace_conj_rpow_eq_sum_singularValuesSorted [source]

Expresses a conjugated trace power as a sum of singular values.

This is private infrastructure for the Lieb-Thirring inequality.

```
private lemma trace_conj_rpow_eq_sum_singularValuesSorted
    {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (α : ℝ) :
    ((A.conj B.mat) ^ α).trace =
```

```
    ∑ i : Fin (Fintype.card d),
      singularValuesSorted ((A ^ (1 / 2 : ℝ)).mat * B.mat) i ^ (2 * α)
```

lemma lieb_thirring_le_one_posDef [source]

```
private lemma lieb_thirring_le_one_posDef
    {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : B.mat.PosDef)
    {r : ℝ} (hr0 : 0 < r) (hr1 : r ≤ 1) :
    ((A ^ r).conj (B ^ r).mat).trace ≤ ((A.conj B.mat) ^ r).trace
```

lemma trace_conj_mono_sqrt [source]

```
private lemma trace_conj_mono_sqrt {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : 0 ≤ B)
    {t : ℝ} (ht : 0 < t) :
    (((A + B + t • 1)⁻¹).conj A.sqrt.mat).trace ≤ (((A + t • 1)⁻¹).conj A.sqrt.mat).trace
```

lemma trace_conj_sqrt_eq_trace_mul_left [source]

```
private lemma trace_conj_sqrt_eq_trace_mul_left {A X : HermitianMat d ℂ} (hA : 0 ≤ A) :
    (((X).conj A.sqrt.mat).trace : ℂ) = (A.mat * X.mat).trace
```

lemma scalarRpowApprox_eq_mul_integral_sub_integral [source]

```
private lemma scalarRpowApprox_eq_mul_integral_sub_integral {x q T : ℝ}
    (hx : 0 < x) (hq : 0 < q) (hT : 0 < T) :
    scalarRpowApprox q T x =
      x * (∫ t in (0)..T, t ^ (q - 1) / (x + t)) -
        (∫ t in (0)..T, t ^ (q - 1) / (1 + t))
```

lemma tendsto_mul_intervalIntegral_rpow_div_add [source]

```
private lemma tendsto_mul_intervalIntegral_rpow_div_add {x q : ℝ} (hx : 0 < x)
    (hq : 0 < q) (hq1 : q < 1) :
    Filter.Tendsto
      (fun T => x * (∫ t in (0)..T, t ^ (q - 1) / (x + t)))
      Filter.atTop
      (nhds (rpowConst q * x ^ q))
```

lemma intervalIntegrable_weighted_div_nonneg [source]

```
private lemma intervalIntegrable_weighted_div_nonneg {x p T : ℝ}
    (hx : 0 ≤ x) (hp : 0 < p) (hT : 0 < T) :
    IntervalIntegrable (fun t => t ^ (p - 1) * (x / (x + t))) volume 0 T
```

lemma tendsto_intervalIntegral_weighted_div_nonneg [source]

```
private lemma tendsto_intervalIntegral_weighted_div_nonneg {x p : ℝ}
    (hx : 0 ≤ x) (hp : 0 < p) (hp1 : p < 1) :
    Filter.Tendsto
      (fun T => ∫ t in (0)..T, t ^ (p - 1) * (x / (x + t)))
      Filter.atTop
      (nhds (rpowConst p * x ^ p))
```

lemma trace_mul_inv_shift_add_le [source]

```
private lemma trace_mul_inv_shift_add_le {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : 0 ≤ B)
    {t : ℝ} (ht : 0 < t) :
    (((A + B).mat * (A + B + t • 1).mat⁻¹).trace : ℂ) ≤
      ((A.mat * (A + t • 1).mat⁻¹).trace : ℂ) + ((B.mat * (B + t • 1).mat⁻¹).trace : ℂ)
```

lemma trace_mul_inv_shift_eq_sum_div [source]

```
private lemma trace_mul_inv_shift_eq_sum_div {A : HermitianMat d ℂ} (hA : 0 ≤ A) {t : ℝ}
    (ht : 0 < t) :
    (((A.mat * (A + t • 1).mat⁻¹).trace : ℂ)).re =
      ∑ i, A.H.eigenvalues i / (A.H.eigenvalues i + t)
```

lemma intervalIntegrable_weighted_trace_mul_inv_shift [source]

```
private lemma intervalIntegrable_weighted_trace_mul_inv_shift {A : HermitianMat d ℂ}
    (hA : 0 ≤ A) {p T : ℝ} (hp : 0 < p) (hT : 0 < T) :
    IntervalIntegrable
      (fun t => t ^ (p - 1) * (((A.mat * (A + t • 1).mat⁻¹).trace : ℂ)).re)
      volume 0 T
```

lemma tendsto_intervalIntegral_weighted_trace_mul_inv_shift [source]

```
private lemma tendsto_intervalIntegral_weighted_trace_mul_inv_shift {A : HermitianMat d ℂ}
    (hA : 0 ≤ A) (p : ℝ) (hp : 0 < p) (hp1 : p < 1) :
    Filter.Tendsto
      (fun T => ∫ t in (0)..T, t ^ (p - 1) * (((A.mat * (A + t • 1).mat⁻¹).trace : ℂ)).re)
      Filter.atTop
      (nhds (rpowConst p * (A ^ p).trace))
```

2.78 QuantumInfo.ForMathlib.Majorization

QuantumInfo/ForMathlib/Majorization.lean on GitHub.

lemma compoundMatrix_unitary [source]

The compound matrix of a unitary matrix is unitary.

```
lemma compoundMatrix_unitary (U : Matrix d d ℂ)
    (hU : U ∈ Matrix.unitaryGroup d ℂ) (k : ℕ) :
    compoundMatrix U k ∈ Matrix.unitaryGroup {S : Finset d // S.card = k} ℂ
```

def compoundUnitary [source]

The `k`-th compound matrix bundled as a unitary.

```
noncomputable def compoundUnitary (U : Matrix.unitaryGroup d ℂ) (k : ℕ) :
    Matrix.unitaryGroup {S : Finset d // S.card = k} ℂ
```

lemma compound_card_pos [source]

The index set of the `k`-th compound matrix is nonempty when `k ≤ card d`.

```
lemma compound_card_pos (k : ℕ) (hk : k ≤ Fintype.card d) :
    0 < Fintype.card {S : Finset d // S.card = k}
```

def compoundZero [source]

The canonical zero index in `Fin (Fintype.card {S : Finset d // S.card = k})`, witnessed by `compound_card_pos`.

```
def compoundZero (k : ℕ) (hk : k ≤ Fintype.card d) :
    Fin (Fintype.card {S : Finset d // S.card = k})
```

2.79 QuantumInfo.ForMathlib.Matrix

QuantumInfo/ForMathlib/Matrix.lean on GitHub.

theorem fromBlocks_gram_posSemidef [source]

The block Gram matrix `[[Y^H Y, Y^H X], [X^H Y, X^H X]]` is positive semidefinite.

```
theorem fromBlocks_gram_posSemidef {m n k : Type*} [Fintype m] [Fintype n] [Fintype k]
    (X : Matrix k n ℂ) (Y : Matrix k m ℂ) :
    (fromBlocks (Y^H * Y) (Y^H * X) (X^H * Y) (X^H * X)).PosSemidef
```

2.80 QuantumInfo.ForMathlib.MatrixNorm.TraceNorm

QuantumInfo/ForMathlib/MatrixNorm/TraceNorm.lean on GitHub.

theorem traceNorm_neg [source]

The trace norm of the negative is equal to the trace norm.

```
@[simp]
theorem traceNorm_neg (A : Matrix m n R) : traceNorm (-A) = traceNorm A
```

lemma inner_A_mulVec_eq [source]

```
private lemma inner_A_mulVec_eq (A : Matrix n n ℂ) (v w : n → ℂ) :
    inner ℂ (WithLp.toLp 2 (A.mulVec v)) (WithLp.toLp 2 (A.mulVec w)) =
      star v ⬝ᵥ ((A^H * A).mulVec w)
```

theorem exists_svd_sqrt_eigenvalues [source]

Singular value decomposition for square complex matrices, with singular values expressed as square roots of the eigenvalues of `A^H * A`.

```
theorem exists_svd_sqrt_eigenvalues (A : Matrix n n ℂ) :
    let hH : (A^H * A).IsHermitian
```

lemma traceNorm_eq_sum_sqrt_eigenvalues [source]

```
private lemma traceNorm_eq_sum_sqrt_eigenvalues (A : Matrix n n ℂ) :
    let hH : (A^H * A).IsHermitian
```

theorem traceNorm_eq_sum_singularValues [source]

The trace norm of a square complex matrix is the sum of its singular values.

```
theorem traceNorm_eq_sum_singularValues [DecidableEq n] (A : Matrix n n ℂ) :
    A.traceNorm = ∑ i, singularValues A i
```

theorem traceNorm_eq_sum_singularValuesSorted [source]

The trace norm of a square complex matrix is the sum of its sorted singular values.

```
theorem traceNorm_eq_sum_singularValuesSorted [DecidableEq n] (A : Matrix n n ℂ) :
    A.traceNorm = ∑ i : Fin (Fintype.card n), singularValuesSorted A i
```

theorem singularValues_le_opNorm [source]

Every singular value is bounded by the operator norm.

```
theorem singularValues_le_opNorm [DecidableEq n] (A : Matrix n n ℂ) (i : n) :
    singularValues A i ≤ ‖A‖
```

theorem traceNorm_mul_le_opNorm_traceNorm [source]

The trace norm is bounded by the operator norm on the left times the trace norm on the right.

```
theorem traceNorm_mul_le_opNorm_traceNorm [DecidableEq n] (A B : Matrix n n ℂ) :
    (A * B).traceNorm ≤ ‖A‖ * B.traceNorm
```

theorem traceNorm_conjTranspose [source]

The trace norm is invariant under conjugate transpose.

```
theorem traceNorm_conjTranspose (A : Matrix n n ℂ) :
    A^H.traceNorm = A.traceNorm
```

theorem traceNorm_mul_le_traceNorm_opNorm [source]

The trace norm is bounded by trace norm on the left times operator norm on the right.

```
theorem traceNorm_mul_le_traceNorm_opNorm [DecidableEq n] (A B : Matrix n n ℂ) :
    (A * B).traceNorm ≤ A.traceNorm * ‖B‖
```

theorem traceNorm_sandwich_le [source]

Multiplication on both sides by contractions does not increase trace norm.

```
theorem traceNorm_sandwich_le [DecidableEq n] {S M T : Matrix n n ℂ} (hS : ‖S‖ ≤ 1)
    (hT : ‖T‖ ≤ 1) : (S * M * T).traceNorm ≤ M.traceNorm
```

theorem abs_trace_le_traceNorm [source]

The absolute value of the trace is bounded by the trace norm.

```
theorem abs_trace_le_traceNorm (A : Matrix n n ℂ) :
    ‖A.trace‖ ≤ A.traceNorm
```

theorem traceNorm_eq_max_re_tr_U [source]

For square complex matrices, the trace norm is the maximum of `re (Tr[U * A])` over unitaries `U`.

```
theorem traceNorm_eq_max_re_tr_U (A : Matrix n n ℂ) :
    IsGreatest {x : ℝ | ∃ U : unitaryGroup n ℂ, Complex.re ((U.val * A).trace) = x} A.traceNorm
```

theorem traceNorm_add_le [source]

The trace norm satisfies the triangle inequality for square complex matrices.

```
theorem traceNorm_add_le (A B : Matrix n n ℂ) : (A + B).traceNorm ≤ A.traceNorm + B.traceNorm
```

theorem traceNorm_eq_trace [source]

A positive semidefinite matrix has trace norm equal to its trace.

```
theorem PosSemidef.traceNorm_eq_trace {A : Matrix m m R} (hA : A.PosSemidef) :
    A.traceNorm = A.trace
```

2.81 QuantumInfo.Measurements.POVM

QuantumInfo/Measurements/POVM.lean on GitHub.

theorem measureForget_eq_kraus [source]

```
theorem measureForget_eq_kraus (Λ : POVM X d) :
    Λ.measureForget = CPTPMap.of_kraus_CPTPMap (fun i ↦ (Λ.mats i) ^ (1/2 : ℝ)) (by
      simpa [-one_div, fun x ↦ HermitianMat.pow_half_mul (Λ.nonneg x), HermitianMat.ext_iff]
        using Λ.normalized
    )
```

2.82 QuantumInfo.Regularized

QuantumInfo/Regularized.lean on GitHub.

def realNegOrderIso [source]

```
private def realNegOrderIso : ℝ ≃o ℝ^d where
```

theorem limsup_eq_neg_liminf_neg [source]

```
private theorem limsup_eq_neg_liminf_neg {fn : ℕ → ℝ} {_lb _ub : ℝ}Expand commentComment on
    line R131Resolved
    (hl : ∀ n, _lb ≤ fn n) (hu : ∀ n, fn n ≤ _ub) :
    Filter.atTop.limsup fn = -Filter.atTop.liminf (fun n => -fn n)
```

2.83 QuantumInfo.States.Mixed.Fidelity

QuantumInfo/States/Mixed/Fidelity.lean on GitHub.

theorem fidelity_eq_traceNorm_sqrt_mul_sqrt [source]

Fidelity can be rewritten as the trace norm of the product of square roots.

```
theorem fidelity_eq_traceNorm_sqrt_mul_sqrt (ρ σ : MState d) :
    fidelity ρ σ = (σ.M.sqrt.mat * ρ.M.sqrt.mat).traceNorm
```

2.84 QuantumInfo.States.Mixed.MState

QuantumInfo/States/Mixed/MState.lean on GitHub.

theorem pure_inner [source]

```
theorem pure_inner : ⟪pure ψ, pure φ⟫_Prob = ‖Braket.dot ψ φ‖^2
```

2.85 QuantumInfo.States.Pure.BargmannInvariant

QuantumInfo/States/Pure/BargmannInvariant.lean on GitHub.

def bargmannInvariantThree [source]

The three-vertex Bargmann invariant: ⟨ψ₁|ψ₂⟩ · ⟨ψ₂|ψ₃⟩ · ⟨ψ₃|ψ₁⟩. This is a gauge-invariant complex number whose argument is the geometric phase of the geodesic triangle.

```
def bargmannInvariantThree (ψ₁ ψ₂ ψ₃ : Ket d) : ℂ
```

def bargmannPhaseThree [source]

The geometric (Pancharatnam-Berry) phase of three states.

```
def bargmannPhaseThree (ψ₁ ψ₂ ψ₃ : Ket d) : ℝ
```

lemma bargmannInvariantThree_degenerate [source]

The Bargmann invariant of three identical states is 1.

```
@[simp]
lemma bargmannInvariantThree_degenerate (ψ : Ket d) :
    bargmannInvariantThree ψ ψ ψ = 1
```

lemma bargmannPhaseThree_degenerate [source]

The geometric phase of three identical states is 0.

```
lemma bargmannPhaseThree_degenerate (ψ : Ket d) :
    bargmannPhaseThree ψ ψ ψ = 0
```

lemma bargmannInvariantThree_reverse [source]

Reversing the cyclic order conjugates the invariant.

```
lemma bargmannInvariantThree_reverse (ψ₁ ψ₂ ψ₃ : Ket d) :
    bargmannInvariantThree ψ₃ ψ₂ ψ₁ = starRingEnd ℂ (bargmannInvariantThree ψ₁ ψ₂ ψ₃)
```

lemma bargmannPhaseThree_reverse [source]

Reversing the cyclic order negates the geometric phase (mod 2π).

```
lemma bargmannPhaseThree_reverse (ψ₁ ψ₂ ψ₃ : Ket d) :
    (bargmannPhaseThree ψ₃ ψ₂ ψ₁ : Real.Angle) =
    -(bargmannPhaseThree ψ₁ ψ₂ ψ₃ : Real.Angle)
```

lemma bargmannInvariantThree_cyclic [source]

Cyclic permutation of the three states preserves the Bargmann invariant.

```
lemma bargmannInvariantThree_cyclic (ψ₁ ψ₂ ψ₃ : Ket d) :
    bargmannInvariantThree ψ₂ ψ₃ ψ₁ = bargmannInvariantThree ψ₁ ψ₂ ψ₃
```

lemma bargmannPhaseThree_cyclic [source]

Cyclic permutation preserves the geometric phase.

```
lemma bargmannPhaseThree_cyclic (ψ₁ ψ₂ ψ₃ : Ket d) :
    bargmannPhaseThree ψ₂ ψ₃ ψ₁ = bargmannPhaseThree ψ₁ ψ₂ ψ₃
```

lemma norm_bargmannInvariantThree_le_one [source]

The Bargmann invariant has norm at most 1: each inner product factor is bounded by Cauchy-Schwarz on unit vectors.

```
lemma norm_bargmannInvariantThree_le_one (ψ₁ ψ₂ ψ₃ : Ket d) :
    ‖bargmannInvariantThree ψ₁ ψ₂ ψ₃‖ ≤ 1
```

2.86 QuantumInfo.States.Pure.BlochSphere

QuantumInfo/States/Pure/BlochSphere.lean on GitHub.

abbrev BlochSphere [source]

The Bloch sphere: unit sphere in `EuclideanSpace ℝ (Fin 3)`.

```
abbrev BlochSphere
```

def blochVecRaw [source]

The raw Bloch vector for angles (α, θ). Internal helper for `blochPoint`.

```
private def blochVecRaw (α θ : ℝ) : Fin 3 → ℝ
```

lemma blochVecRaw_norm [source]

The Bloch vector has unit norm: $sin^2\alpha(cos^2\theta + sin^2\theta) + cos^2\alpha = 1$.

```
private lemma blochVecRaw_norm (α θ : ℝ) :
    Real.sqrt ((blochVecRaw α θ 0) ^ 2 + (blochVecRaw α θ 1) ^ 2 +
      (blochVecRaw α θ 2) ^ 2) = 1
```

def blochPoint [source]

A point on the Bloch sphere parameterized by polar angle `α` and azimuthal angle `θ`.

```
def blochPoint (α θ : ℝ) : BlochSphere
```

lemma blochPoint_val [source]

The underlying vector of a `blochPoint`.

```
lemma blochPoint_val (α θ : ℝ) :
    (blochPoint α θ : EuclideanSpace ℝ (Fin 3)) =
    (WithLp.equiv 2 _).symm (blochVecRaw α θ)
```

def solidAngle [source]

Solid angle of a geodesic triangle on the Bloch sphere, computed via the Van Vleck formula using `Complex.arg` for full-quadrant support.

`Ω = 2 · arg(den + num · i)` where `den = 1 + n₁·n₂ + n₂·n₃ + n₃·n₁` and `num = n₁ · (n₂ × n₃)`.

```
def solidAngle (p₁ p₂ p₃ : BlochSphere) : ℝ
```

lemma dot_blochPoint [source]

The dot product of two Bloch points expressed in angle differences.

```
lemma dot_blochPoint (α₁ θ₁ α₂ θ₂ : ℝ) :
    (blochPoint α₁ θ₁ : EuclideanSpace ℝ (Fin 3)) ⬝ᵥ
    (blochPoint α₂ θ₂ : EuclideanSpace ℝ (Fin 3)) =
    Real.sin α₁ * Real.sin α₂ * Real.cos (θ₂ - θ₁) +
    Real.cos α₁ * Real.cos α₂
```

2.87 QuantumInfo.States.Pure.Braket

QuantumInfo/States/Pure/Braket.lean on GitHub.

lemma dot_swap_conj [source]

Swapping the arguments conjugates the bra-ket product: `⟨ψ₂|ψ₁⟩ = conj(⟨ψ₁|ψ₂⟩)`.

```
lemma Braket.dot_swap_conj (ψ₁ ψ₂ : Ket d) :
    ⟨ψ₂‖ψ₁⟩ = starRingEnd ℂ ⟨ψ₁‖ψ₂⟩
```

def ketToEuclidean [source]

```
private def ketToEuclidean (ψ : Ket d) : EuclideanSpace ℂ d
```

lemma ketToEuclidean_norm [source]

```
private lemma ketToEuclidean_norm (ψ : Ket d) : ‖ketToEuclidean ψ‖ = 1
```

lemma dot_eq_euclidean_inner [source]

```
private lemma dot_eq_euclidean_inner (ψ₁ ψ₂ : Ket d) :
    ⟨ψ₁‖ψ₂⟩ = @inner ℂ (EuclideanSpace ℂ d) _
      (ketToEuclidean ψ₁) (ketToEuclidean ψ₂)
```

lemma norm_dot_le_one [source]

The bra-ket product of normalized states has norm at most 1 (Cauchy-Schwarz).

```
lemma Braket.norm_dot_le_one (ψ₁ ψ₂ : Ket d) : ‖⟨ψ₁‖ψ₂⟩‖ ≤ 1
```